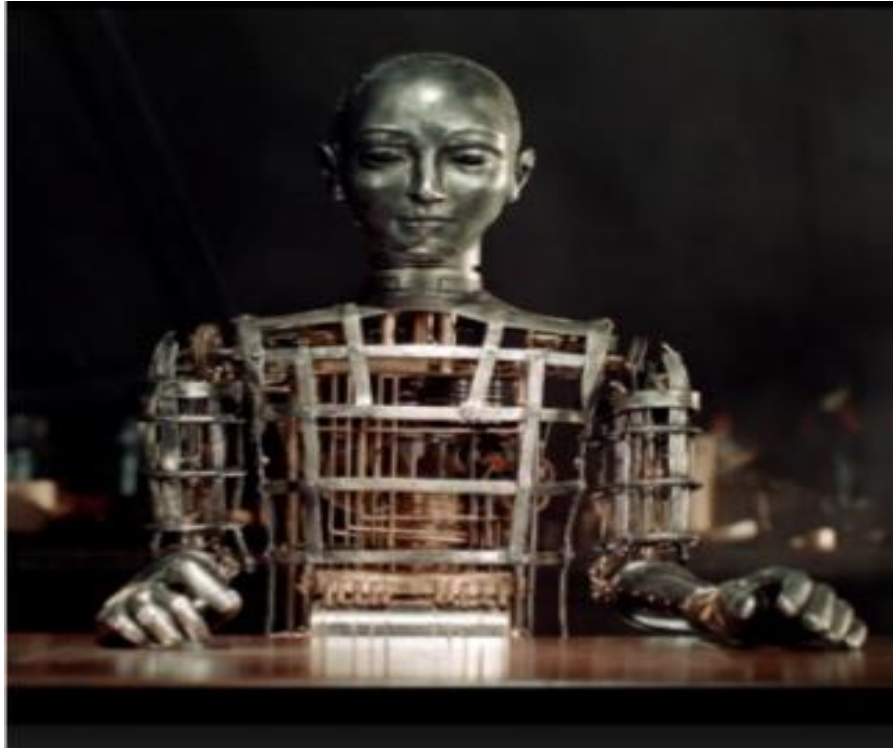


BCSC0011 : THEORY OF AUTOMATA & FORMAL LANGUAGES (TAFL)



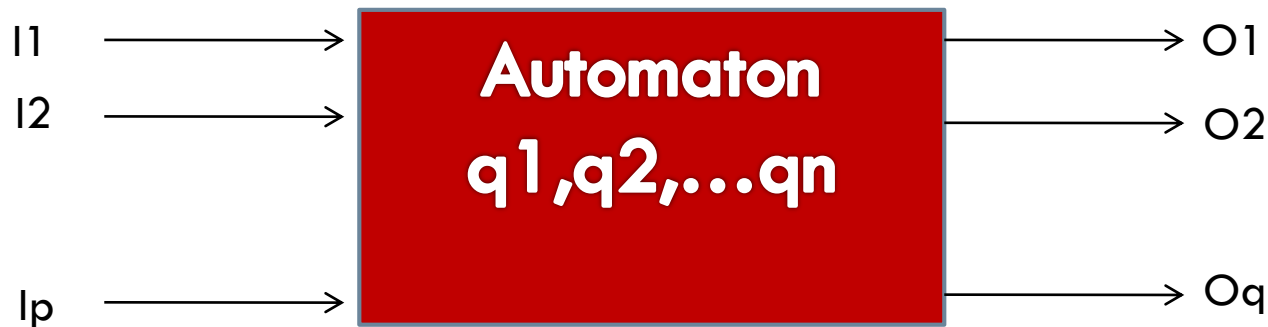
Dr. Sandeep Rathor

What is Automata?

- It is the plural of automaton, it means “something that works automatically”
- **A system where energy, materials and information, are transformed for performing some specific task, without direct participation of man.**
- **Example:** Automatic photo printing machine, Packing machine, etc.

Model of Discrete Automaton

- In Computer Science, **Automaton** = an abstract computing device which process discrete information.



Input: $I_1, I_2, \dots, I_p$

Output: $O_1, O_2, \dots, O_q$

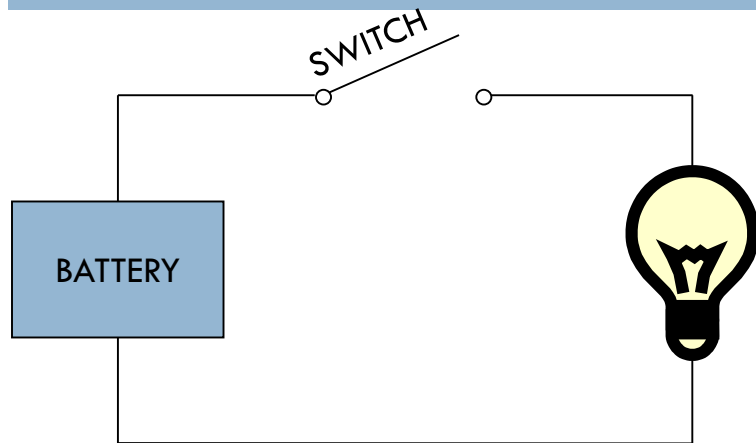
States: $q_1, q_2, \dots, q_n$

By: Sandeep Rathor

Why do we need abstract models?

- ▢ Abstract model is **free from “Programming Language”**
- ▢ It's **easy to manipulate** these theoretical machines **mathematically** to prove things about their capabilities.

A simple computer



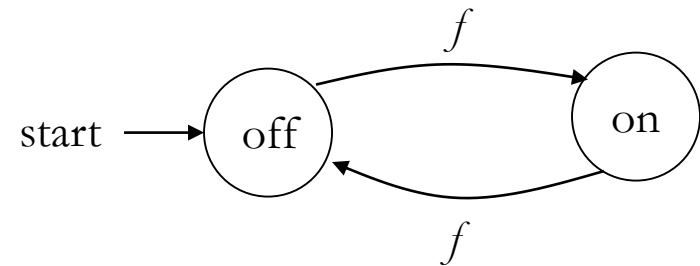
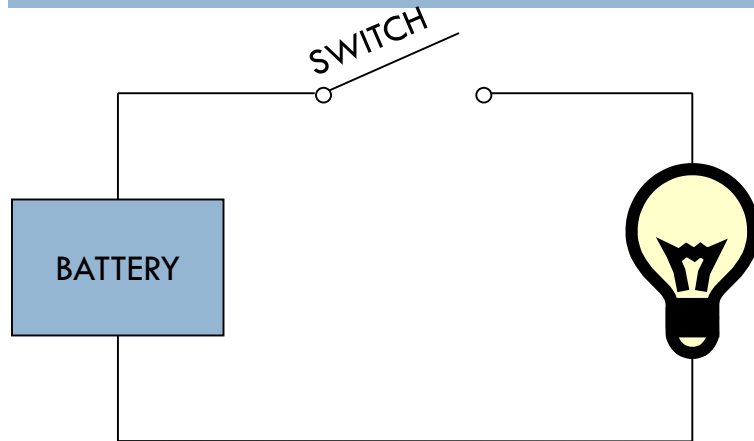
input: switch

output: light bulb

actions: flip switch

states: on, off

A simple “computer”



input: switch

output: light bulb

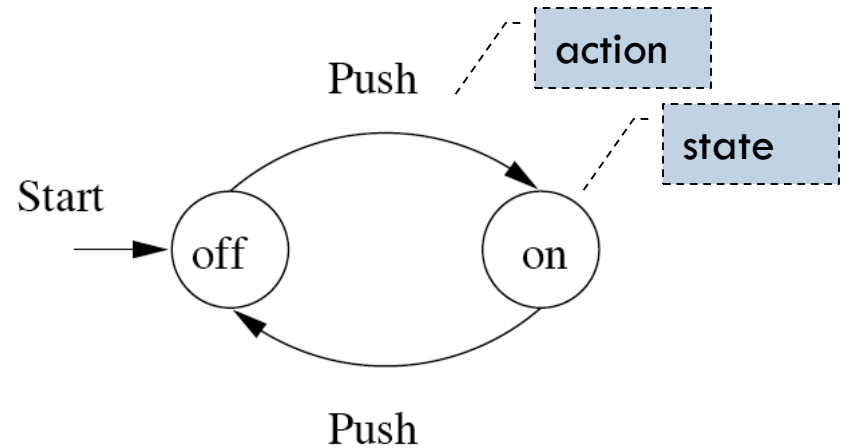
actions: f for “flip switch”

states: on, off

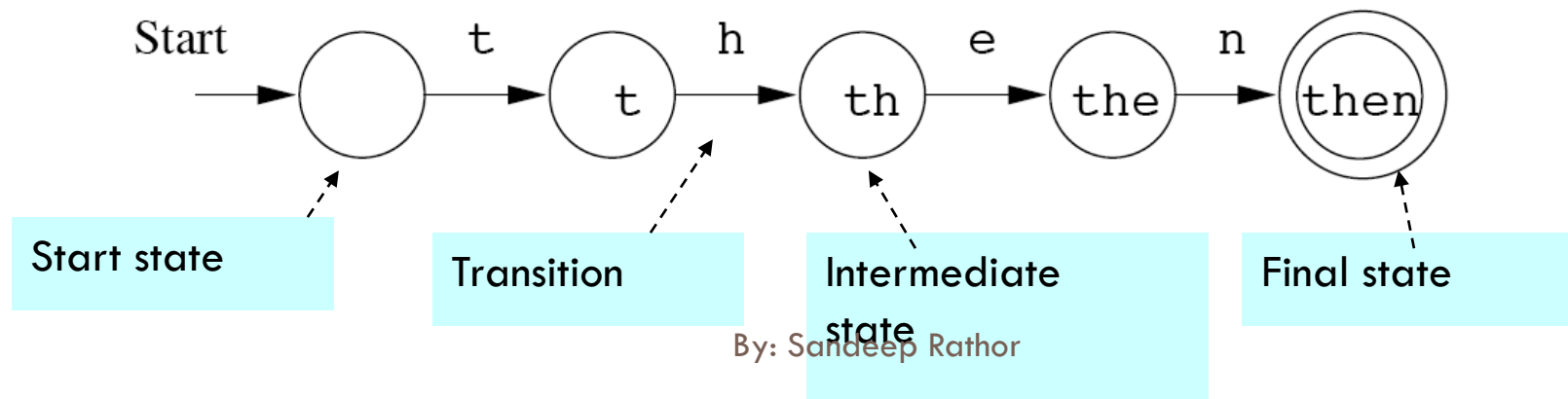
bulb is on if and only if
there was an **odd** number
of flips

Finite Automata : Examples

□ On/Off switch



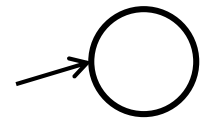
□ Modeling recognition of the word “*then*”



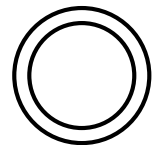
Transition Diagram

- Directed graph consists of set of vertices and edges where vertices represent “states” and edges represent “input/output”

- Circle with an arrow is called initial state



- Two concentric circle represents the final state



Alphabets

⌘ A finite non empty set of symbols.

Symbol : Σ

1. English lang. small letter $\Sigma = \{a, b, c \dots z\}$
2. Binary number $\Sigma = \{0, 1\}$
3. Decimal number $\Sigma = \{0, 1, 2, \dots, 9\}$
4. Alphanumeric : $\Sigma = \{A-Z, a-z, 0-9\}$

Strings

- ⌘ A word or a string is a finite sequence of symbols taken from Σ .
- ⌘ Empty string : ϵ
- ⌘ Length of string w is denoted by $|w|$ is number of non empty characters in the string.

Ex. $x = 01000 \quad |x| = 5$

$x = 01\epsilon 01\epsilon 00\epsilon \quad |x| = ?$

- ⌘ $xy =$ concatenation of two string.

Strings

A **string** over alphabet Σ is a finite sequence of symbols in Σ .

- The **empty string** will be denoted by ε
- Examples

abfbz is a string over $\Sigma_1 = \{a, b, c, d, \dots, z\}$

9021 is a string over $\Sigma_2 = \{0, 1, \dots, 9\}$

ab#bc is a string over $\Sigma_3 = \{a, b, \dots, z, \#\}$

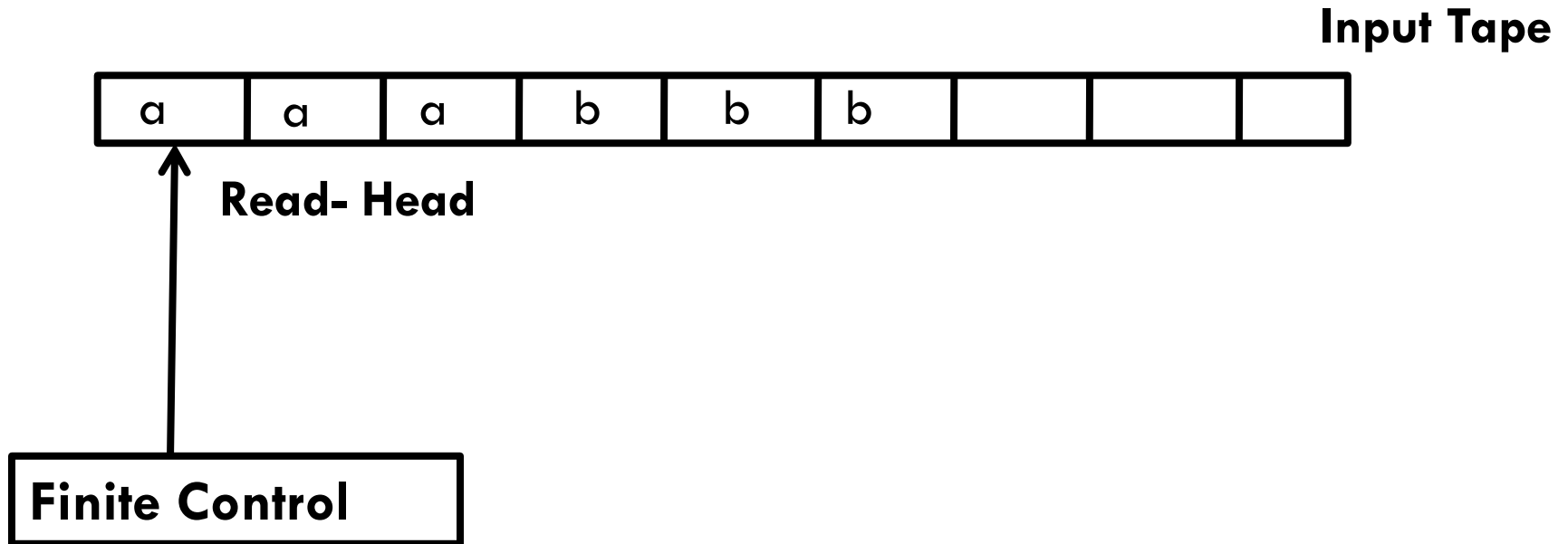
))()(is a string over $\Sigma_4 = \{ (,) \}$

Languages

⌘ L is said to be a language over alphabet Σ only if $L \subseteq \Sigma^*$.

⌘ Σ^* Set of all string over the Σ

Deterministic Finite Automata (DFA)



Model Diagram of DFA

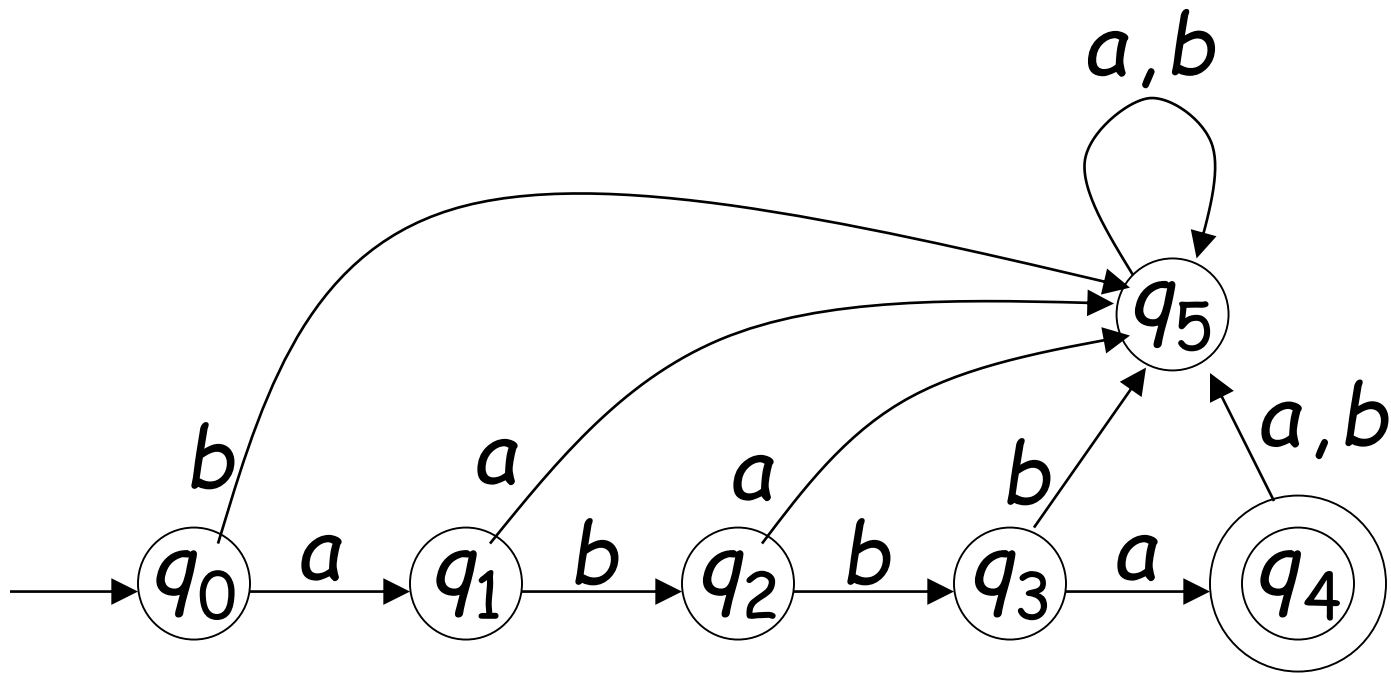
Deterministic Finite Automata (DFA)

- A **deterministic finite automaton** (DFA) is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where
 - Q is a finite set of **states**
 - Σ is an input **alphabet**
 - $\delta: Q \times \Sigma \rightarrow Q$ is a **transition function**
 - $q_0 \in Q$ is the **initial state**
 - $F \subseteq Q$ is a set of **accepting states** (or **final states**).

Input Alphabet

Σ

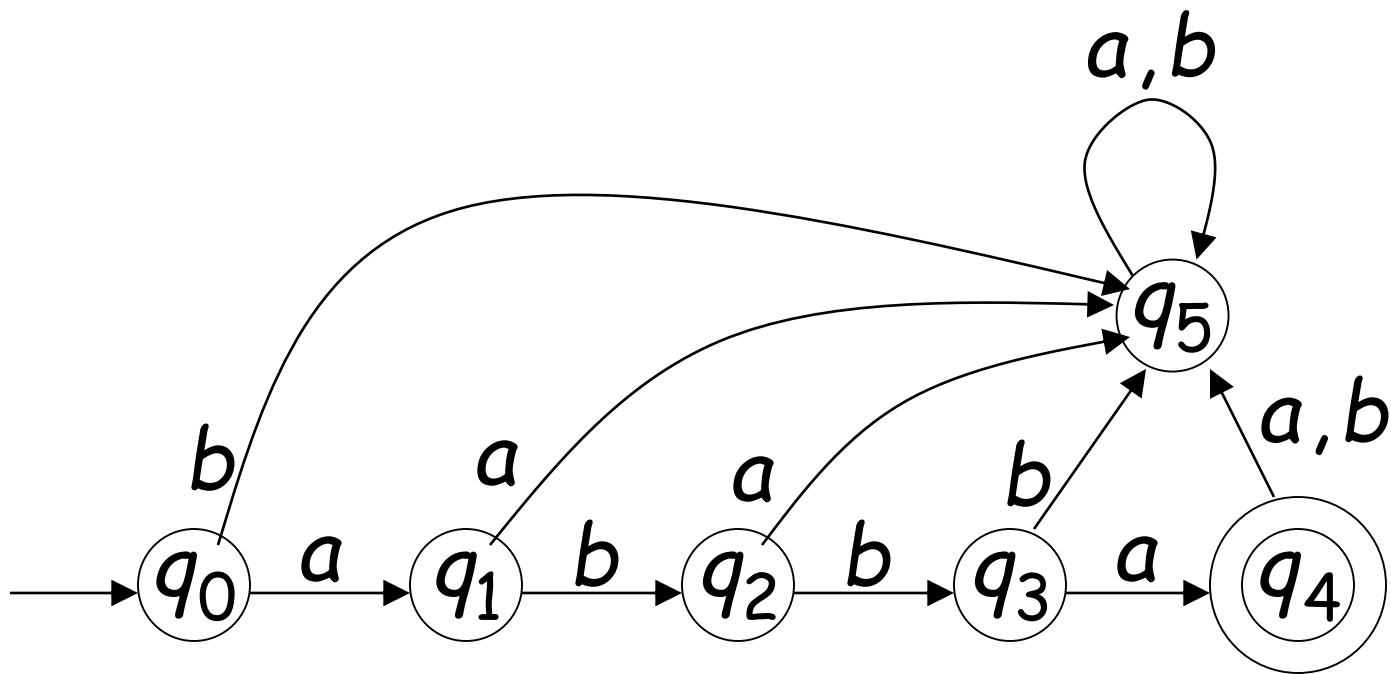
□ $\Sigma = \{a, b\}$



Set of States

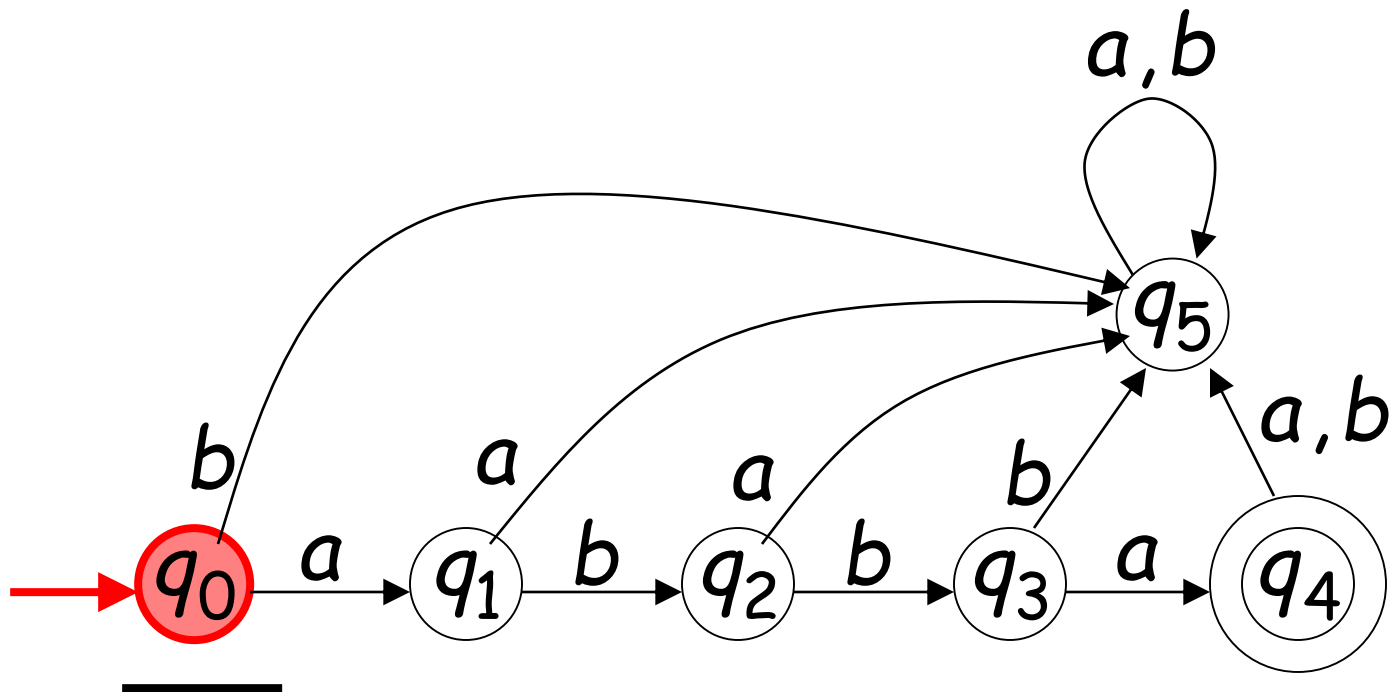
Q

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$



Initial State

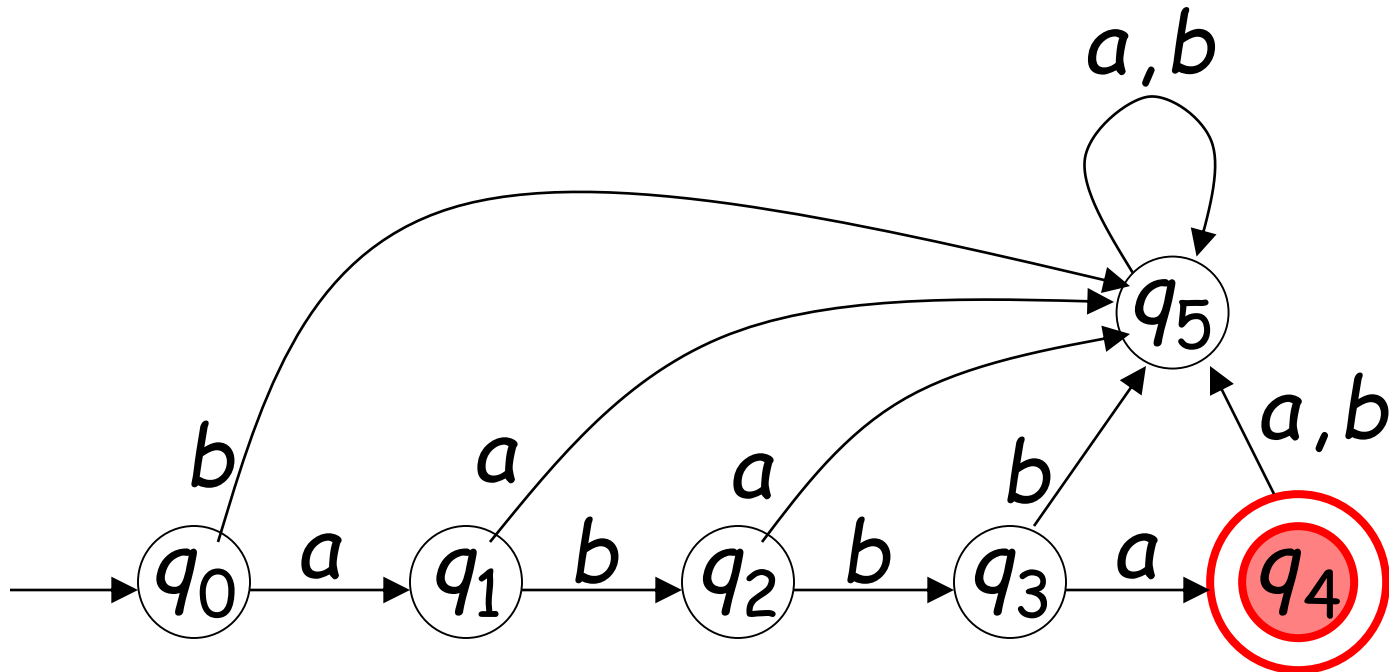
q_0



Set of Accepting States or Final State

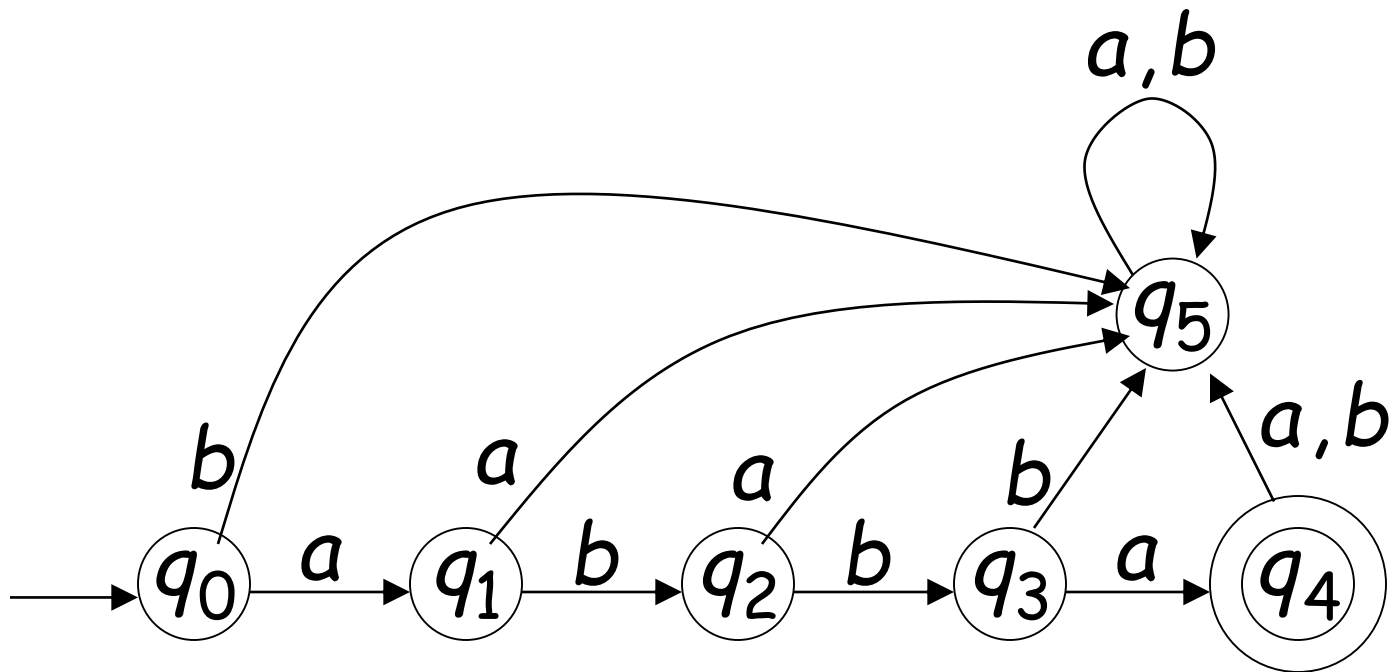
F

□ $F = \{q_4\}$

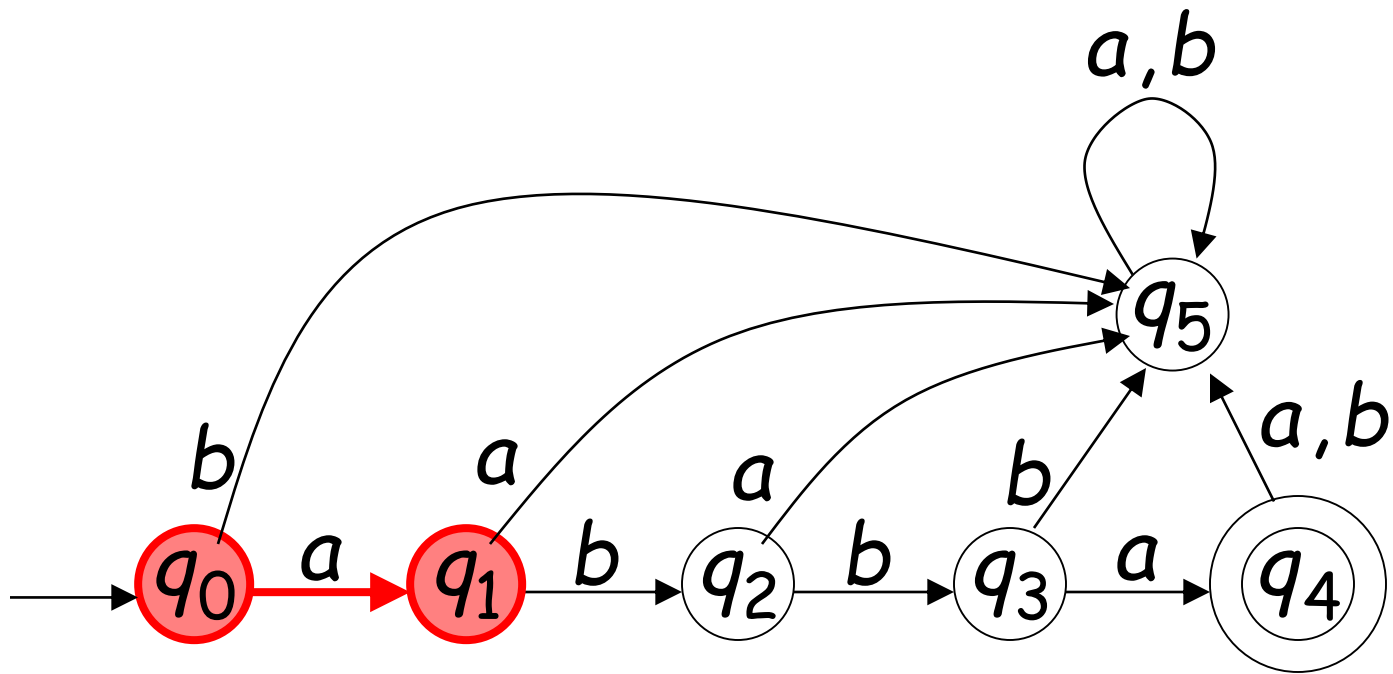


Transition Function δ

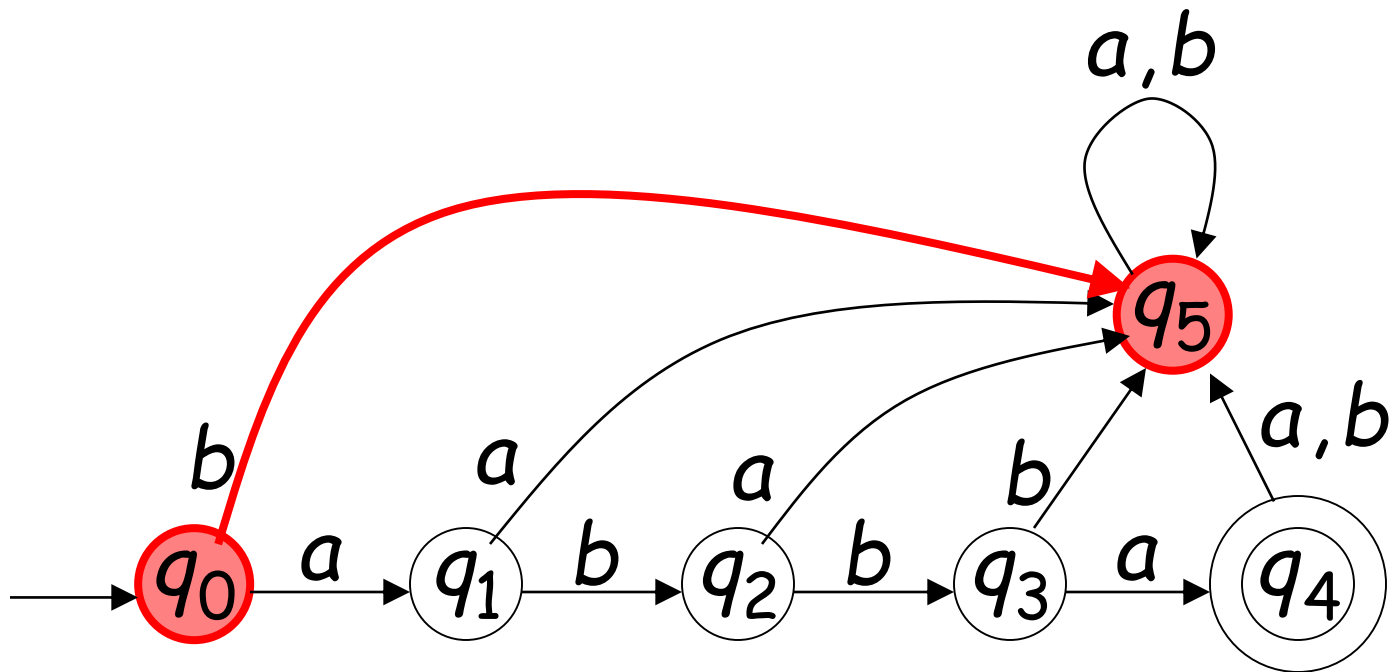
$$\delta : Q \times \Sigma \rightarrow Q$$



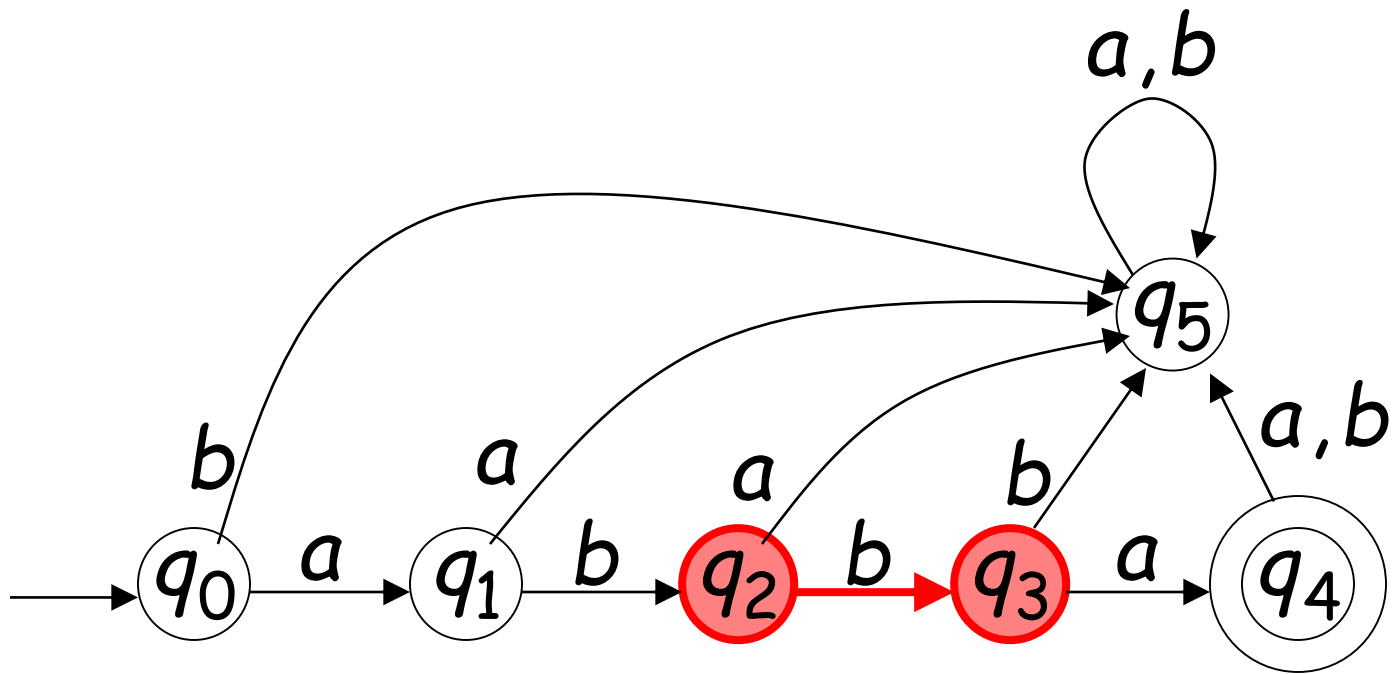
$$\delta(q_0, a) = q_1$$



$$\delta(q_0, b) = q_5$$

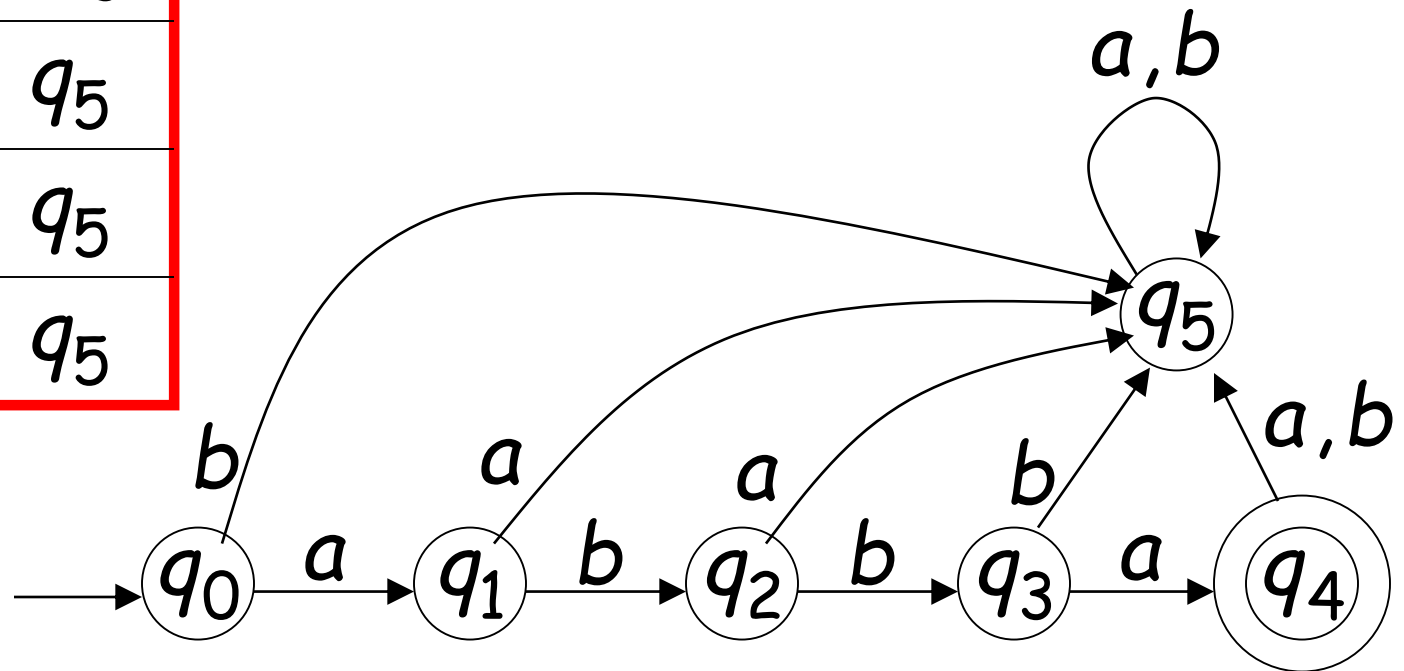


$$\delta(q_2, b) = q_3$$



Transition Function (δ) Contd...

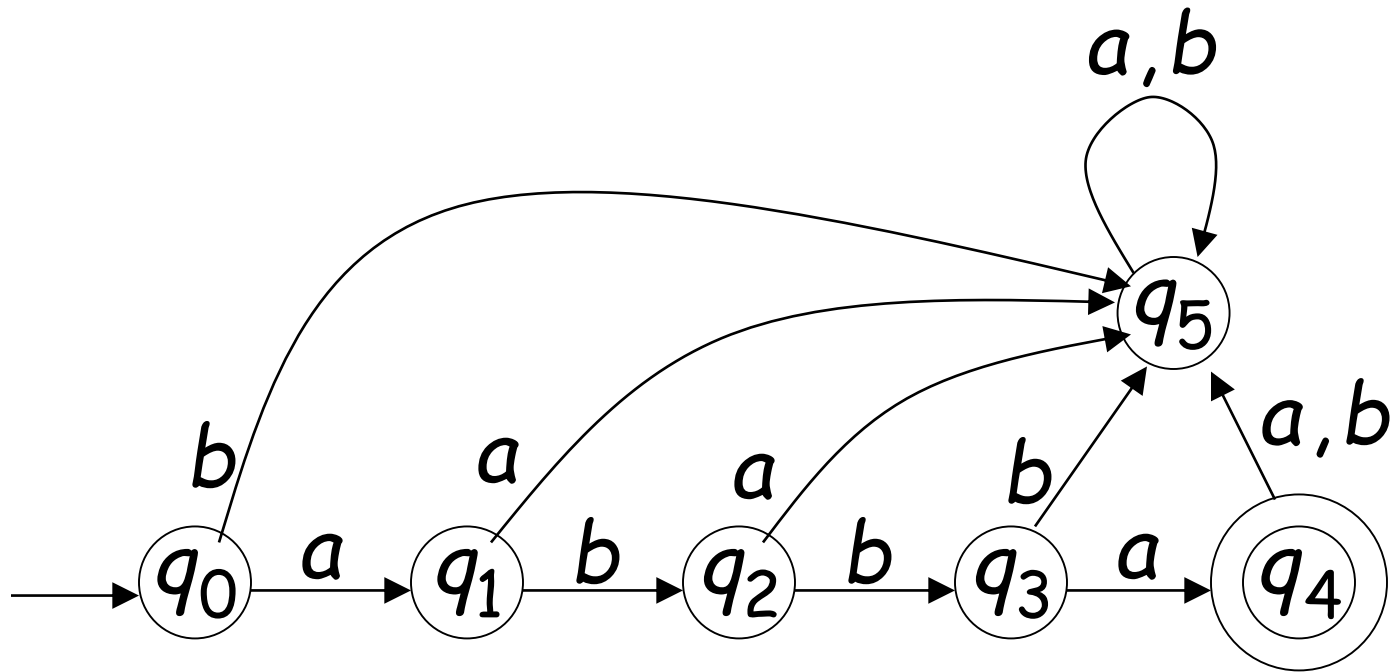
δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5



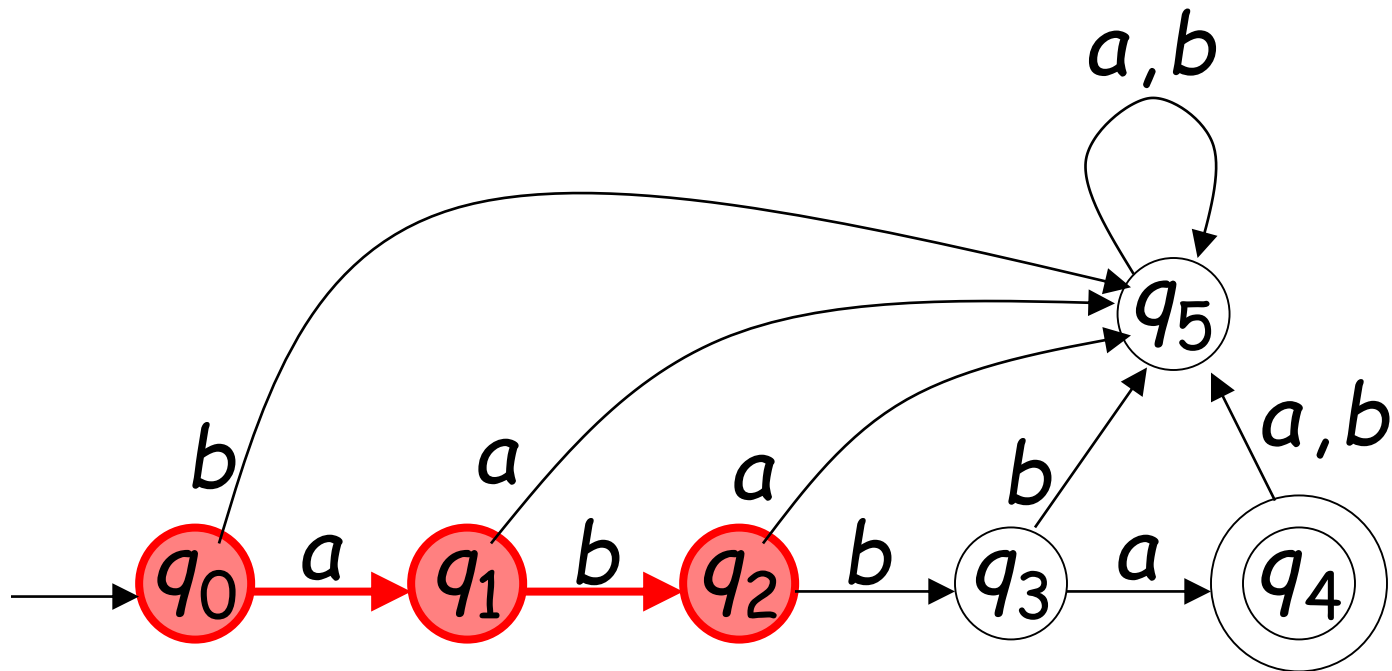
Extended Transition Function δ^*



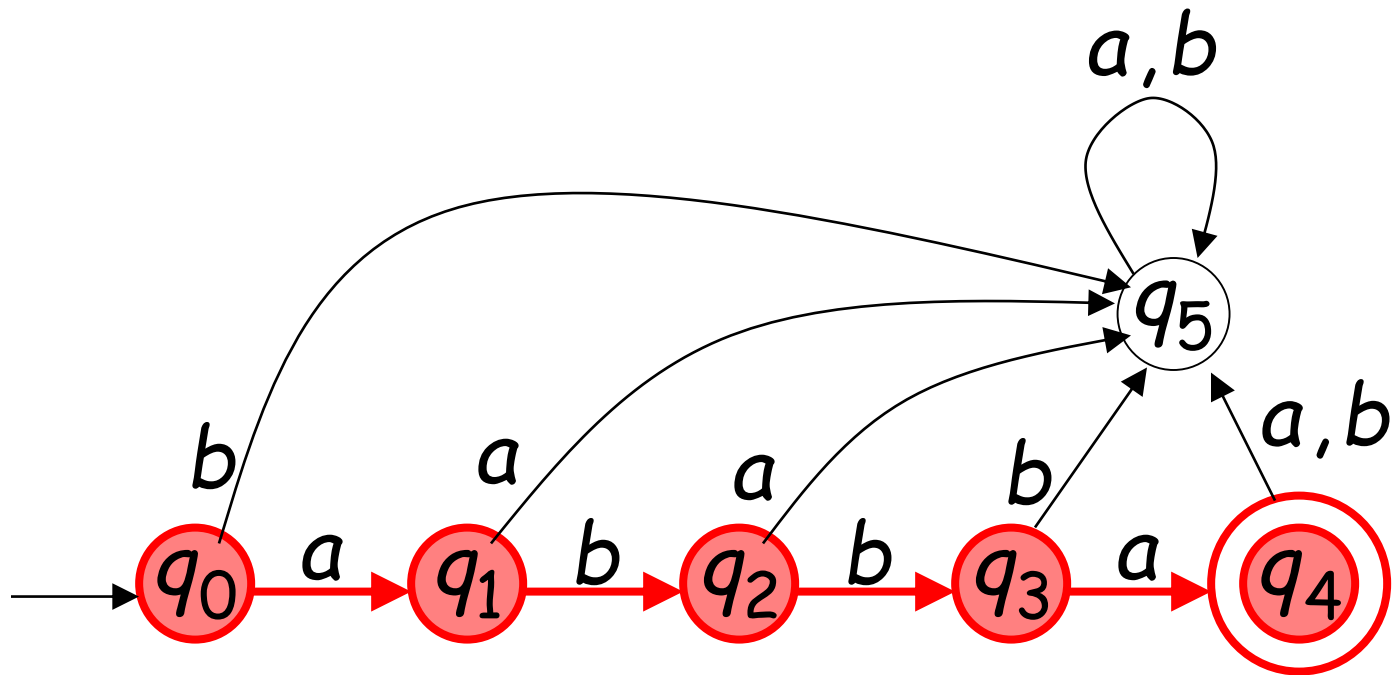
$$\delta^*: Q \times \Sigma^* \rightarrow Q$$



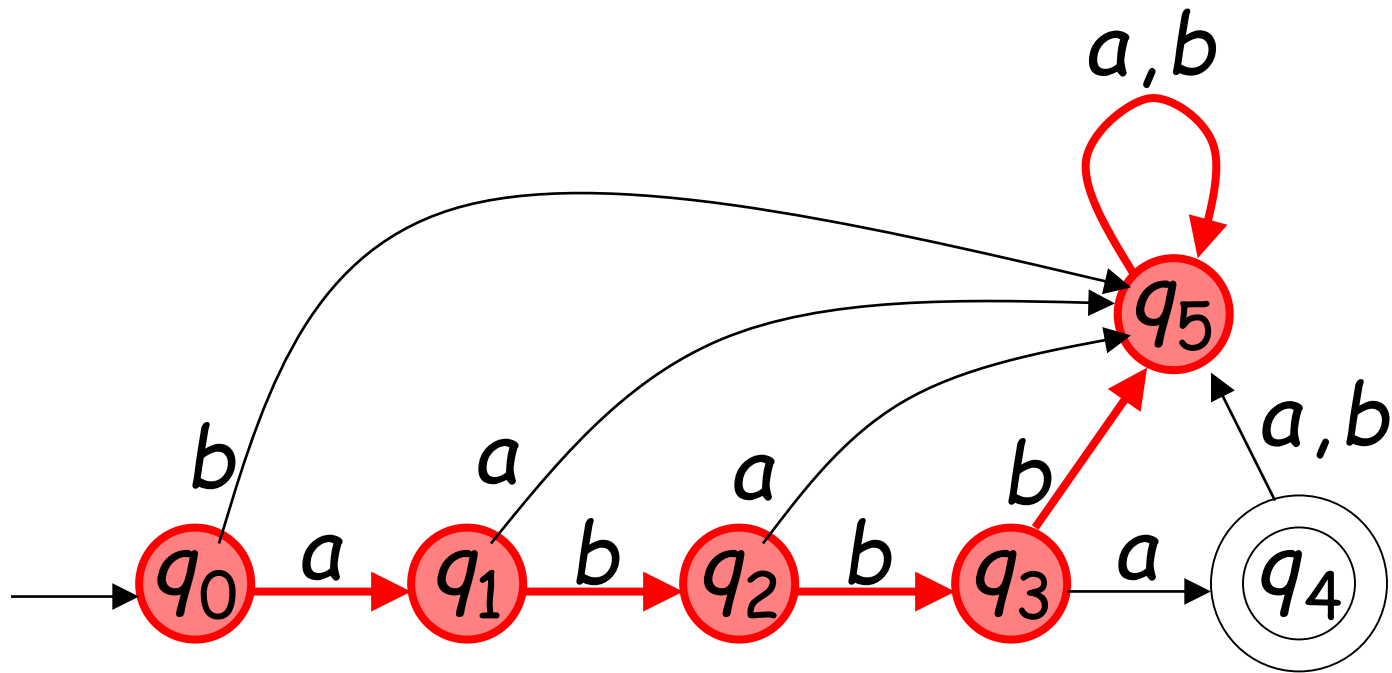
$$\delta^*(q_0, ab) = q_2$$



$$\delta^*(q_0, abba) = q_4$$



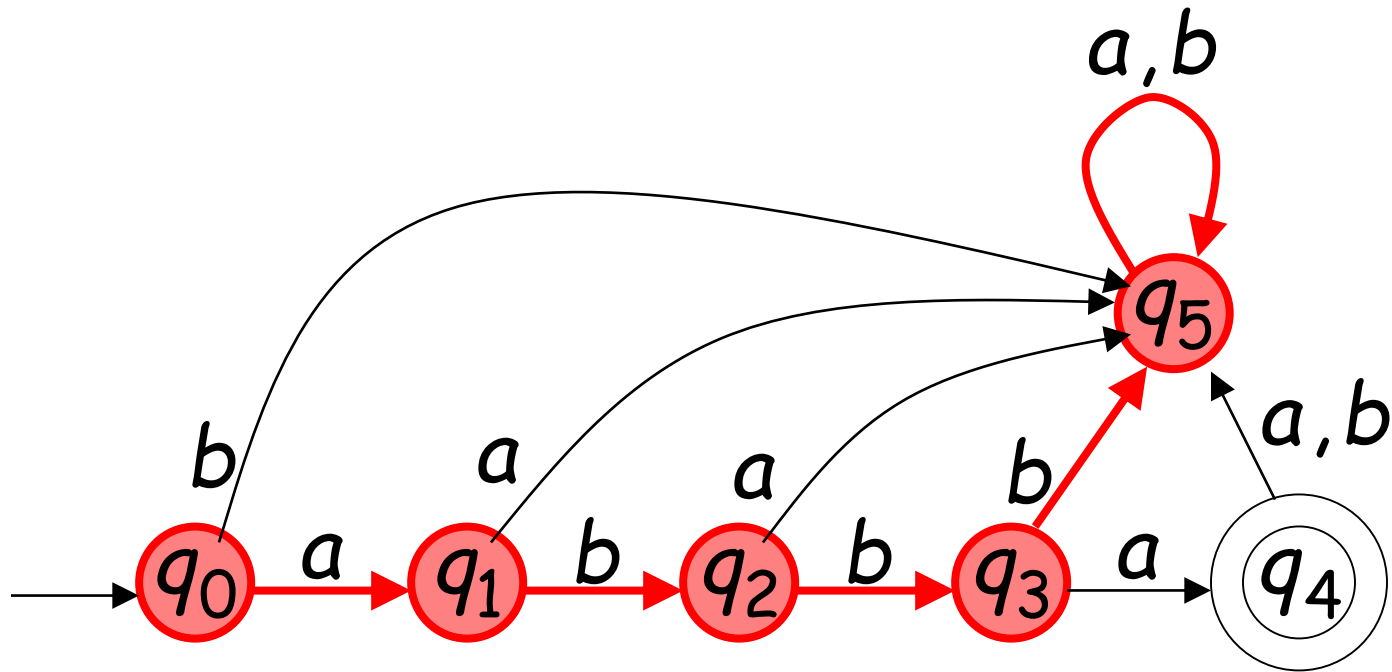
$$\delta^*(q_0, abbbaa) = q_5$$



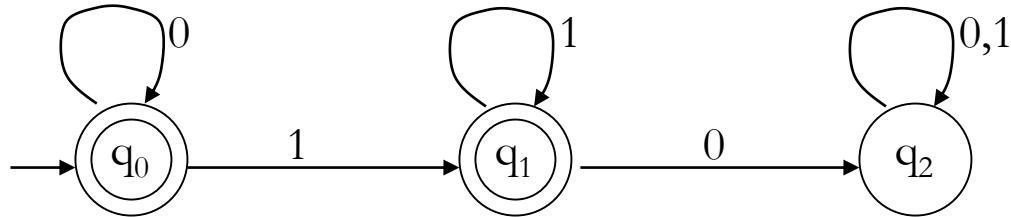
Observation:

Example: There is a walk from q_0 to q_5
with label $abbbaa$

$$\delta^*(q_0, abbbaa) = q_5$$



Example-2



alphabet $\Sigma = \{0, 1\}$

states $Q = \{q_0, q_1, q_2\}$

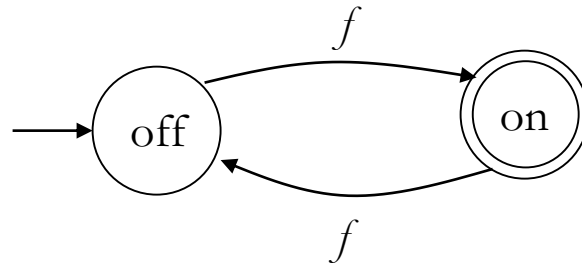
initial state q_0

Final/ accepting states $F = \{q_0, q_1\}$

transition function δ :

		inputs	
		0	1
states	q_0	q_0	q_1
	q_1	q_2	q_1
	q_2	q_2	q_2

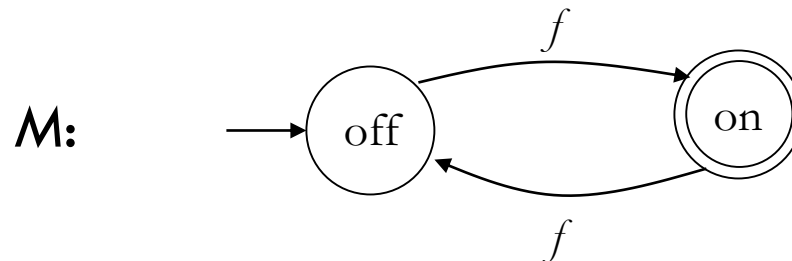
Example of a finite automaton



- There are **states** off and on, the automaton **starts** in off and tries to reach the “**good state**” on
- What sequences of f s lead to the final state?
- Answer: $\{f, fff, fffff, \dots\} = \{f^n : n \text{ is odd}\}$
- This is an **example** of a deterministic finite automaton over alphabet $\{f\}$

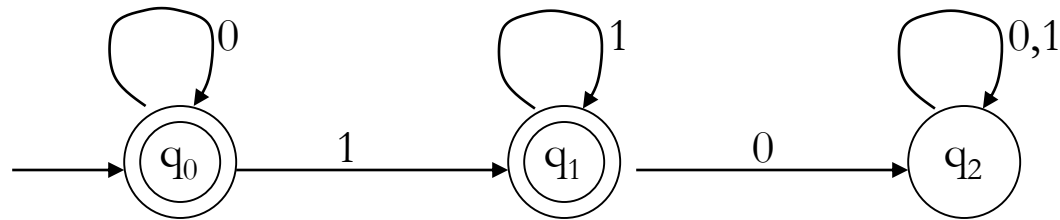
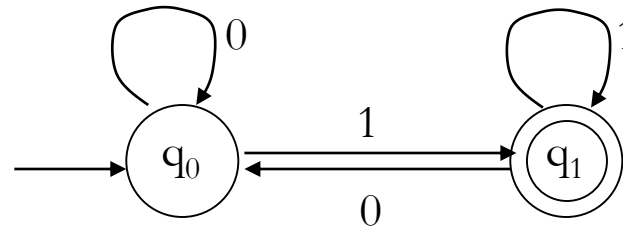
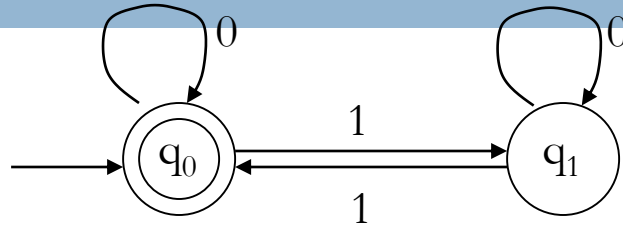
Language of a DFA

The **language of a DFA** $(Q, \Sigma, \delta, q_0, F)$ is the set of all strings over Σ that, starting from q_0 and following the transitions as the string is read left to right, will reach some accepting state.



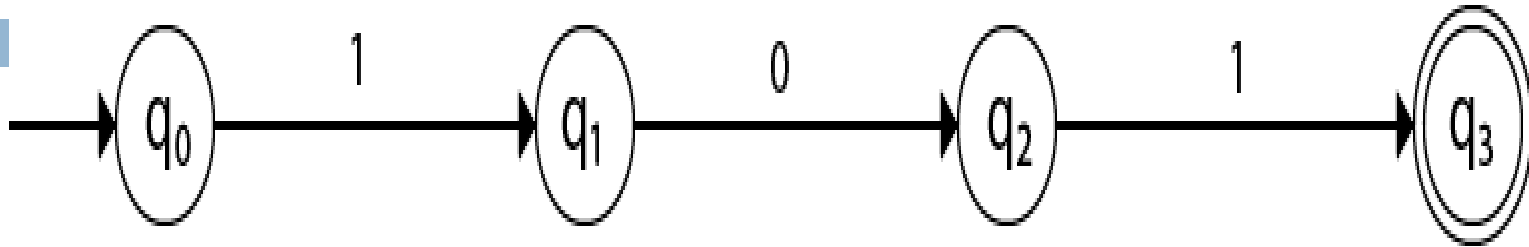
□ Language of M is $\{f, fff, fffff, \dots\} = \{f^n : n \text{ is odd}\}$

Examples

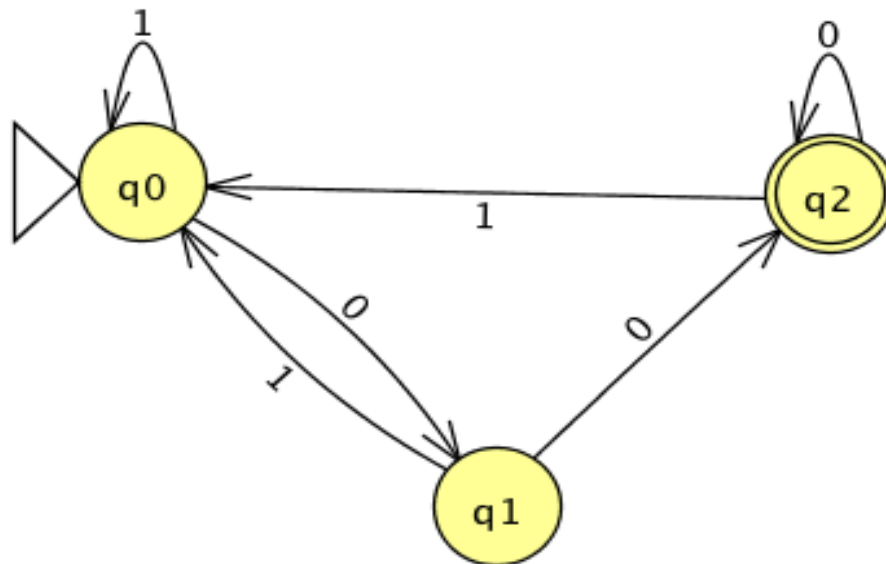


What are the languages of these DFAs?

Example: Design a FA with $\Sigma = \{0, 1\}$ accepts the only input 101.



Example: The set of all strings $\Sigma = \{0, 1\}$ ending in 00.

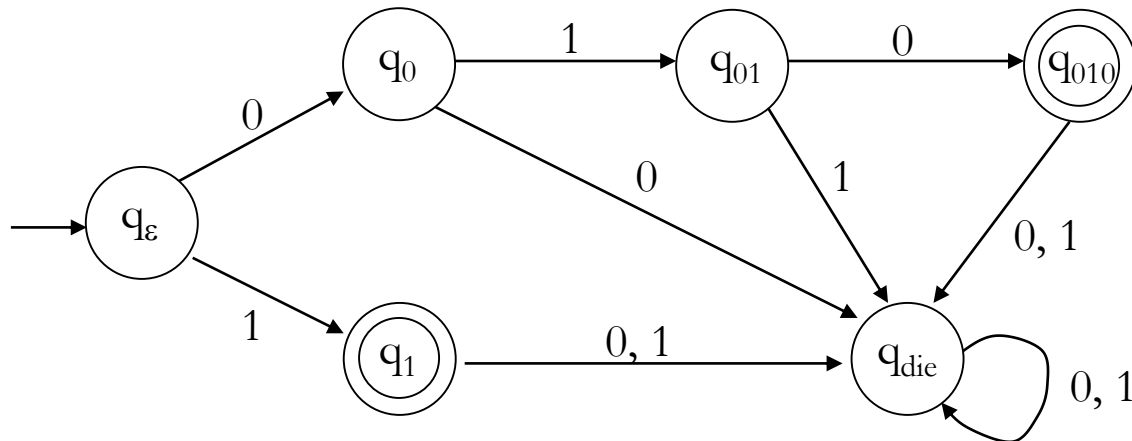


Examples

- Construct a DFA that accepts the language

$$L = \{010, 1\} \quad (\Sigma = \{0, 1\})$$

- Answer



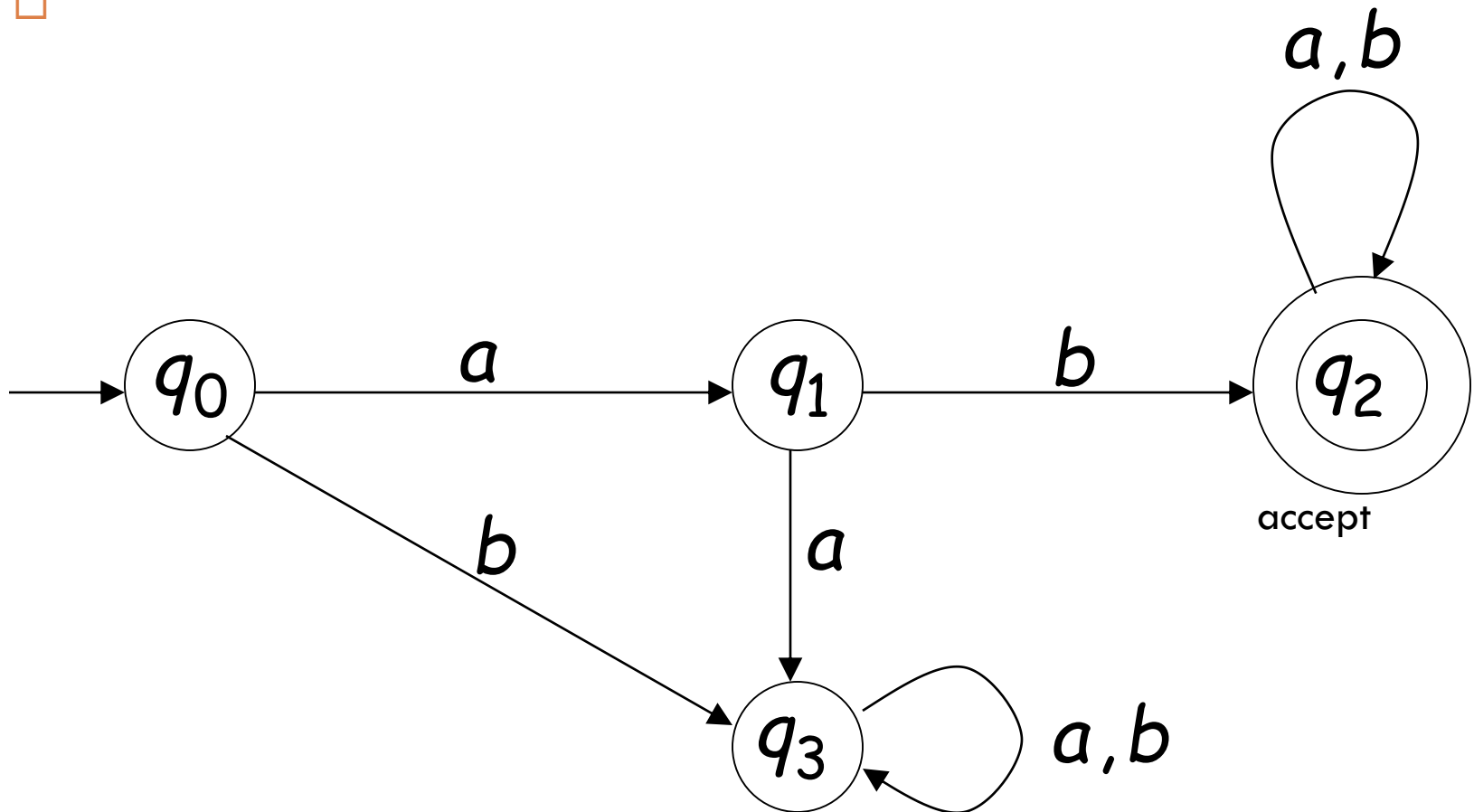
Σ^+ and Σ^*

- Σ is an alphabet , $\Sigma = \{y\}$
- Σ^* is the set of all strings including null or obtained by concatenating zero or more symbols from Σ
- $\Sigma^* = \{\epsilon, y, yy, yyy, yyyy, \dots\}$
- Σ^+ is the set of all strings excluding null or obtained by concatenating one or more symbols
- $\Sigma = \{y\}$
- $\Sigma^+ = \{y, yy, yyy, yyyy, \dots\}$

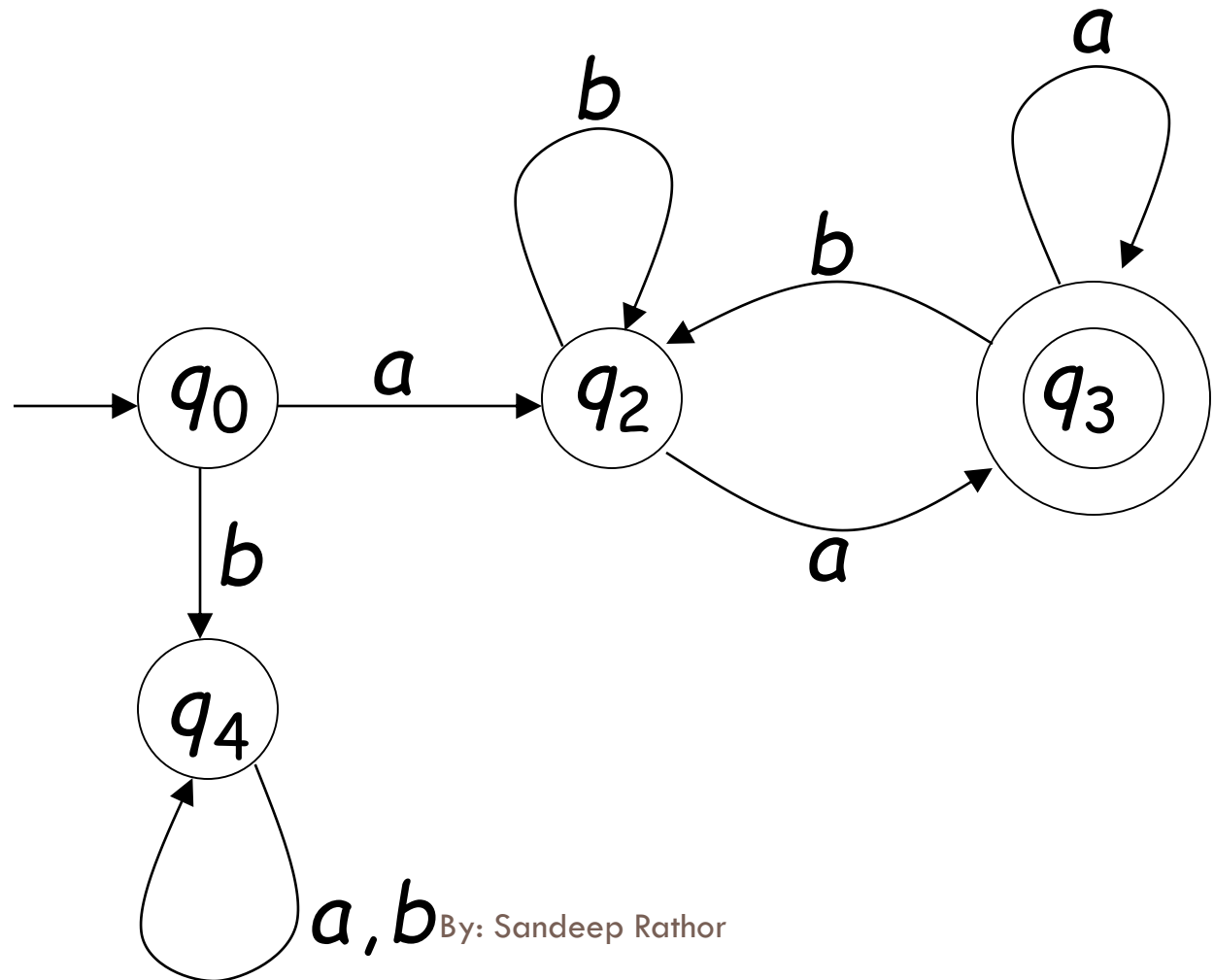
Example-4: Design a DFA for

$L(M) = \{ \text{all strings with prefix } ab \}$

□



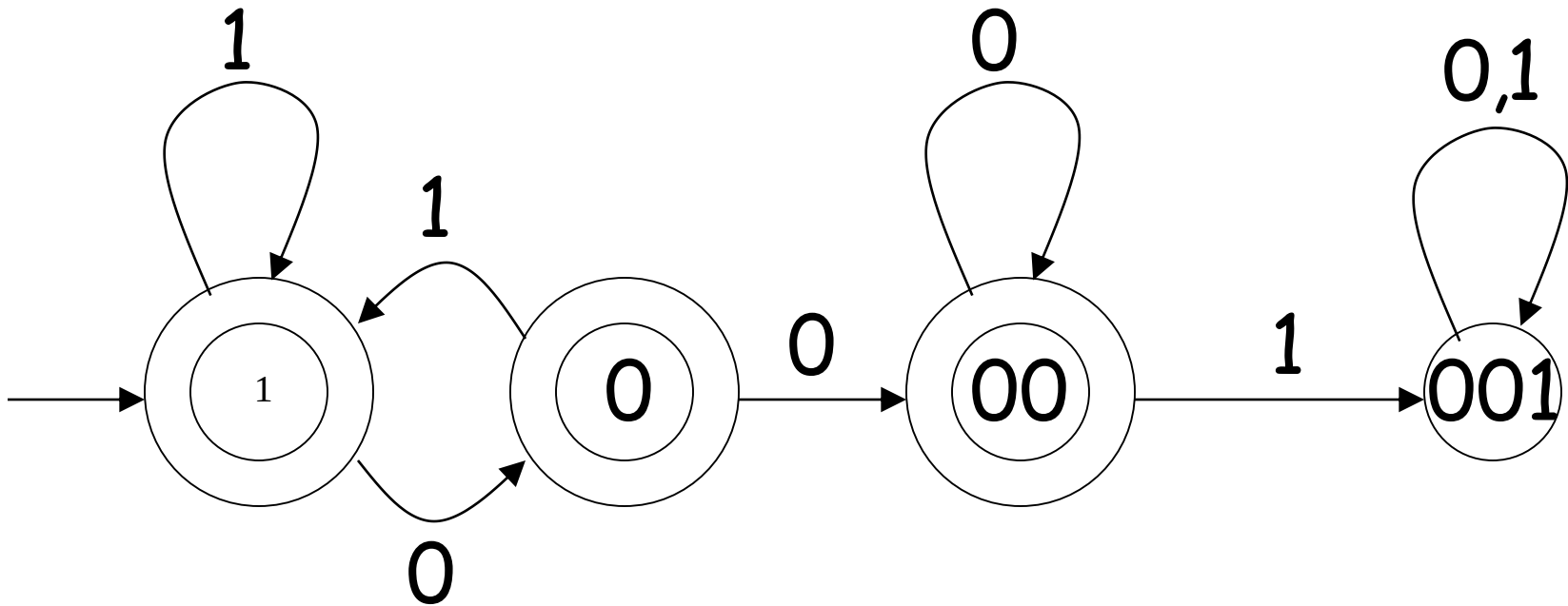
Example-5: DFA for $L(M) = \{awa : w \in \{a,b\}^*\}$



By: Sandeep Rathor

Example-6: DFA for

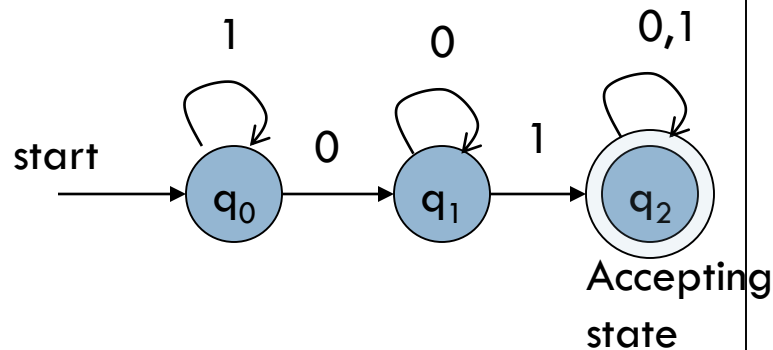
$L(M) = \{ \text{all strings without substring } 001 \}$





Ex-7: Design a DFA for language: $L = \{w \mid w \text{ is a binary string that contains } 01 \text{ as a substring}\}$

DFA for strings containing 01

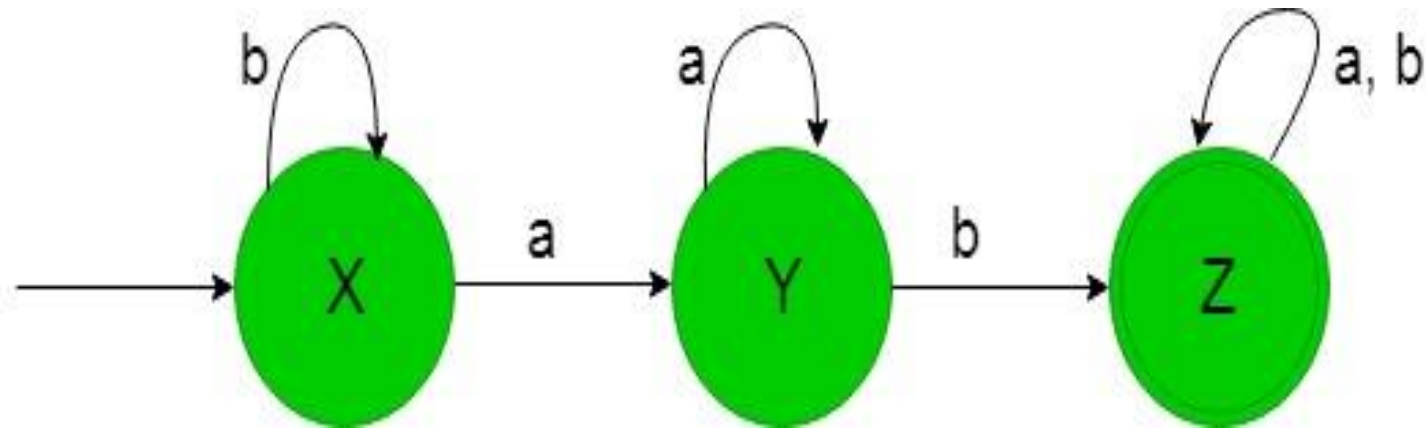


- What if the language allows empty strings?

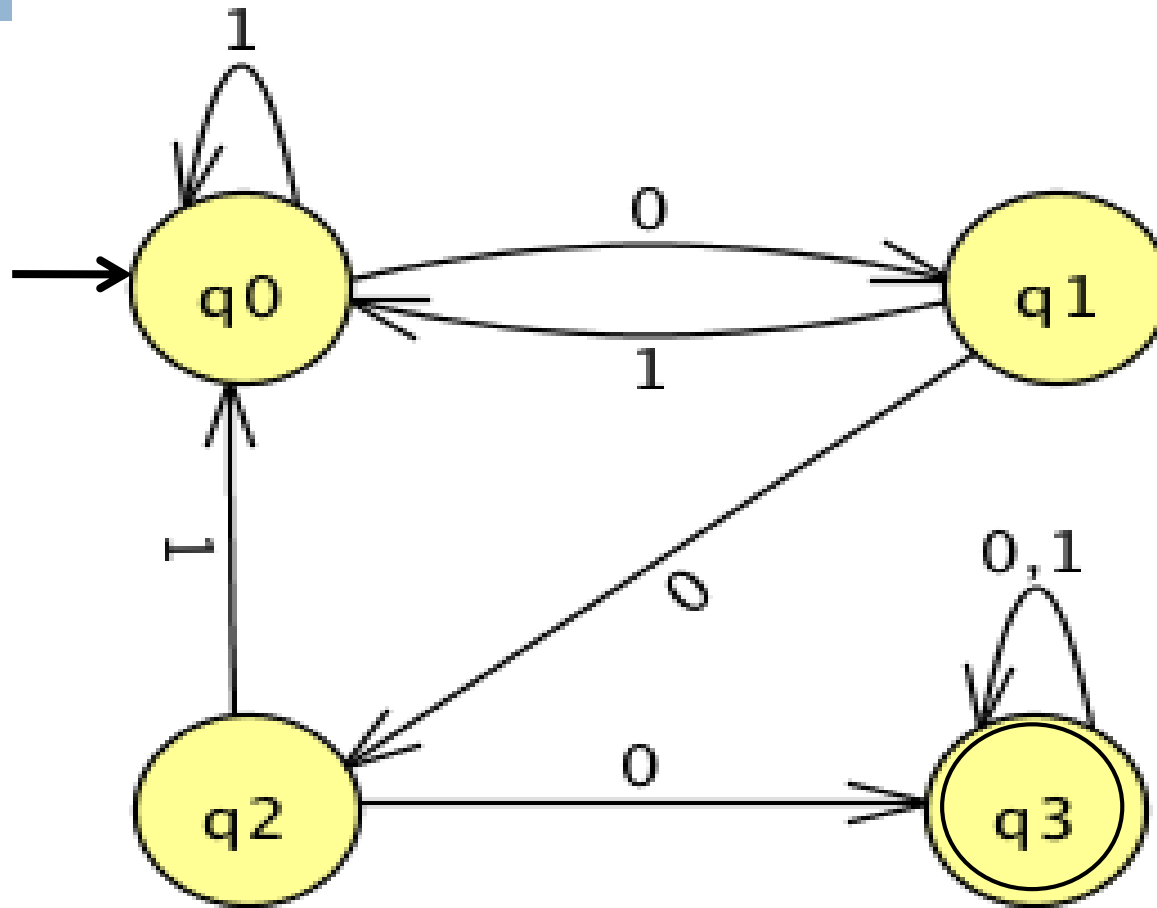
- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{0, 1\}$
- start state = q_0
- $F = \{q_2\}$
- Transition table

		symbols	
δ		0	1
states	q_0	q_1	q_0
	q_1	q_1	q_2
	$*q_2$	q_2	q_2

Construction of a DFA accepting set of string over $\{a, b\}$ where each string containing 'ab' as the substring.

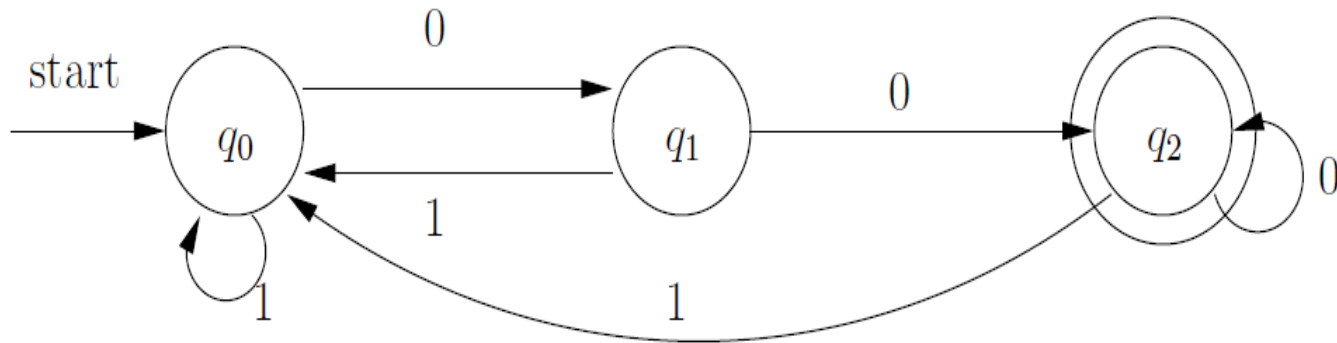


Example: The set of all strings with three consecutive 0's (not necessarily at the end).



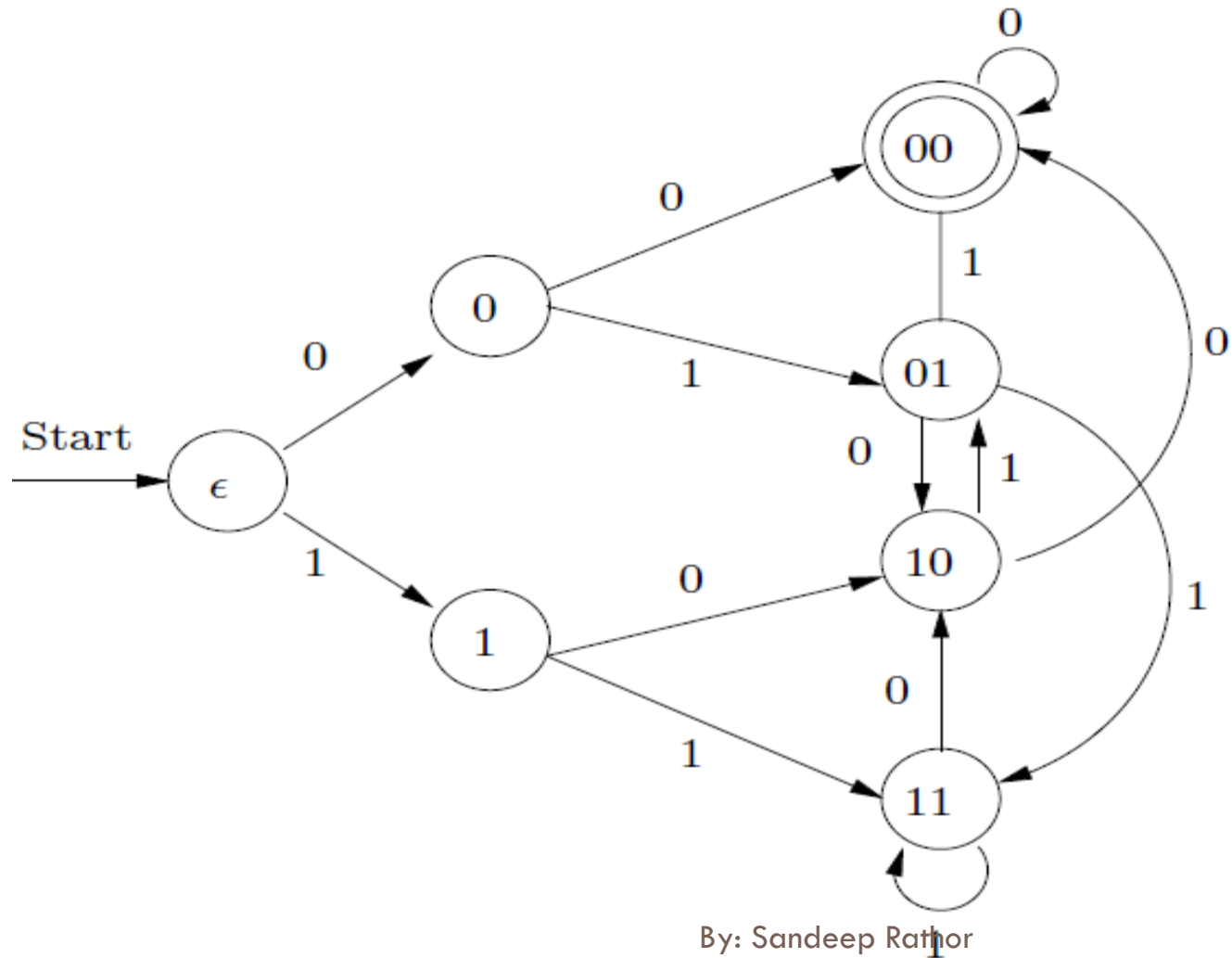
Construct DFA's accepting the following languages over the alphabet $\{0, 1\}$.

1. The set of all strings ending in 00.

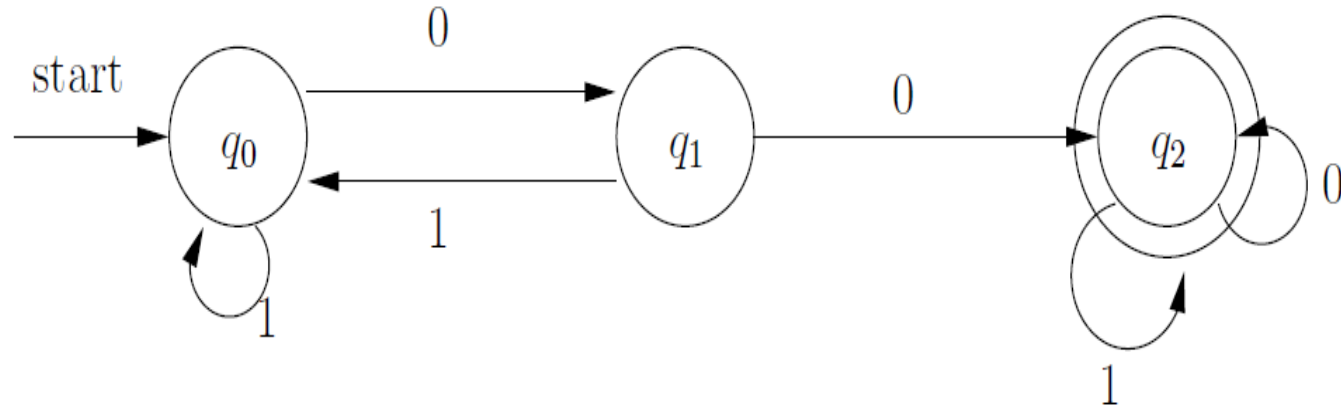


OR

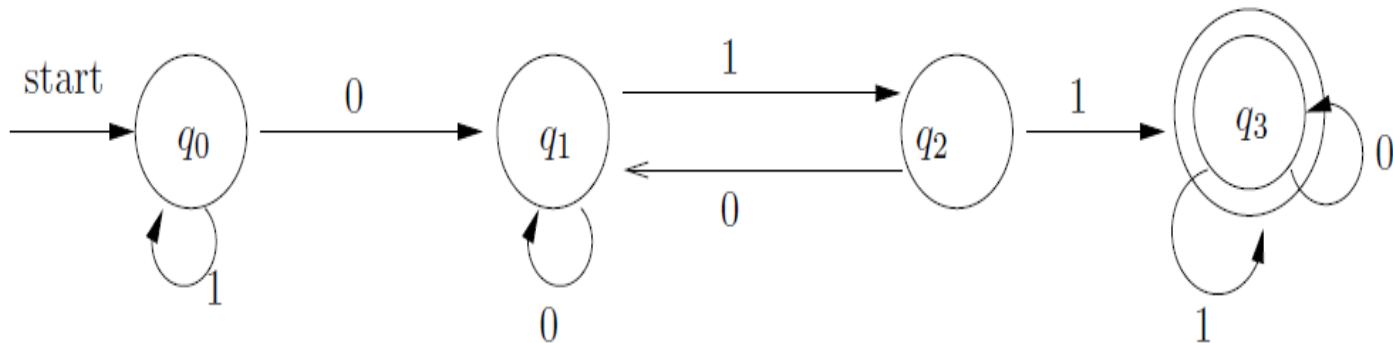
The set of all strings ending in 00.



2. The set of all strings with two consecutive 0's (not necessarily at the end).

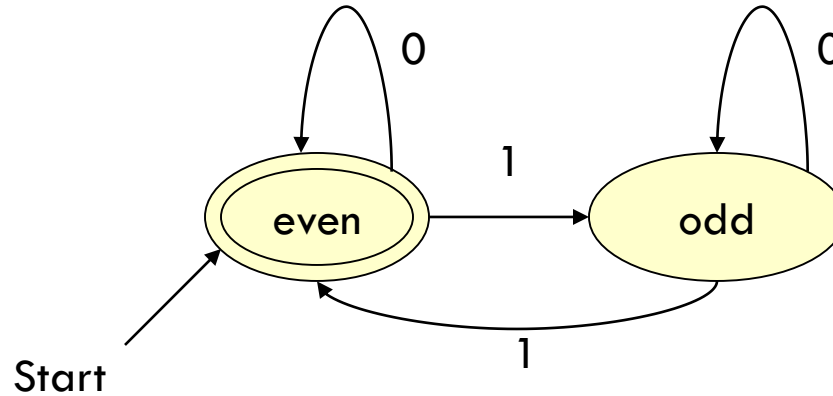


3. The set of strings with 011 as a substring

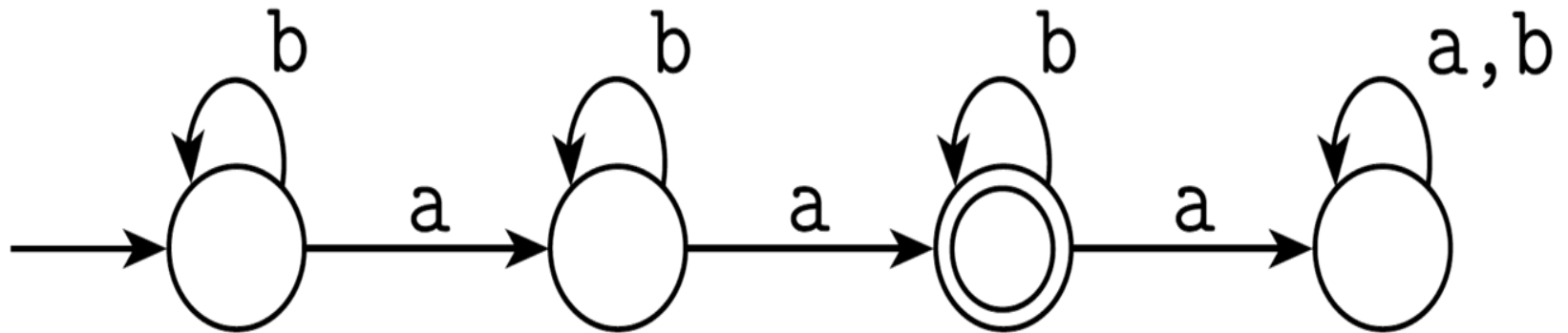


Practice Questions...

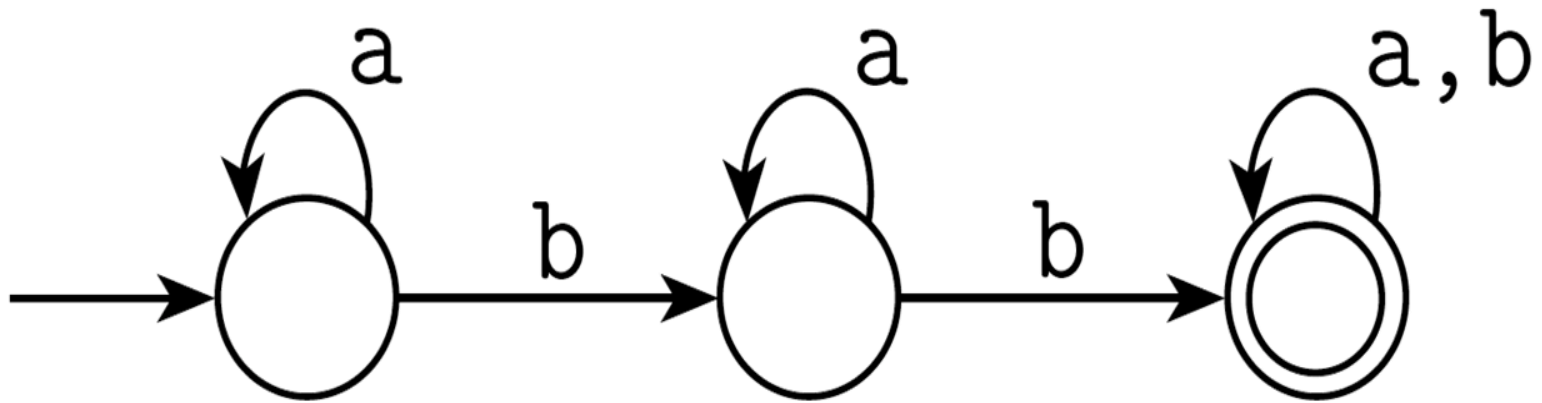
DFA for: An Even Number of 1's



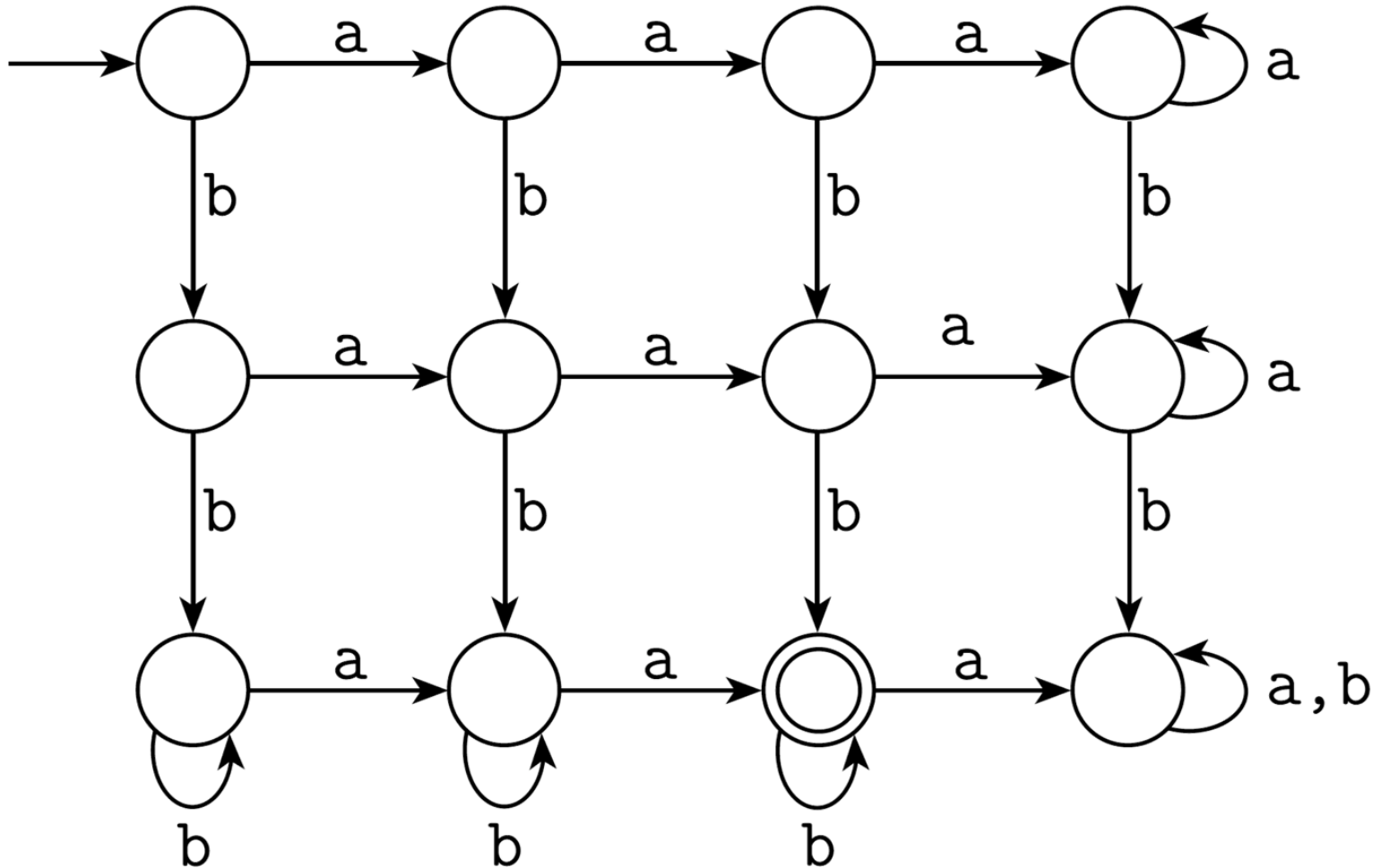
Exactly Two a's



At Least Two b's

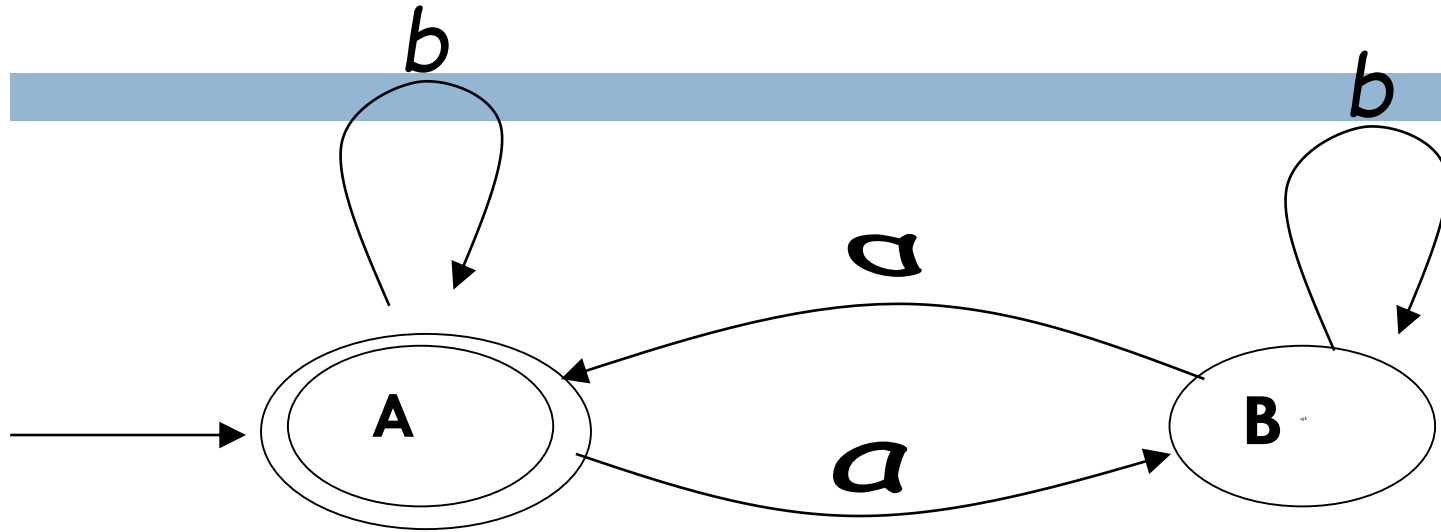


Exactly two a's and at least two b's

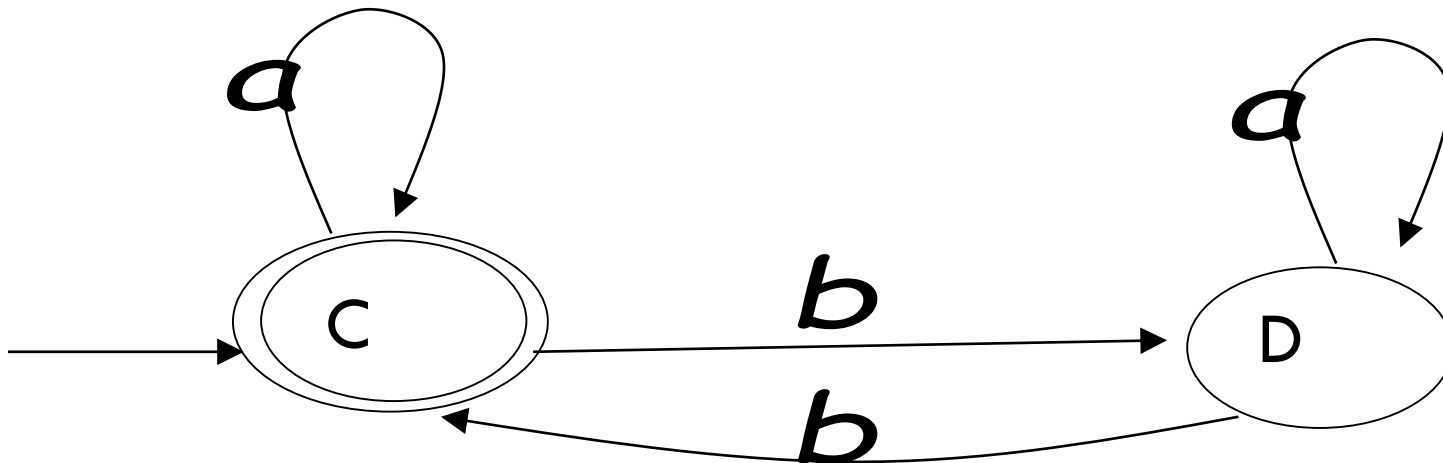


By: Sandeep Rathor

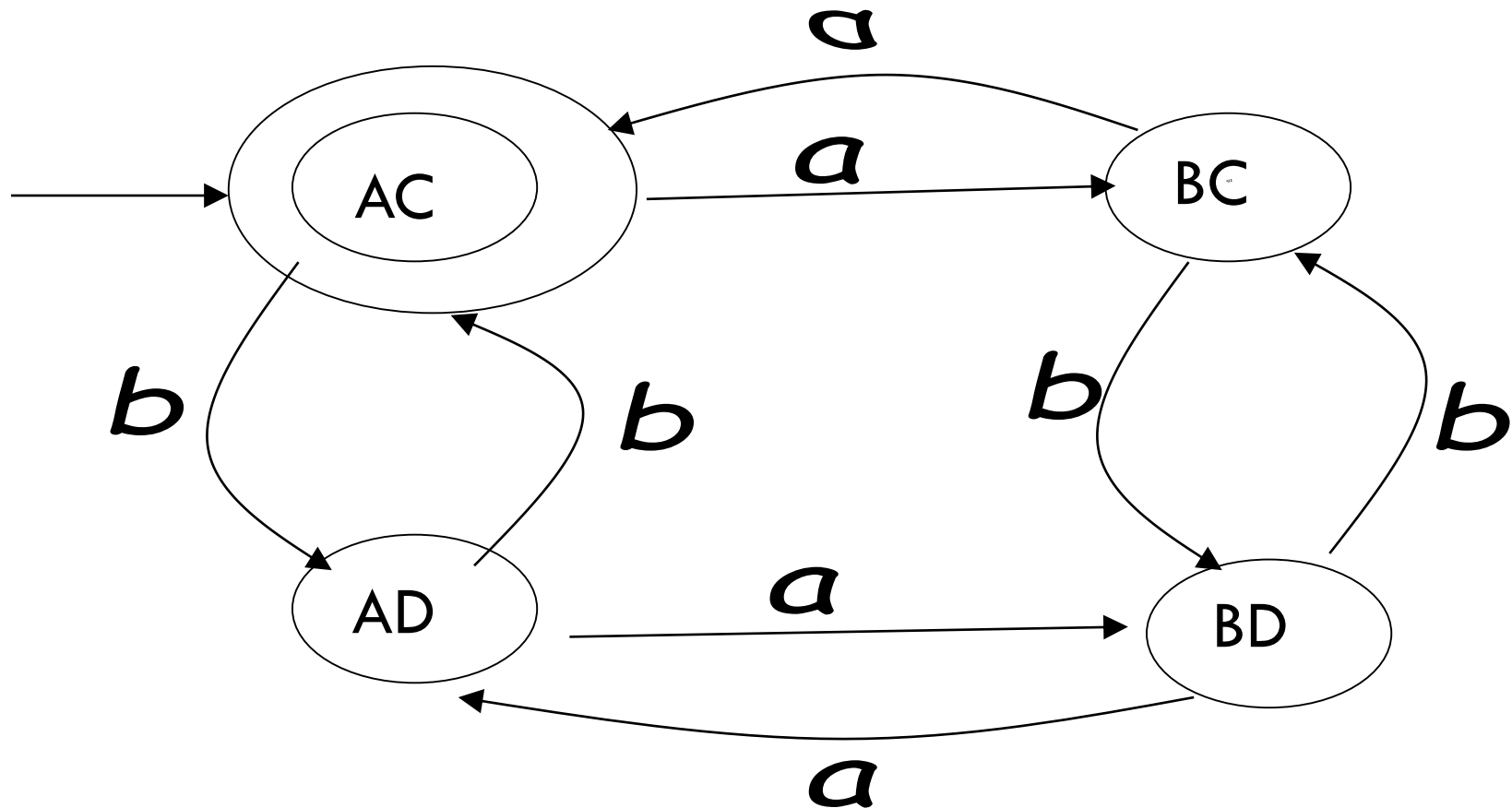
Construct a FA that accepts the strings having no. of 'a' divisible by 2 or even no. of 'a'



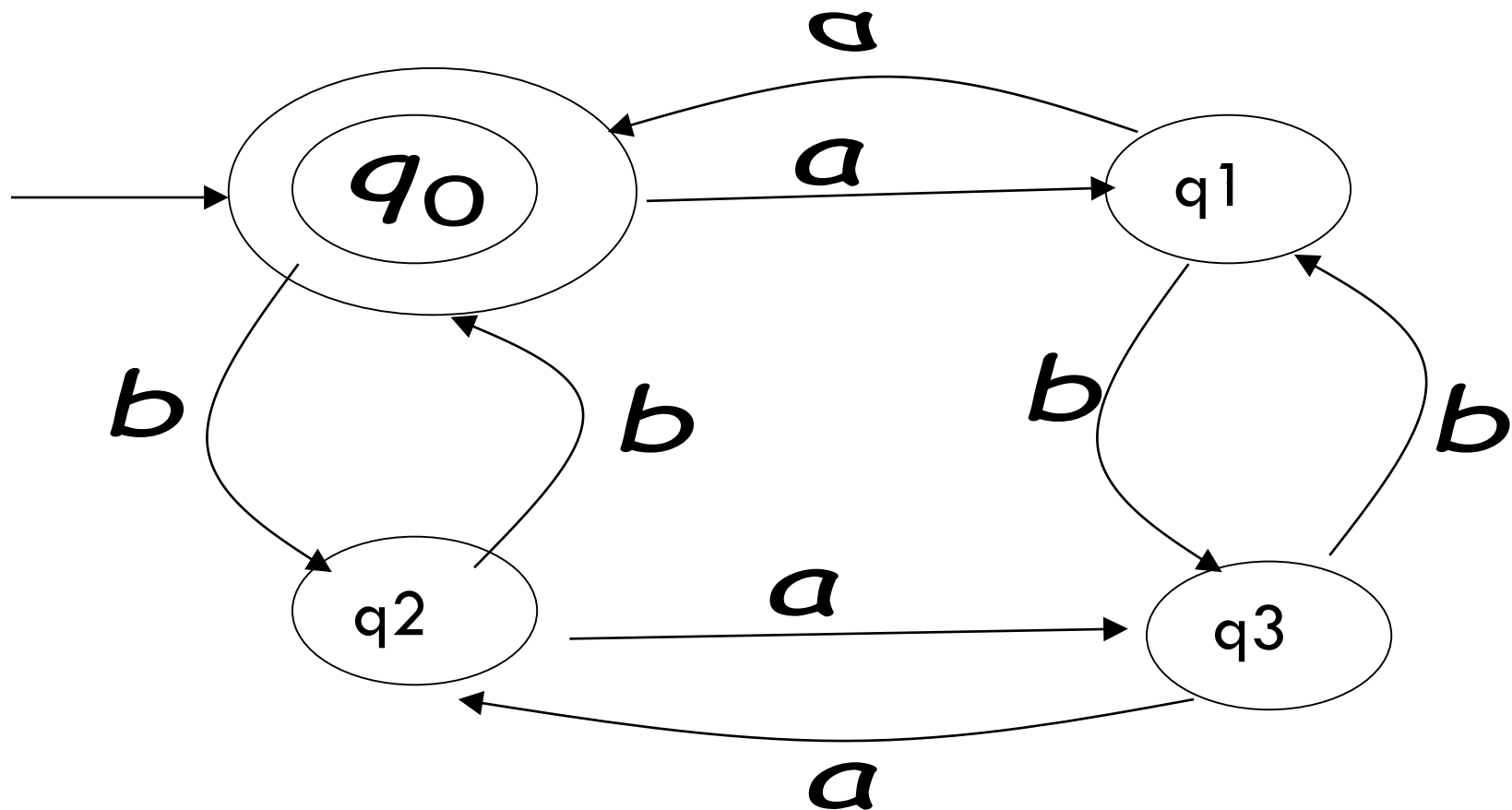
Construct a FA that accepts the strings having no. of 'b' divisible by 2 or even no. of 'b'



Construct a FA that accepts the strings having no. of 'a' divisible by 2 and no. of 'b' divisible by 2

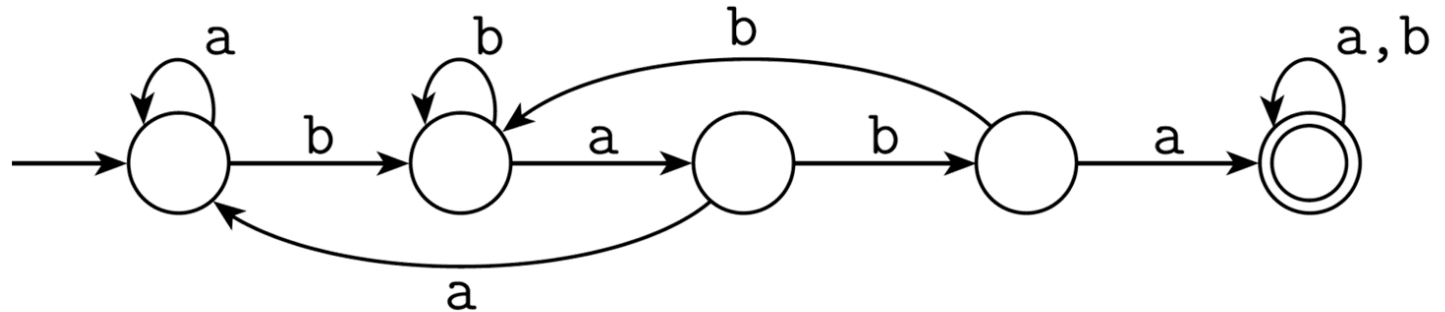


Construct a FA that accepts the strings having no. of 'a' divisible by 2 and no. of 'b' divisible by 2

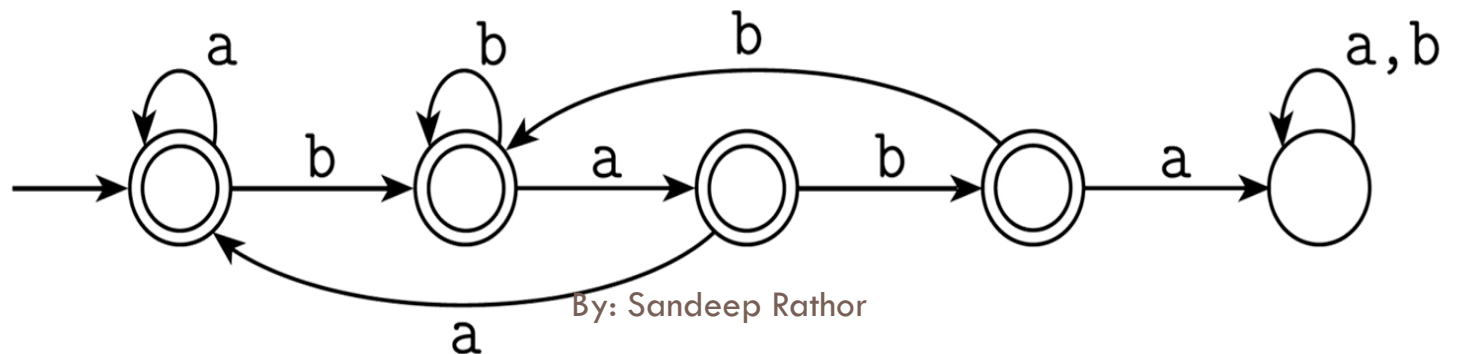


Containing Substrings or Not

- Contains baba:



- Does not contain baba:



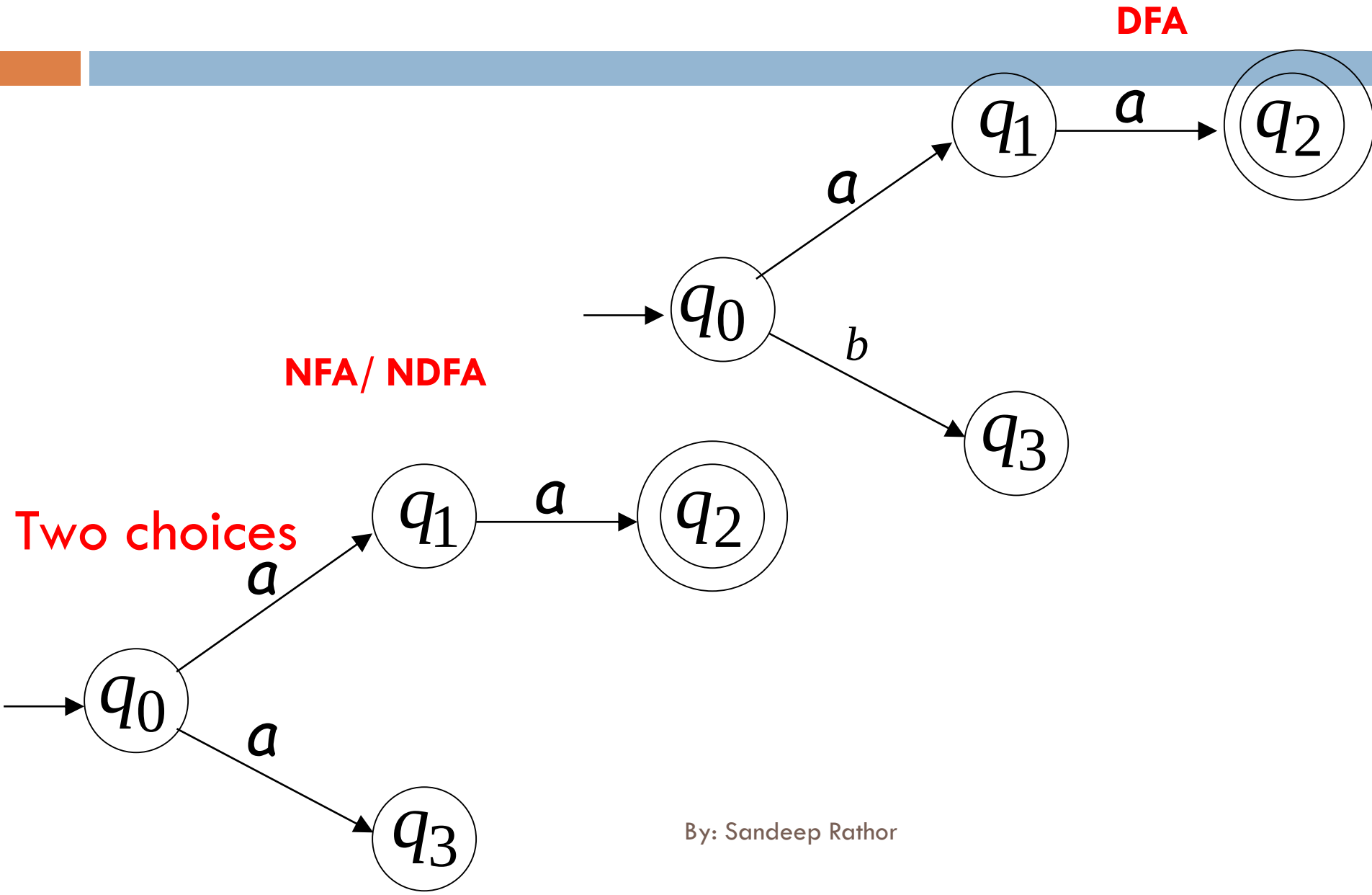
Non-Deterministic Finite Automata (NFA)

A **Non-deterministic Finite Automata** (NFA) is a 5-tuple

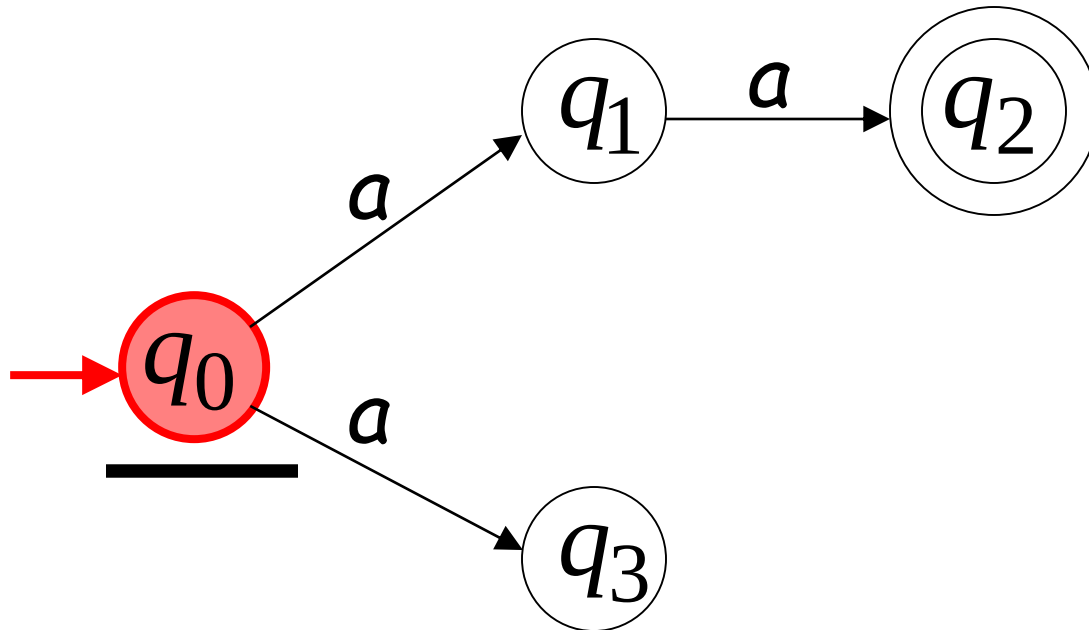
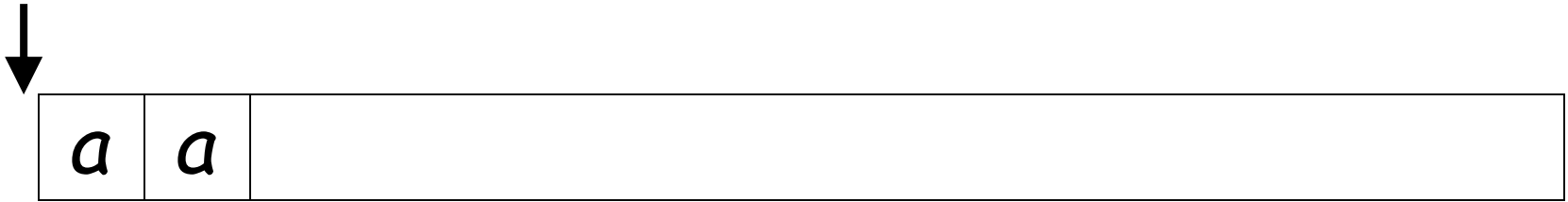
$(Q, \Sigma, \delta, q_0, F)$ where:

- ▣ Q is a finite set of **states**
- ▣ Σ is an **input alphabet**
- ▣ δ : is a **transition function** $Q \times \Sigma \rightarrow 2^Q$ [power set of Q]
- ▣ $q_0 \in Q$ is an **initial state**
- ▣ $F \subseteq Q$ is a set of **accepting states** (or **final states**).

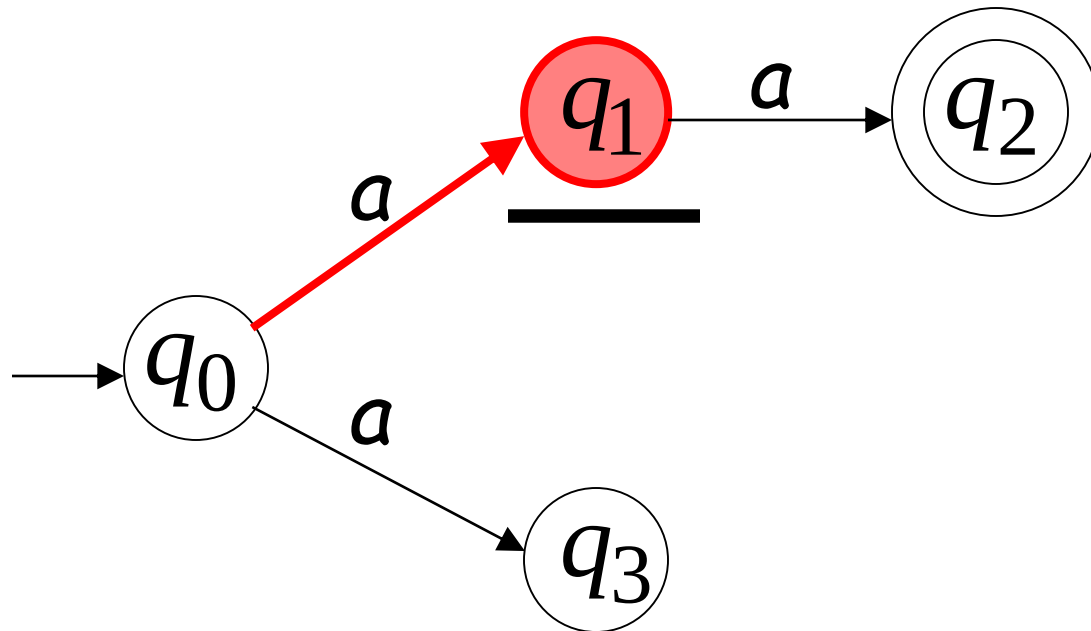
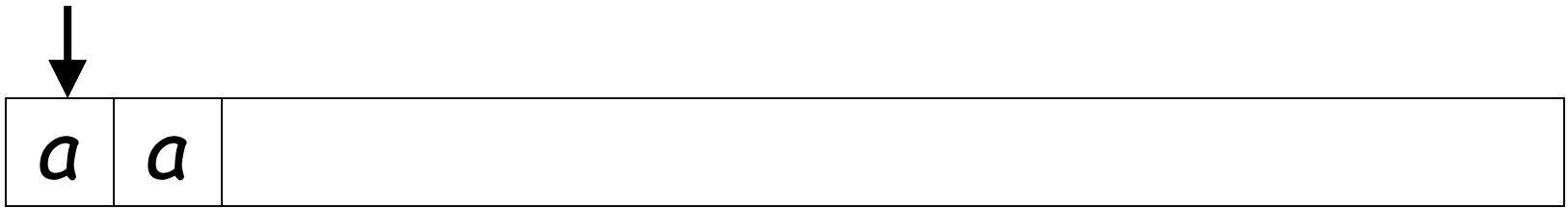
Example of DFA & NFA



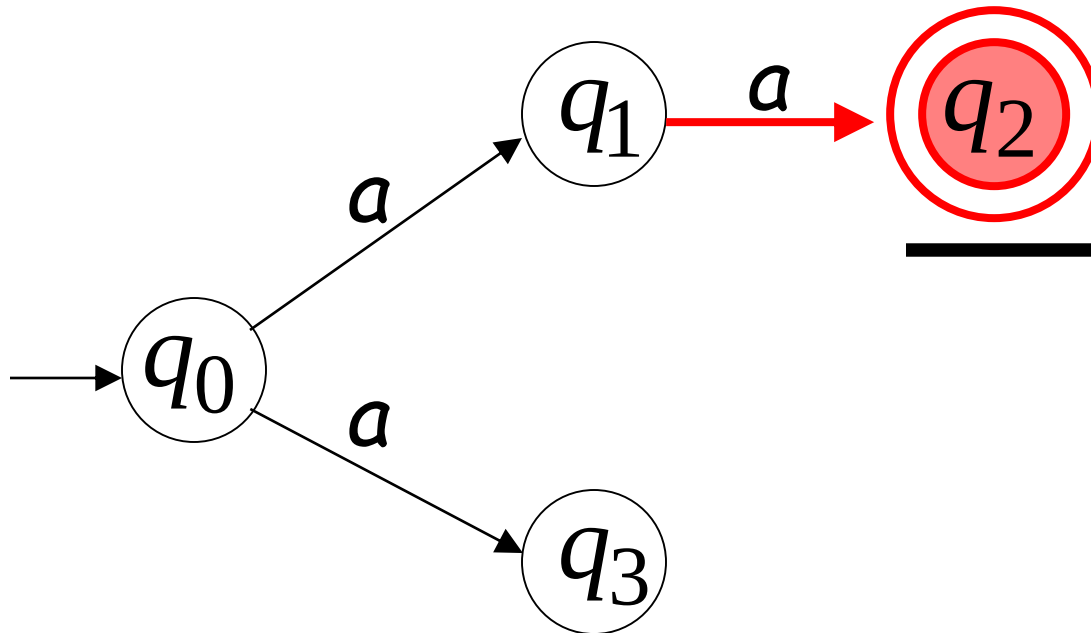
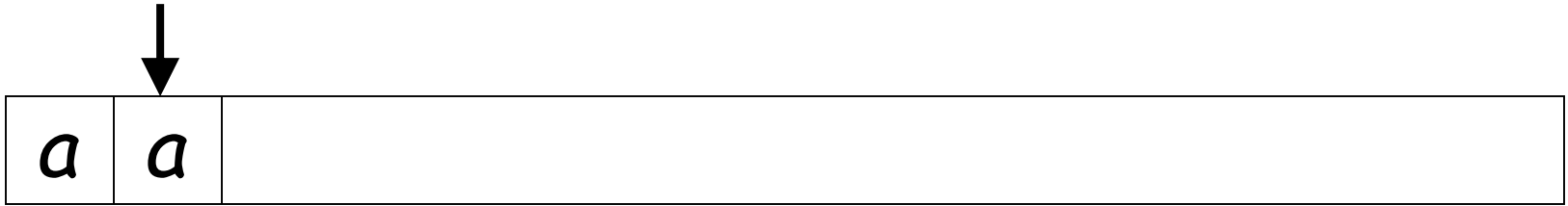
First Choice



First Choice



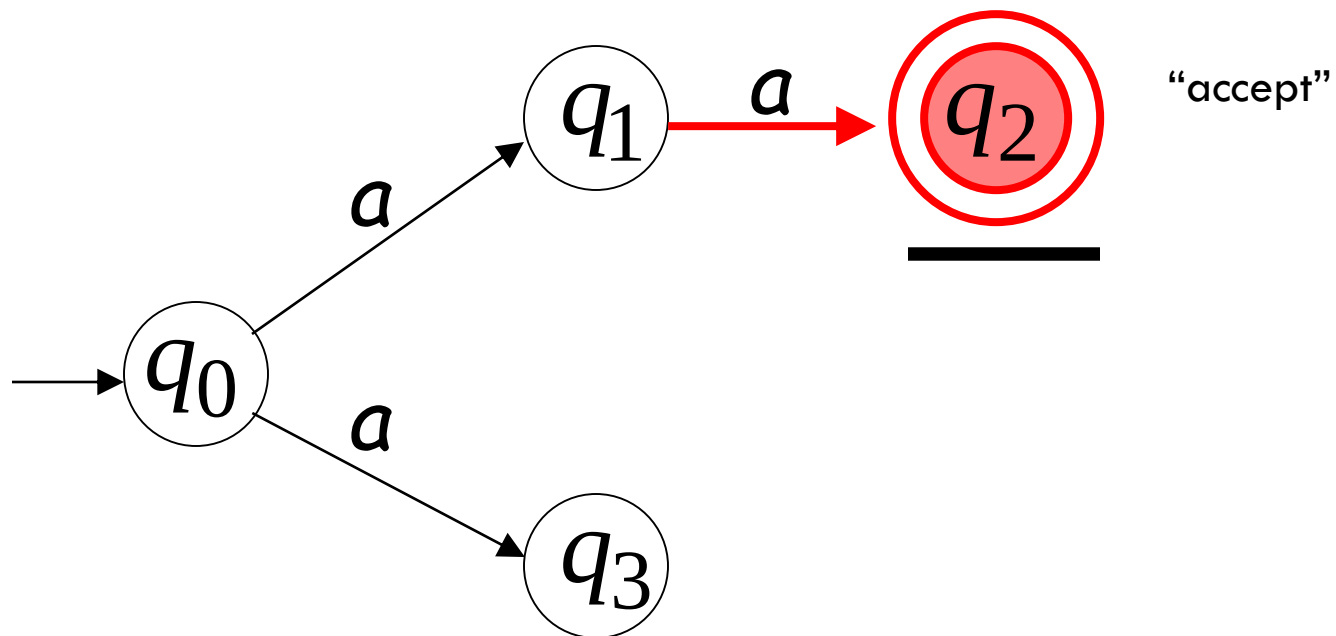
First Choice



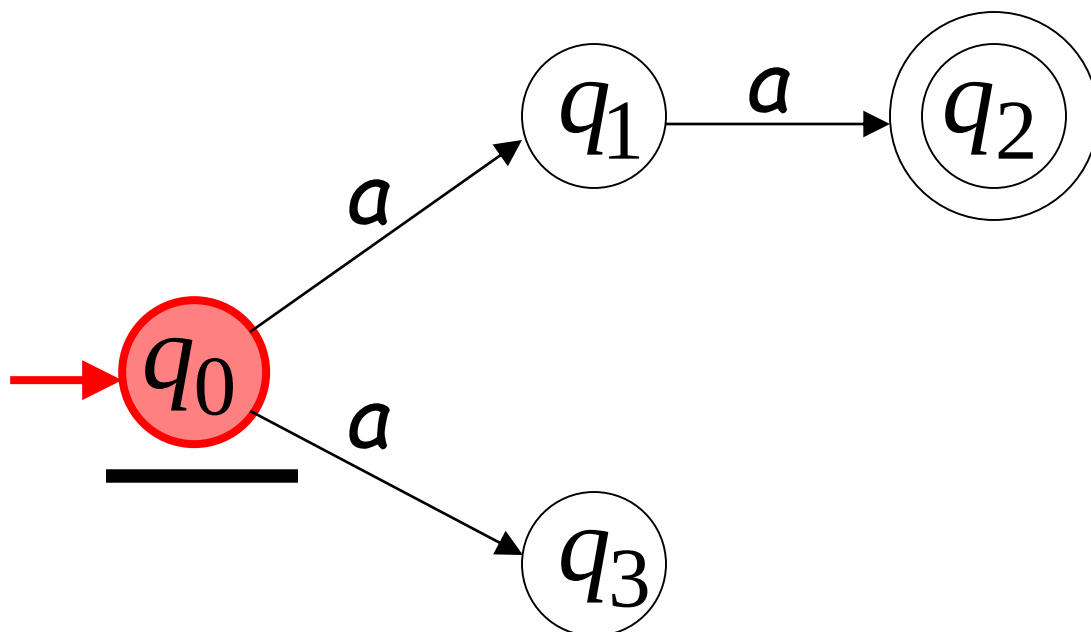
First Choice



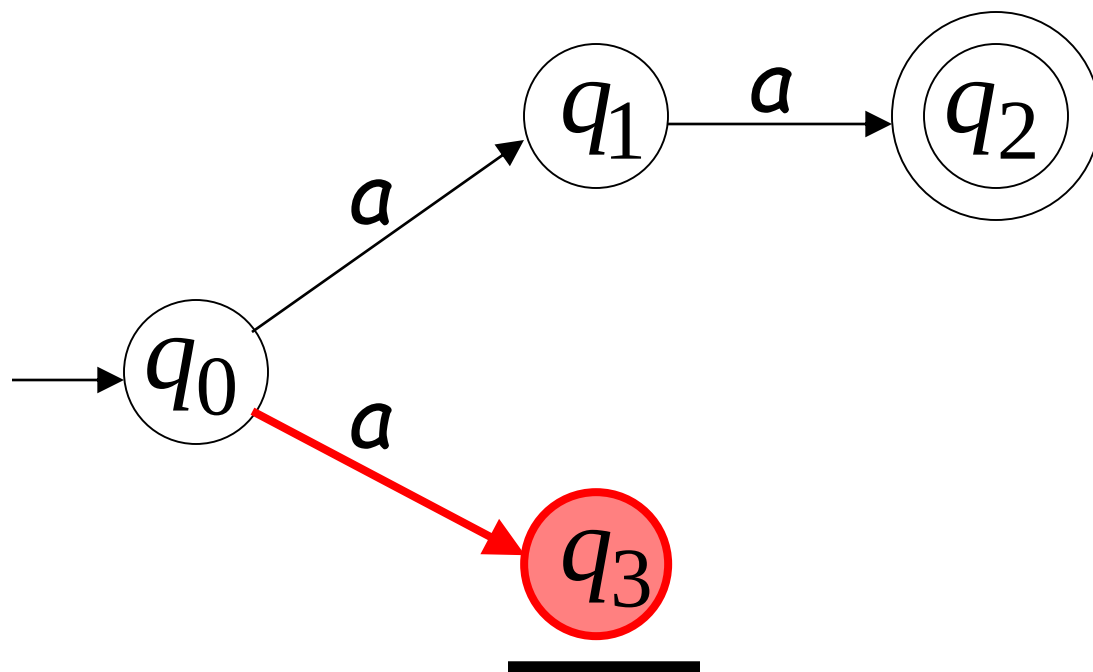
All input is consumed



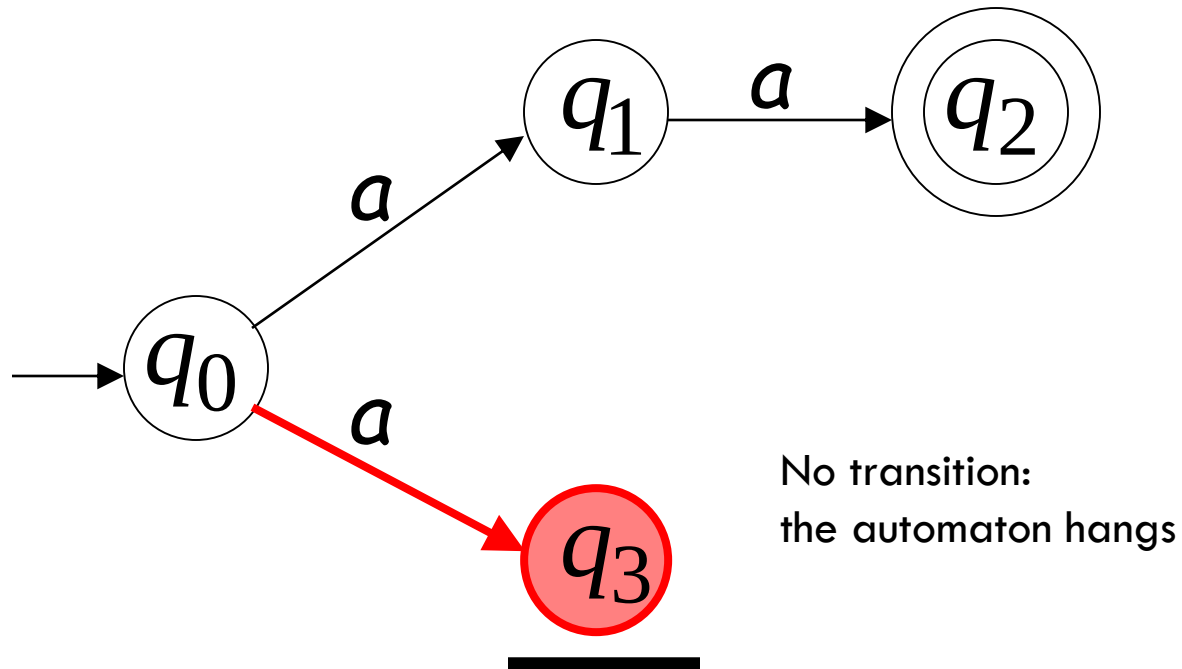
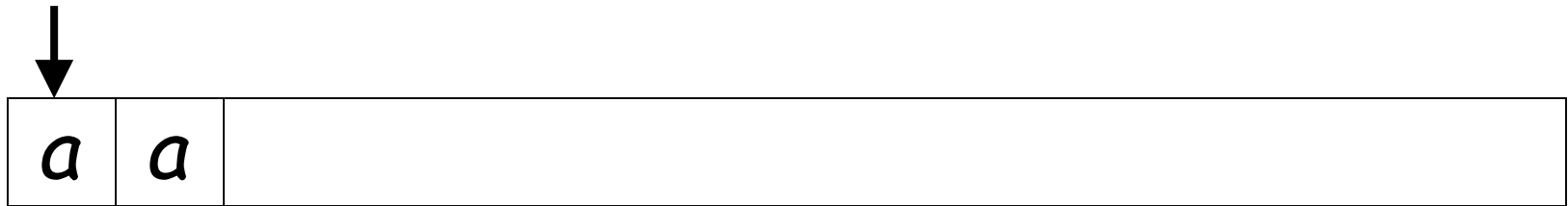
Second Choice



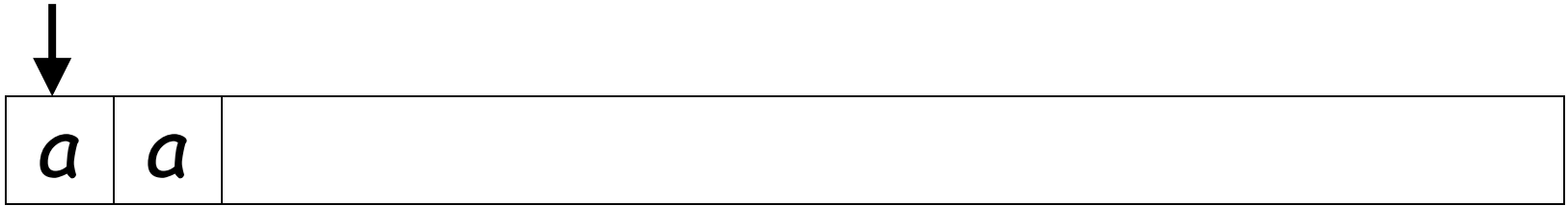
Second Choice



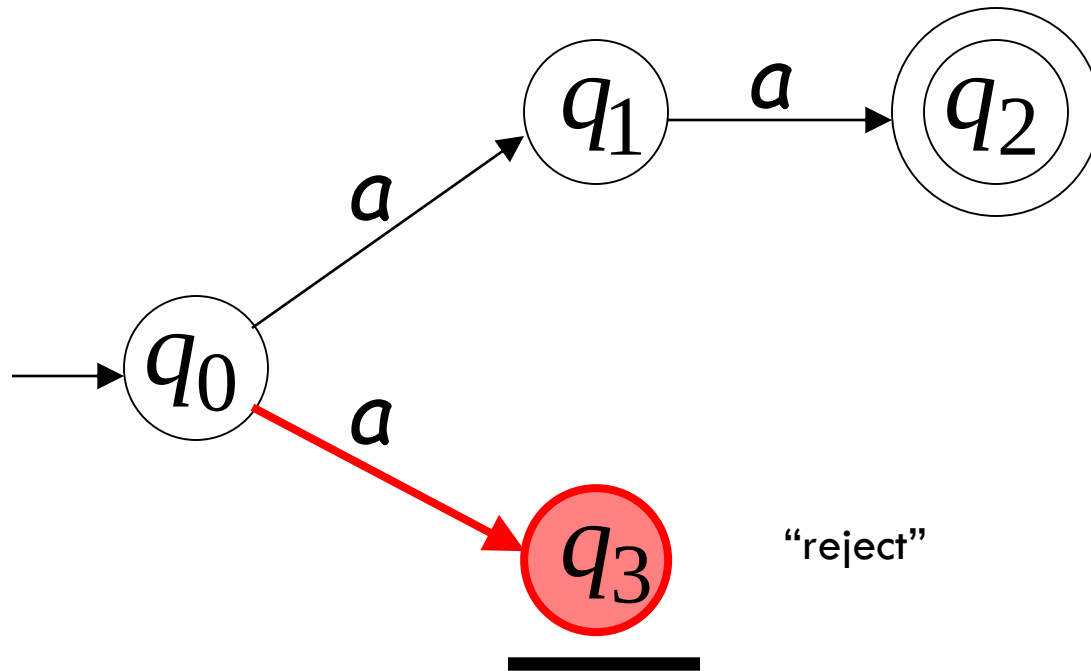
Second Choice



Second Choice

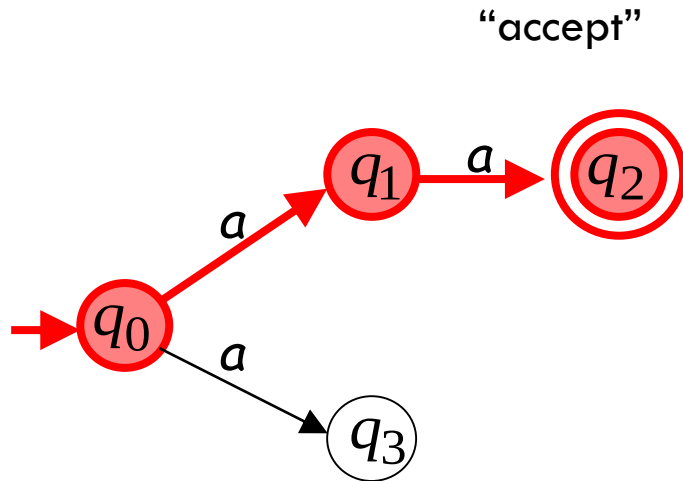


Input cannot be consumed



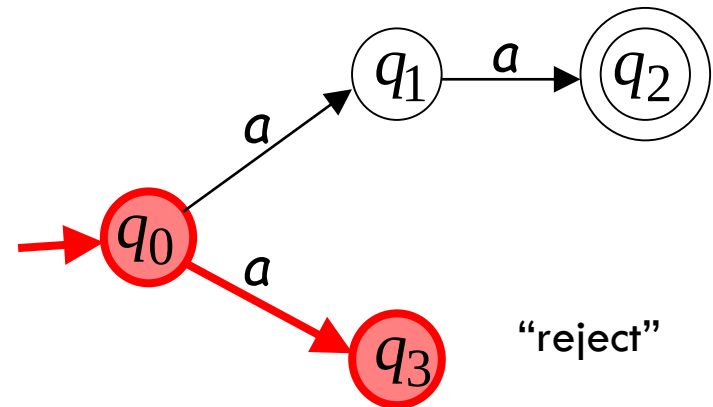
Example

aa is accepted by the NFA:



because this
computation
accepts

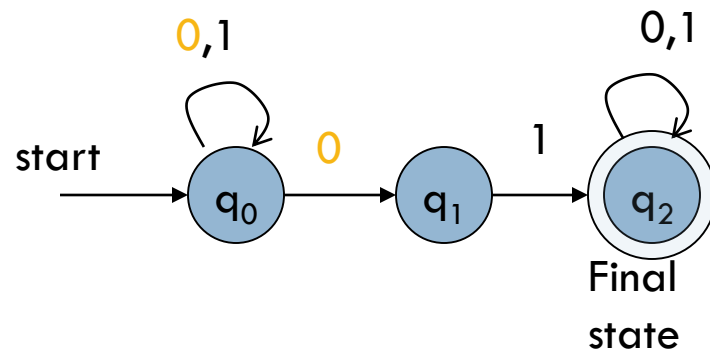
aa



“reject”

NFA for strings containing 01

Why is this non-deterministic?



What will happen if at state q_1 an input of 0 is received?

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{0, 1\}$
- start state = q_0
- $F = \{q_2\}$
- Transition table

		symbols	
δ		0	1
states	q_0	$\{q_0, q_1\}$	$\{q_0\}$
	q_1	Φ	$\{q_2\}$
	$*q_2$	$\{q_2\}$	$\{q_2\}$

Differences: DFA vs. NFA

□ DFA

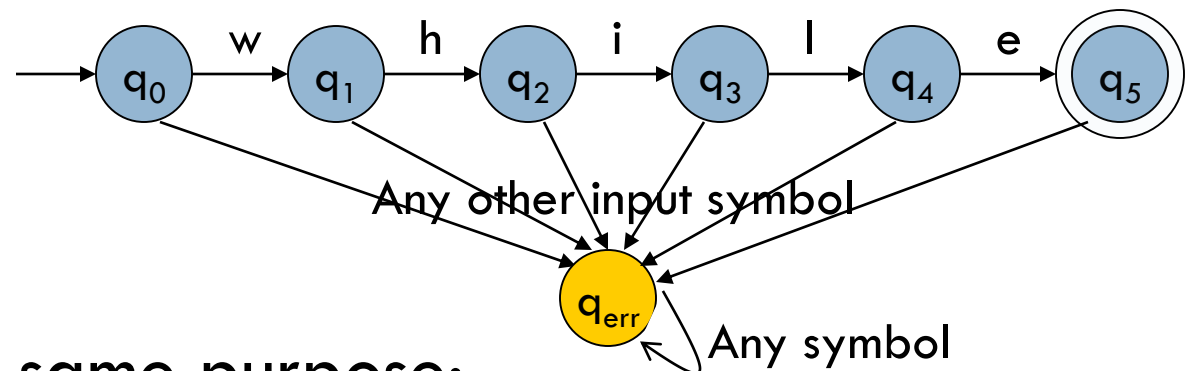
1. All transitions are deterministic
 - Each transition leads to exactly one state
2. For each state, transition on all possible symbols (alphabet) should be defined
3. Accepts input if the last state visited is in F
4. Sometimes harder to construct because of the number of states
5. $\delta: Q \times \Sigma \rightarrow Q$ is a transition function

□ NFA

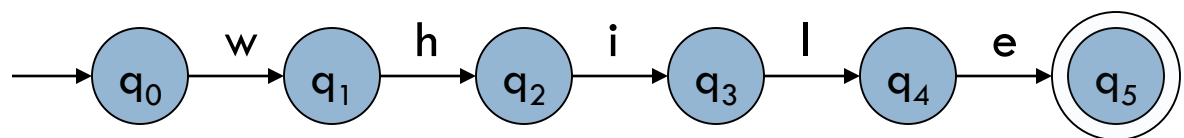
1. Some transitions could be non-deterministic
 - A transition could lead to a subset of states
2. Not all symbol transitions need to be defined explicitly (if undefined will go to an error state – this is just a design convenience, not to be confused with “non-determinism”)
3. Accepts input if *one of* the last states is in F
4. Generally easier than a DFA to construct
5. $Q \times \Sigma \rightarrow 2^Q$ [subset of Q]

What is an “error state” or dummy state?

- A DFA for recognizing the key word “while”



An NFA for the same purpose:

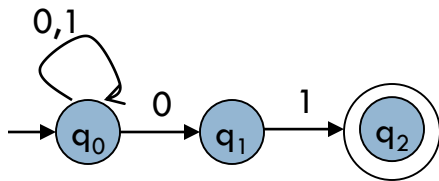


Transitions into a dead state are implicit

NFA to DFA construction: Example

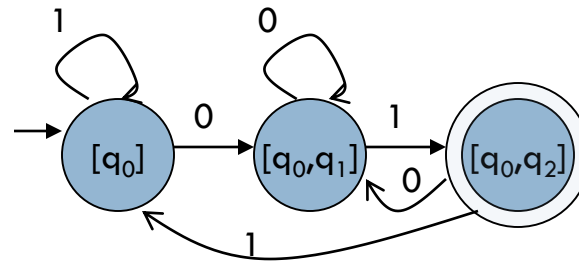
□ $L = \{w \mid w \text{ ends in } 01\}$

NFA:



δ_N	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

DFA:



δ_D	
$\emptyset$	
$\rightarrow [q_0]$	
$[q_1]$	
$*[q_2]$	
$[q_0, q_1]$	
$*[q_0, q_2]$	
$*[q_1, q_2]$	
$*[q_0, q_1, q_2]$	

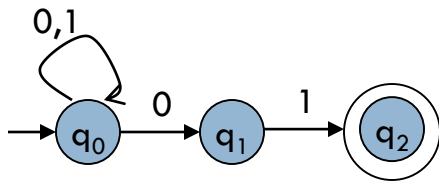
δ_D	0	1
$\rightarrow [q_0]$	$[q_0, q_1]$	$[q_0]$
$[q_0, q_1]$	$[q_0, q_1]$	$[q_0, q_2]$
$*[q_0, q_2]$	$[q_0, q_1]$	$[q_0]$

0. Enumerate all possible subsets
1. Determine transitions
2. Retain only those states reachable from $\{q_0\}$

NFA to DFA Contd...

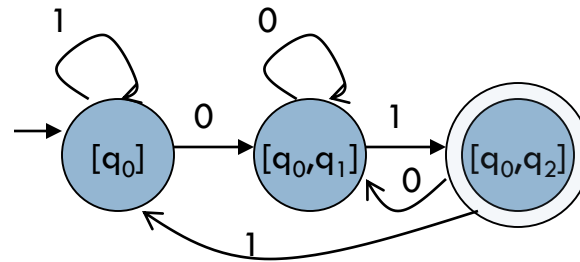
□ $L = \{w \mid w \text{ ends in } 01\}$

NFA:



δ_N	0	1
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

DFA:



δ_D	0	1
$[q_0]$	$[q_0, q_1]$	$[q_0]$

Main Idea:

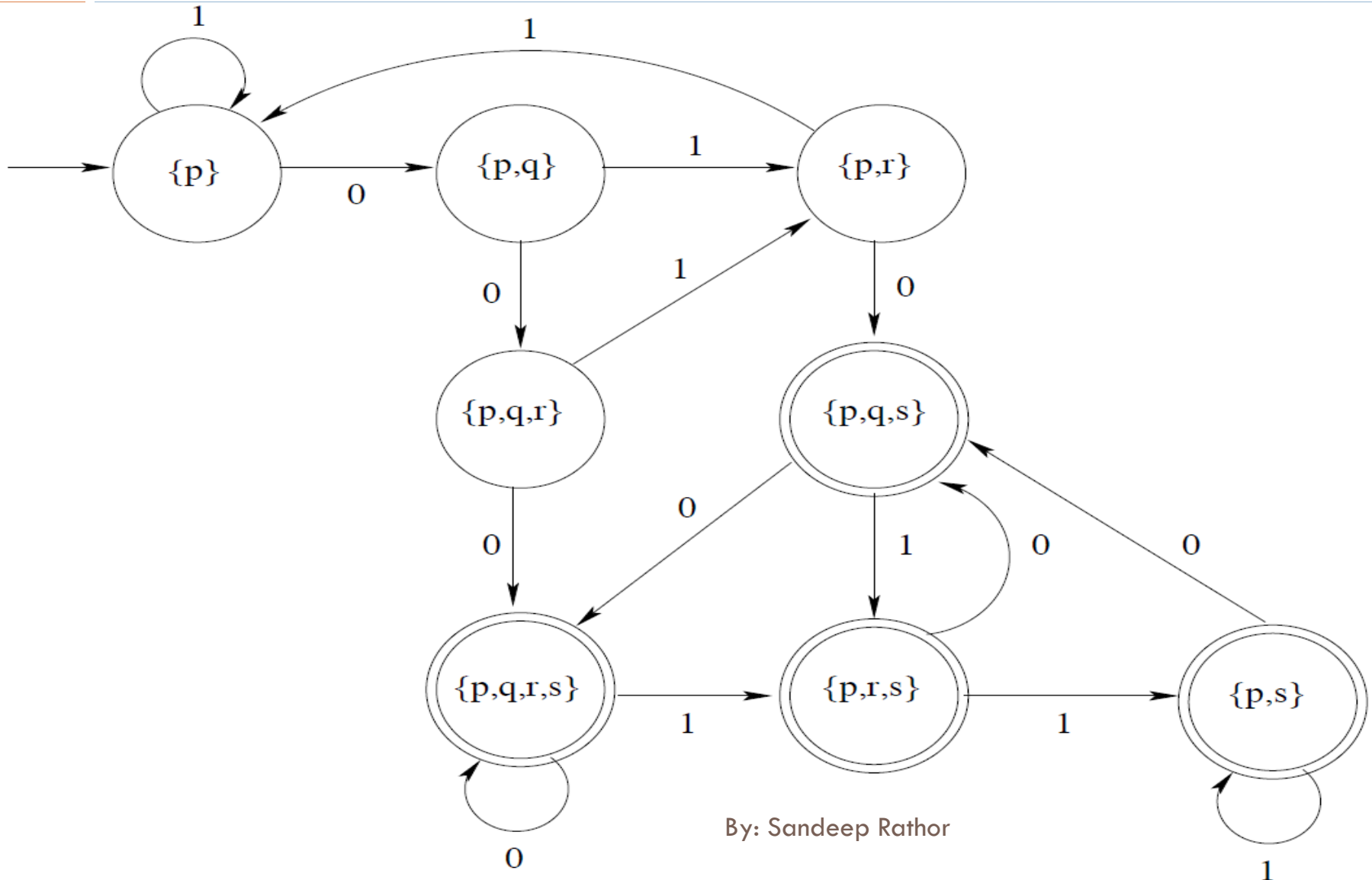
Introduce states as you go
(on a need basis)

Convert the given NFA to DFA

States	0	1
->[p]	[p,q]	[p]
[p,q]	[p,q,r]	[p,r]
[p,r]	[p,q,s]	[p]
[p,q,r]	[p,q,r,s]	[p,r]
[p,q,s]+	[p,q,r,s]	[p,r,s]
[p,r,s]+	[p,q,s]	[p,s]
[p,s]+	[p,q,s]	[p,s]
[p,q,r,s]+	[p,q,r,s]	[p,r,s]

	0	1
$\rightarrow p$	$\{p, q\}$	$\{p\}$
q	$\{r\}$	$\{r\}$
r	$\{s\}$	$\{\}$
$*s$	$\{s\}$	$\{s\}$

Transition diagram





Practice Session: NFA to DFA

By: Sandeep Rathor

1. Convert into DFA

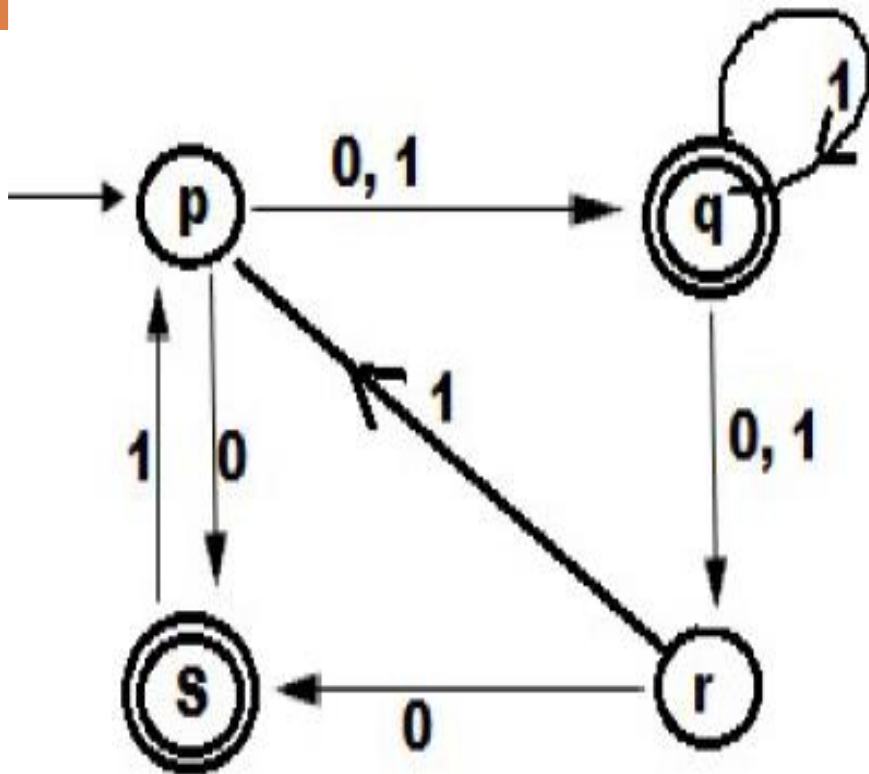
States	0	1
->p+	p	q
q	q	p,q

DFA equivalent to given NFA

States	0	1
->[p]+	[p]	[q]
[q]	[q]	[p,q]
[p,q] +	[p,q]	[p,q]

By: Sandeep Rathor

2. Convert given NFA to DFA



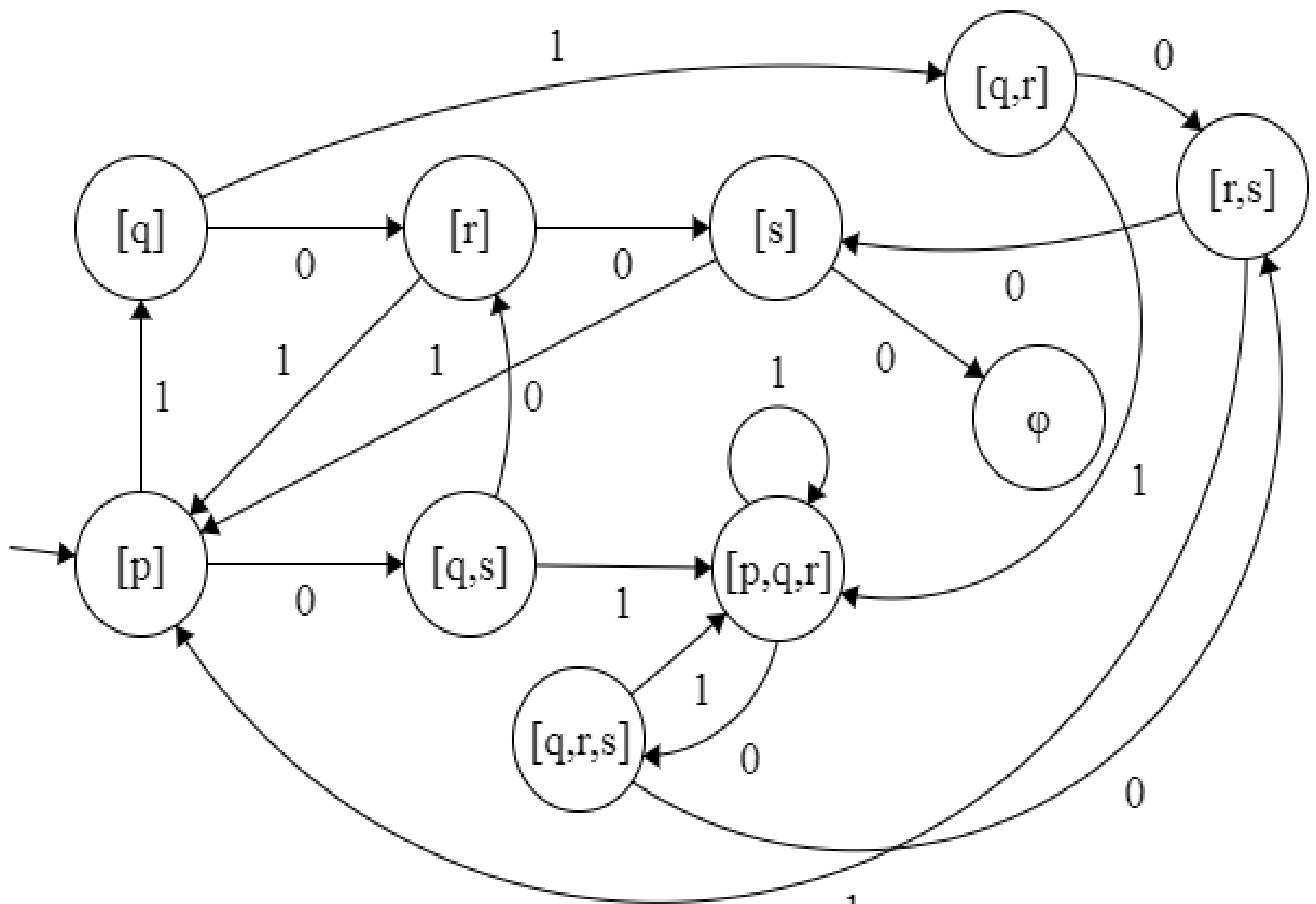
	0	1
$\rightarrow p$	q, s	q
$*q$	r	q, r
r	s	p
$*s$	-	p

Required DFA

State/ Input	0	1
→[p]	[q, s]	[q]
*[q]	[r]	[q, r]
[r]	[s]	[p]
*[s]	φ	[p]
*[q, r]	[r, s]	[p, q, r]
*[q, s]	[r]	[p, q, r]
*[r, s]	[s]	[p]
*[p, q, r]	[q, r, s]	[p, q, r]
*[q, r, s]	[r, s]	[p, q, r]

Transition Diagram...

By: Sandeep Rathor



Practice Contd...

3. Convert given NFA to DFA

States/input	0	1
->P+ (Final)	Q, S	Q
Q+ (Final)	R	R,Q
R	S	S
S	-	P

By: Sandeep Rathor

Answer: Required DFA

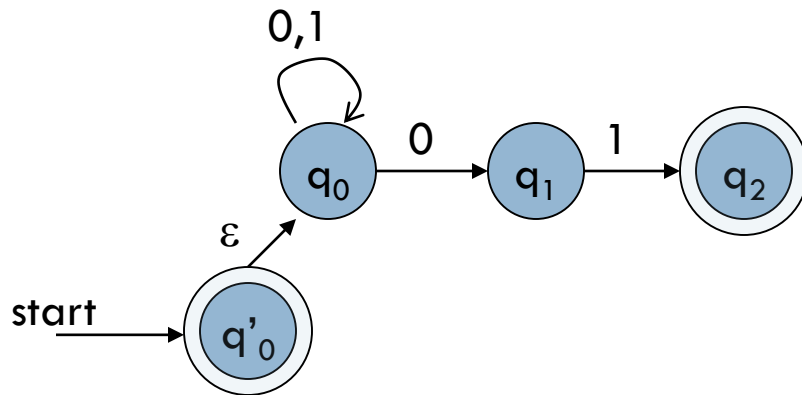
State/ Input	0	1
 [P]+	[Q,S]	[Q]
[Q]+	[R]	[R,Q]
[R]	[S]	[S]
[S]	-	[P]
[Q,S] +	[R]	[P,Q,R]
[R,Q] +	[R,S]	[Q,R,S]
[R,S]	[S]	[P,S]
[P,S]+	[Q,S]	[P,Q]
[P,Q] +	[Q,R,S]	[R,Q]
[P,Q,R] +	[Q,R,S]	[Q,R,S]
[Q,R,S] +	[R,S]	[P,Q,R,S]
[P,Q,R,S] +	[Q,R,S]	[P,Q,R,S]

FA with ϵ -Transitions

- **ϵ -NFAs** are those NFAs with at least one explicit ϵ -transition defined.
- Explicit ϵ -transitions is transition from one state to another state without consuming any additional input symbol

Example of an ε -NFA

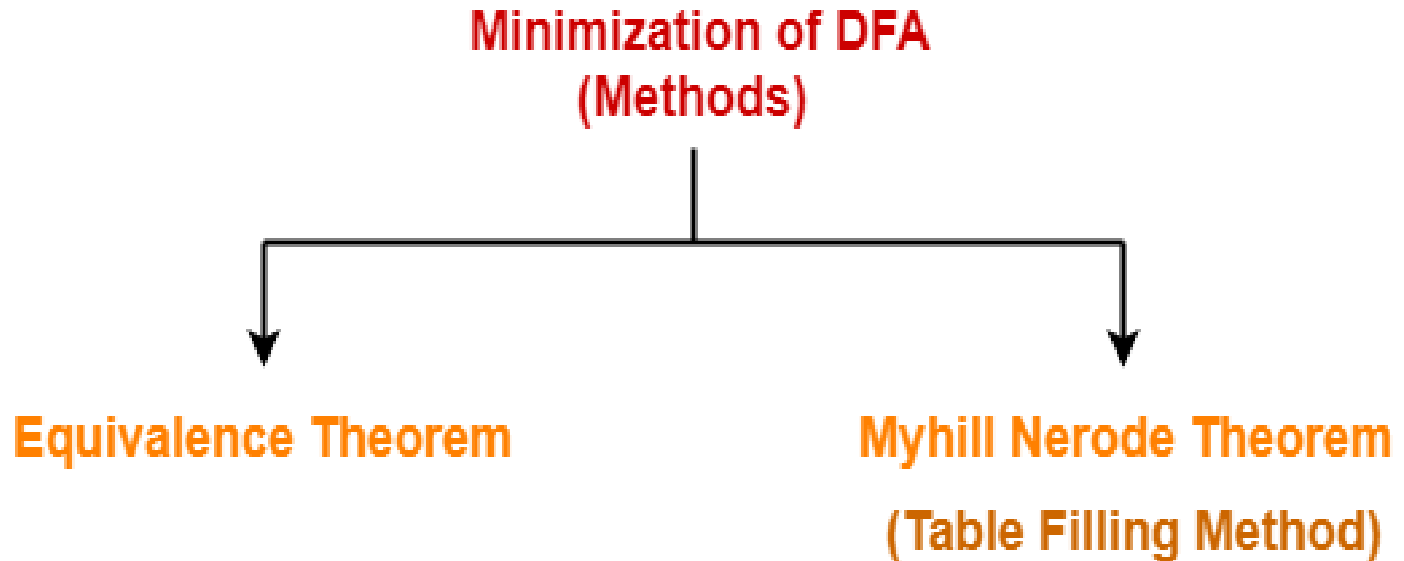
$L = \{w \mid w \text{ is empty, or if non-empty will end in } 01\}$



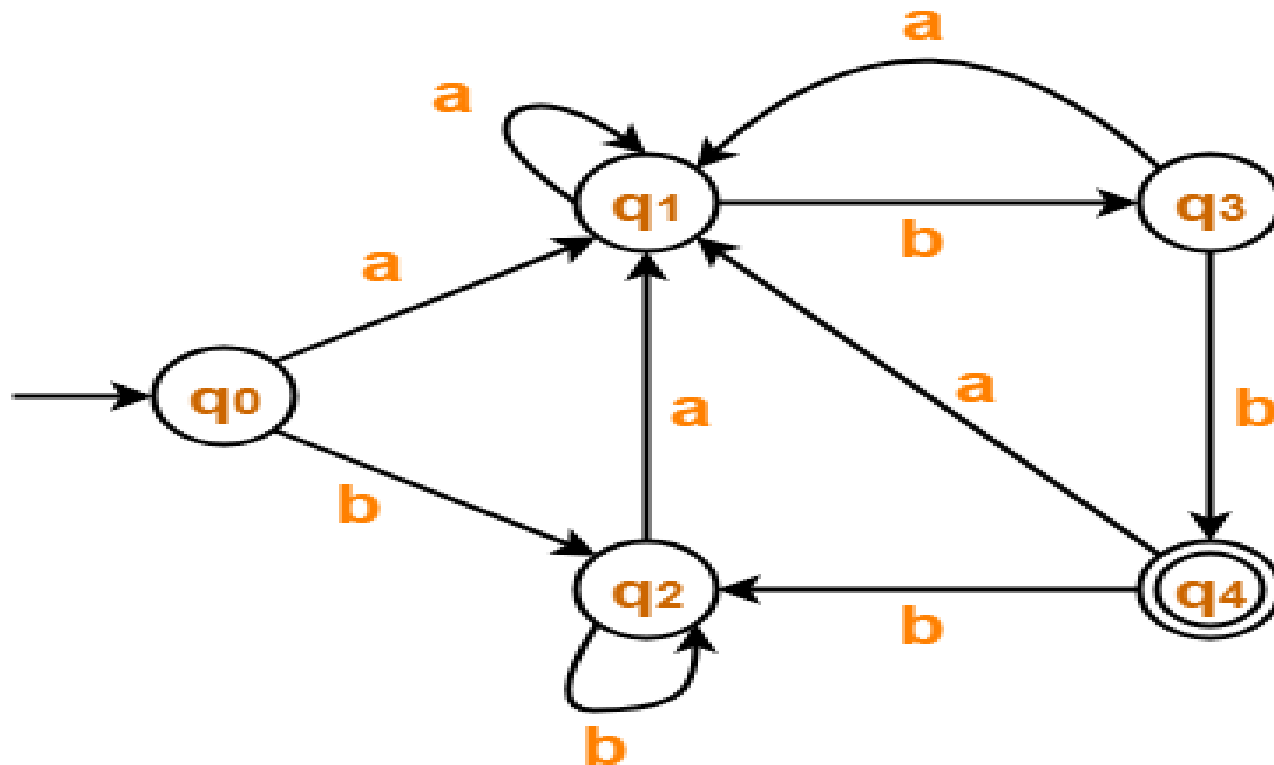
- ε -closure of a state q , **ECLOSE(q)**, is the set of all states (including itself) that can be reached from q by repeatedly making an arbitrary number of ε -transitions.

δ_E	0	1	ε
$\rightarrow *q'_0$	$\emptyset$	$\emptyset$	$\{q'_0, q_0\}$
q_0	$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$	$\{q_1\}$
$*q_2$	$\emptyset$	$\emptyset$	$\{q_2\}$

Minimization of DFA



Minimized the given DFA...



Transition table as per given diagram

	a	b
→q0	q1	q2
q1	q1	q3
q2	q1	q2
q3	q1	*q4
*q4	q1	q2

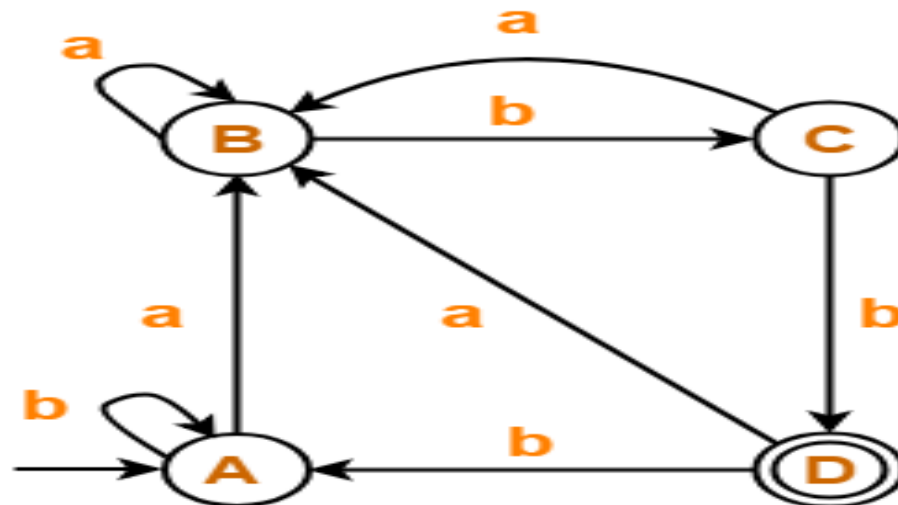
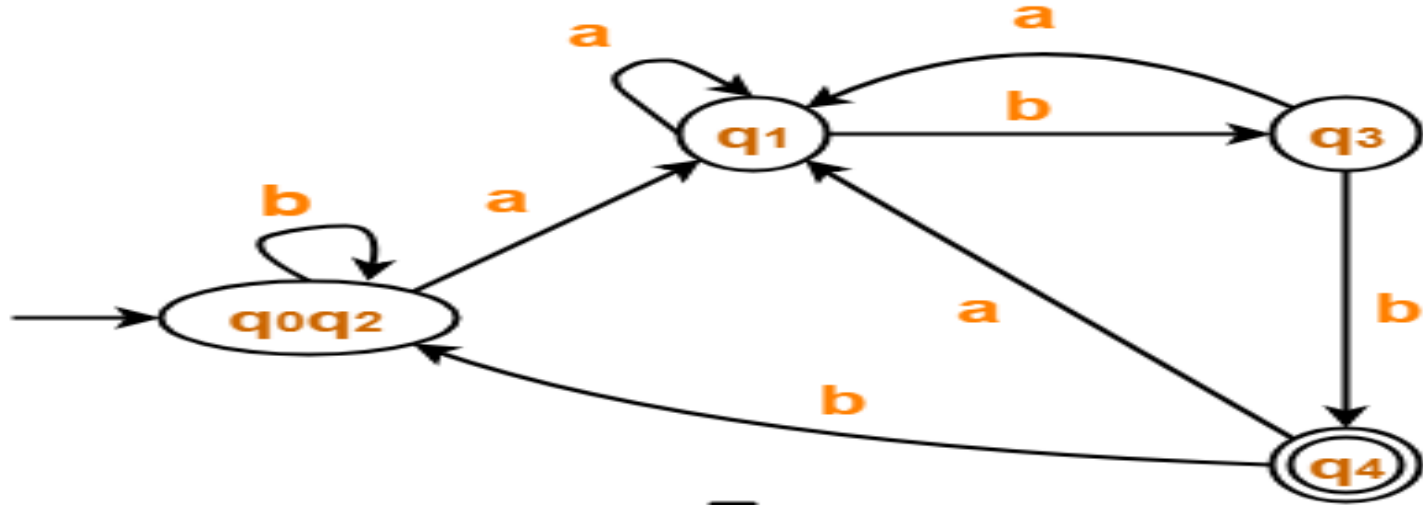
Now using Equivalence Theorem, we have-

$$\begin{aligned}\pi_0 &= \{q_0, q_1, q_2, q_3\} \{q_4\} \\ \pi_1 &= \{q_0, q_1, q_2\} \{q_3\} \{q_4\} \\ \pi_2 &= \{q_0, q_2\} \{q_1\} \{q_3\} \{q_4\} \\ \pi_3 &= \{q_0, q_2\} \{q_1\} \{q_3\} \{q_4\}\end{aligned}$$

By: Sandeep Rathor

Since $\pi_3 = \pi_2$, so we stop.

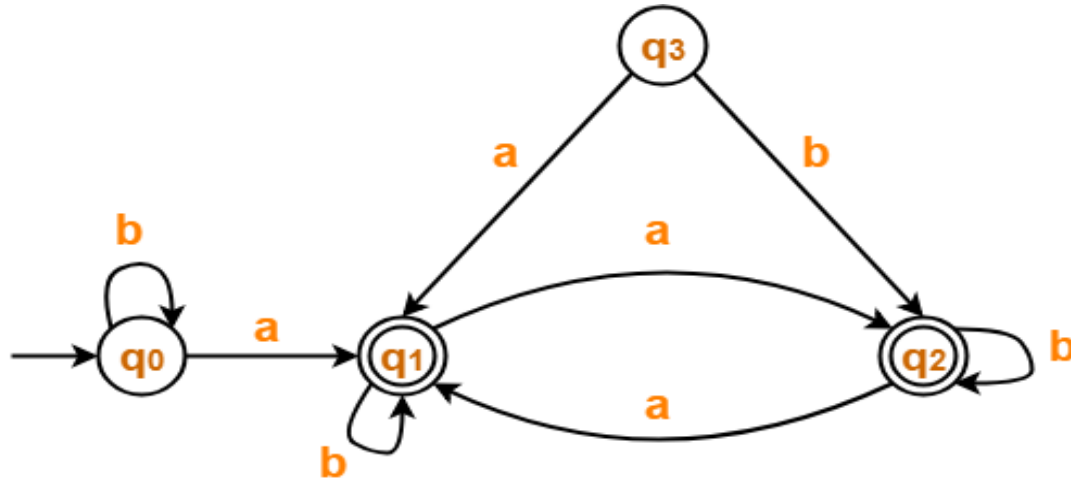
Minimized DFA



By Sandeep Rathor

Minimal DFA

Ex-2, Minimization contd...



Transition Table

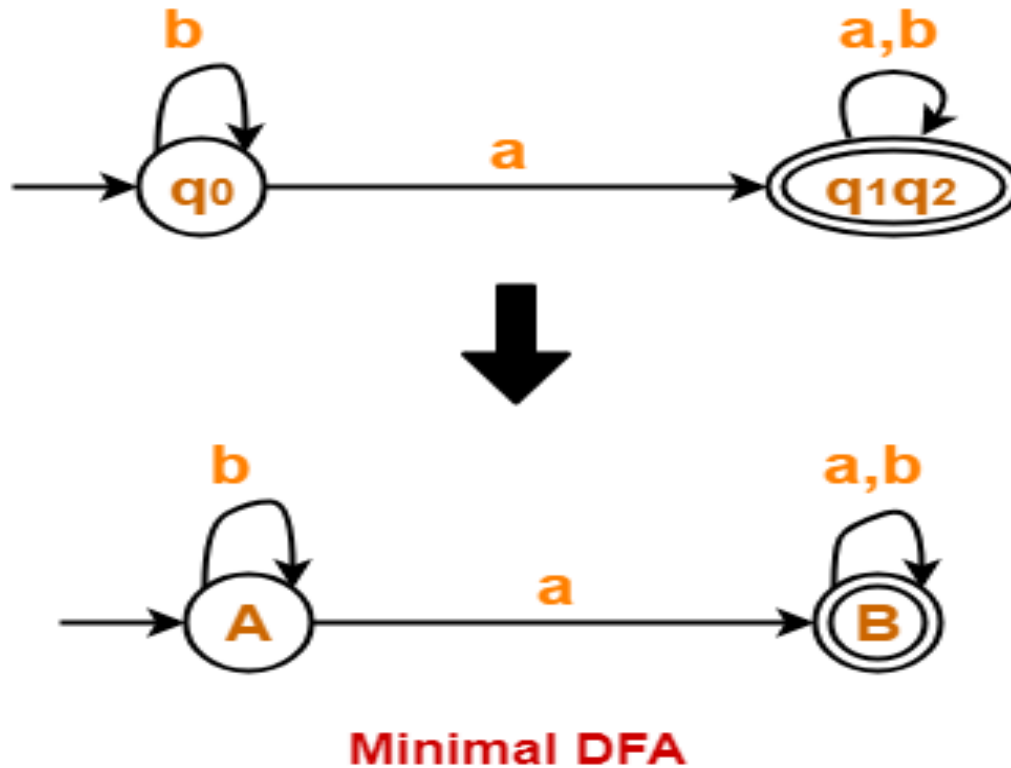
	a	b
→q0	*q1	q0
*q1	*q2	*q1
*q2	*q1	*q2

$$P_0 = \{ q_0 \} \{ q_1, q_2 \}$$

$$P_1 = \{ q_0 \} \{ q_1, q_2 \}$$

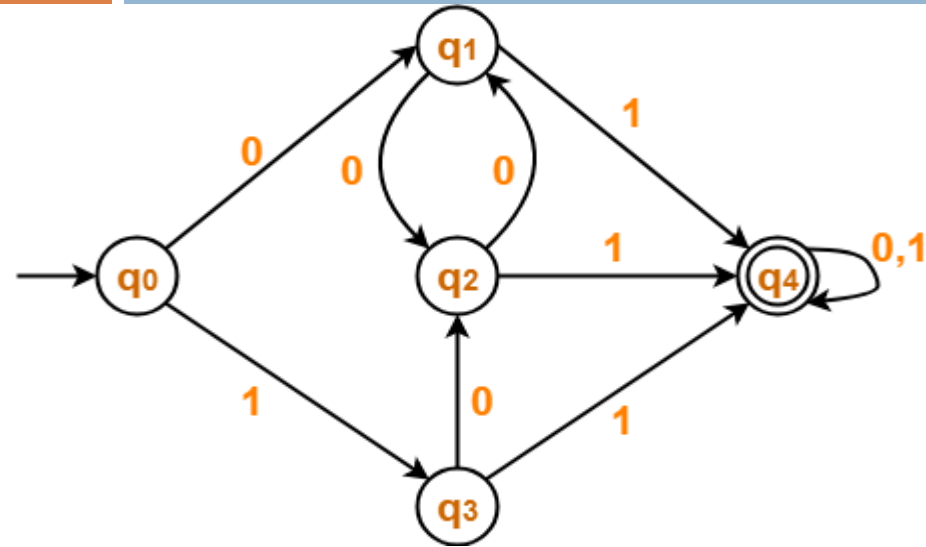
Since $P_1 = P_0$, so we stop.

Minimized automata...



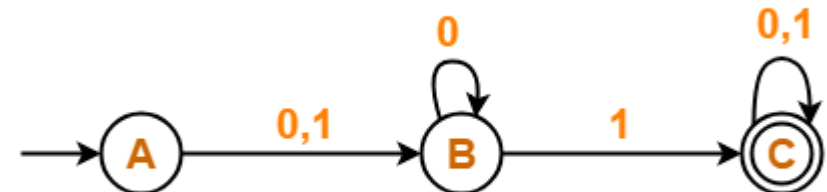
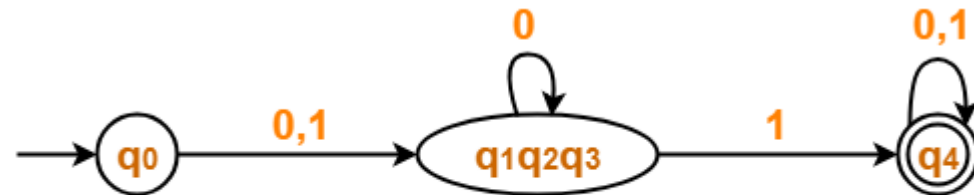
Minimized the following:

Example-3:



$$P_0 = \{ q_0, q_1, q_2, q_3 \} \{ q_4 \}$$
$$P_1 = \{ q_0 \} \{ q_1, q_2, q_3 \} \{ q_4 \}$$
$$P_2 = \{ q_0 \} \{ q_1, q_2, q_3 \} \{ q_4 \}$$

Minimized



By: Sandeep Rathor

Minimal DFA

Example-4:

State q_5 is inaccessible from the initial state.
So, we eliminate it and its associated edges from the DFA.

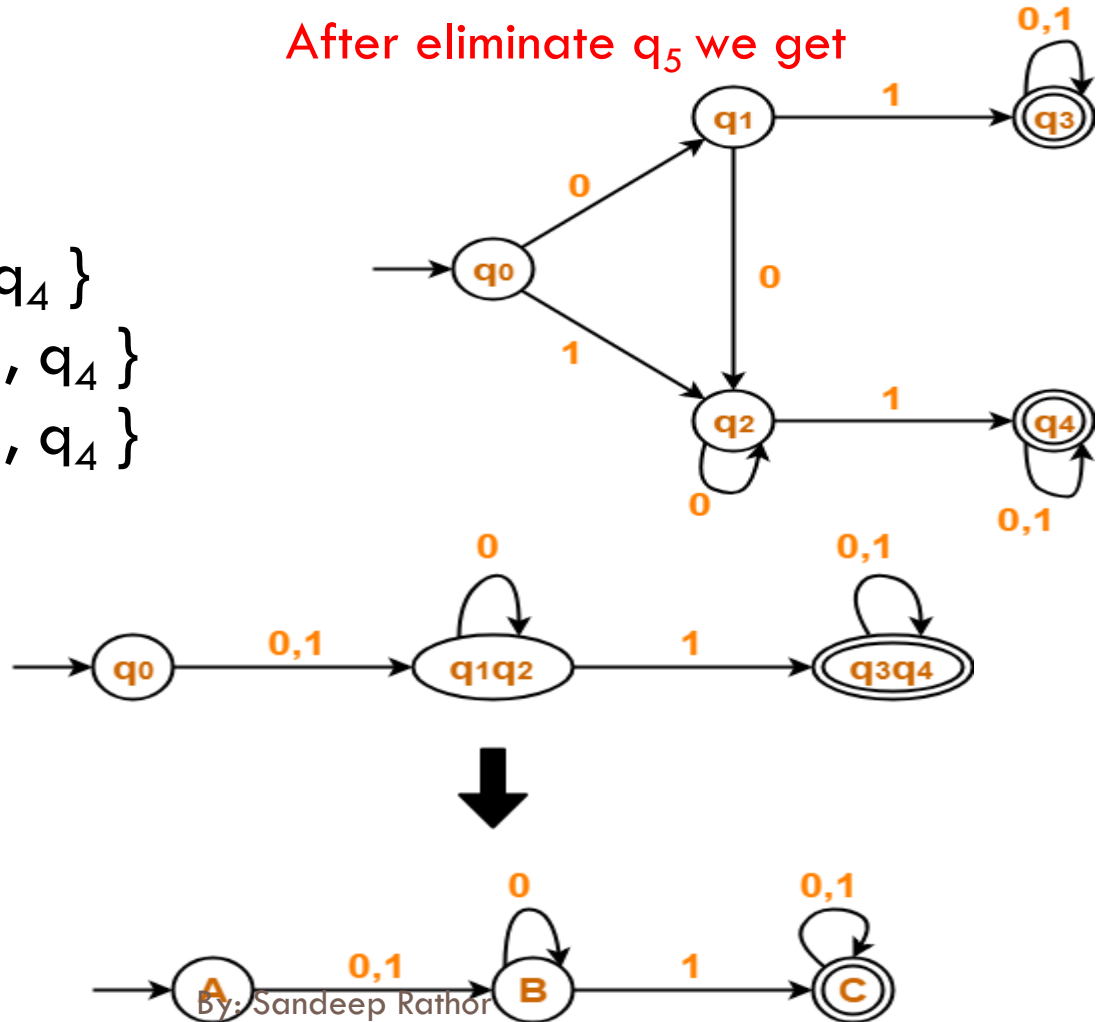
After eliminate q_5 we get

$$P_0 = \{q_0, q_1, q_2\} \{q_3, q_4\}$$

$$P_1 = \{q_0\} \{q_1, q_2\} \{q_3, q_4\}$$

$$P_2 = \{q_0\} \{q_1, q_2\} \{q_3, q_4\}$$

Since $P_2 = P_1$, so we st



Minimal DFA

Example-5 for Practice...

Minimize the given Automata

States/input	0	1
->q ₀	q1	q5
q1	q6	q2
q2+ (Final)	q0	q2
q3	q2	q6
q4	q7	q5
q5	q2	q6
q6	q6	q4
q7	q6	q2

By: Sandeep Rathor

Minimizing (Using Equivalence theorem)

$$\pi_0 = \{\{q_2\}, \{q_0, q_1, q_3, q_4, q_5, q_6, q_7\}\}$$

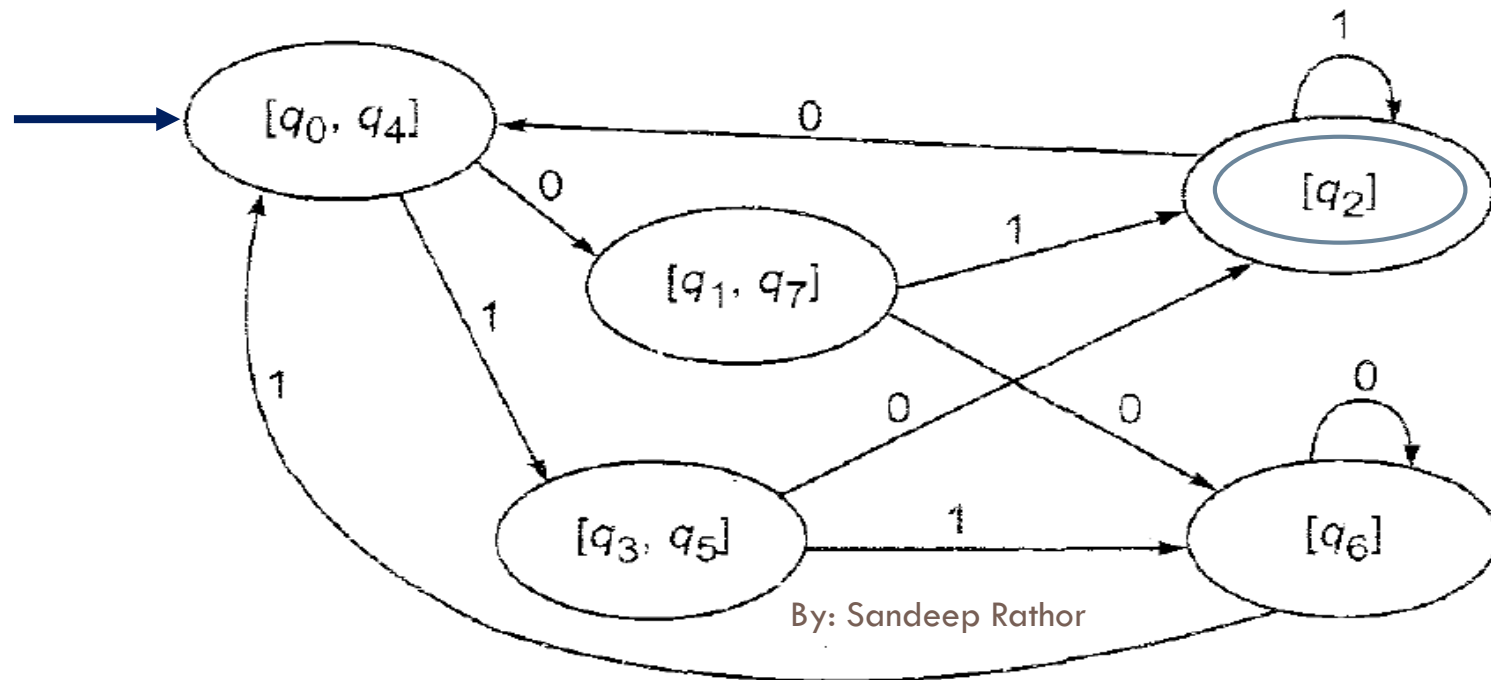
$$\pi_1 = \{\{q_2\}, \{q_0, q_4, q_6\}, \{q_1, q_7\}, \{q_3, q_5\}\}$$

$$\pi_2 = \{\{q_2\}, \{q_0, q_4\}, \{q_6\}, \{q_1, q_7\}, \{q_3, q_5\}\}$$

$$\pi_3 = \{\{q_2\}, \{q_0, q_4\}, \{q_6\}, \{q_1, q_7\}, \{q_3, q_5\}\}$$

$$\pi_2 = \pi_3,$$

State/ Σ	0	1
$\rightarrow [q_0, q_4]$	$[q_1, q_7]$	$[q_3, q_5]$
$[q_1, q_7]$	$[q_6]$	$[q_2]$
$[q_2]$	$[q_0, q_4]$	$[q_2]$
$[q_3, q_5]$	$[q_2]$	$[q_6]$
$[q_6]$	$[q_6]$	$[q_0, q_4]$



Example-6 for Practice...

Minimize the given Automata

States/input	0	1
->q ₀	q ₁	q ₀
q ₁	q ₀	q ₂
q ₂	q ₃	q ₁
q ₃ + (Final)	q ₃	q ₀
q ₄	q ₃	q ₅
q ₅	q ₆	q ₄
q ₆	q ₅	q ₆

Minimizing (Using Equivalence theorem)

$$\pi_0 = \{\{q_3\}, \{q_0, q_1, q_2, q_4, q_5, q_6\}$$

$$\pi_1 = \{\{q_3\}, \{q_0, q_1, q_5, q_6\}, \{q_2, q_4\}$$

$$\pi_2 = \{\{q_3\}, \{q_0, q_6\}, \{q_1, q_5\}, \{q_2, q_4\}$$

$$\pi_3 = \{\{q_3\}, \{q_0, q_6\}, \{q_1, q_5\}, \{q_2, q_4\}$$

$$\pi_3 = \pi_2$$

State/ Σ

a

b

$[q_0, q_6]$

$[q_1, q_5]$

$[q_0, q_6]$

$[q_1, q_5]$

$[q_0, q_6]$

$[q_2, q_4]$

$[q_2, q_4]$

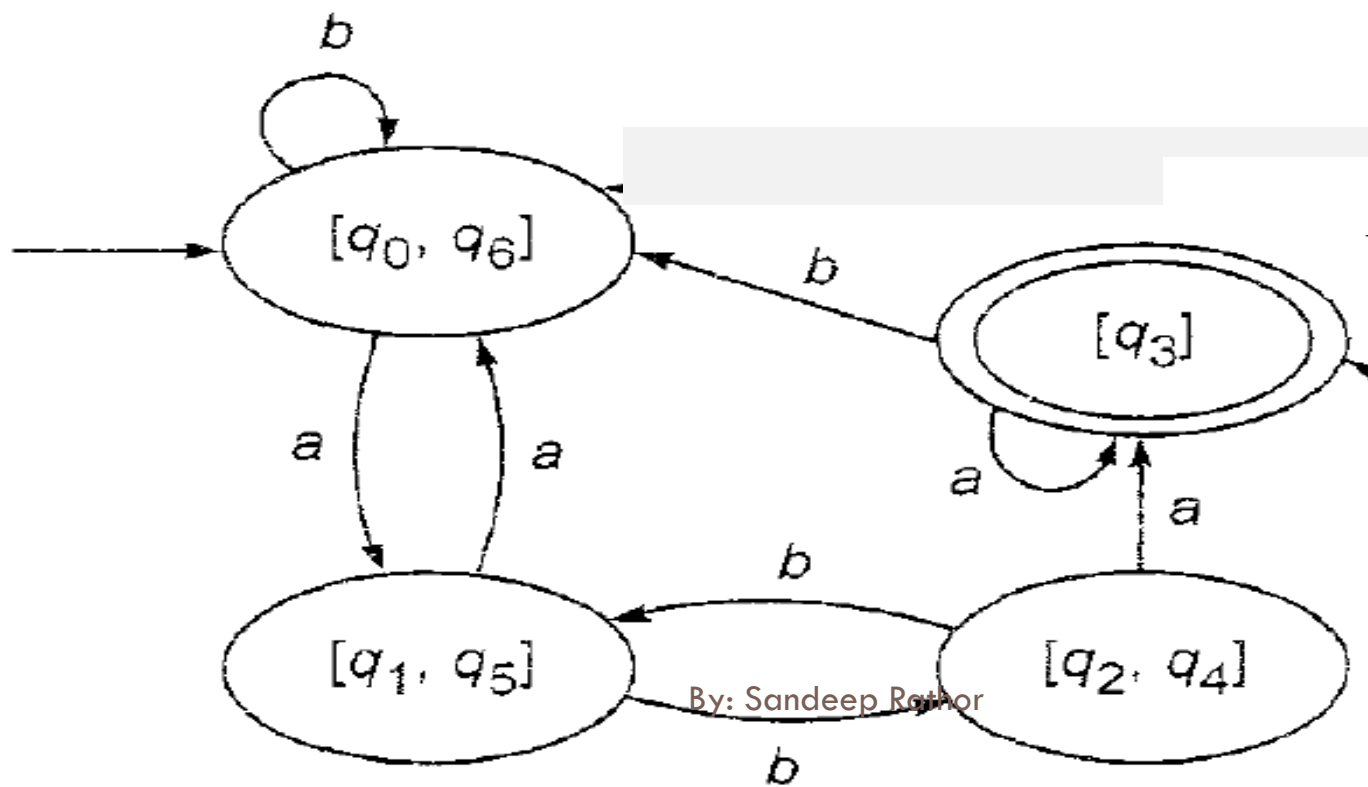
$[q_3]$

$[q_1, q_5]$

$[q_3]$

$[q_3]$

$[q_0, q_6]$



Finite Automata with Output

DFA, NFA, λ -NFA are FA without outputs (language acceptors)

Language transducers: Produces output on input

Finite automata may have outputs corresponding to each transition.

There are two types of finite state machines that generate output –

- **Moore Machine**

- **Mealy Machine**

Moore Machine

A Moore machine can be described by a 6 tuple

i.e. $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$ where –

Q is a finite set of states.

Σ is a finite set of symbols called the input alphabet.

Δ is a finite set of symbols called the output alphabet.

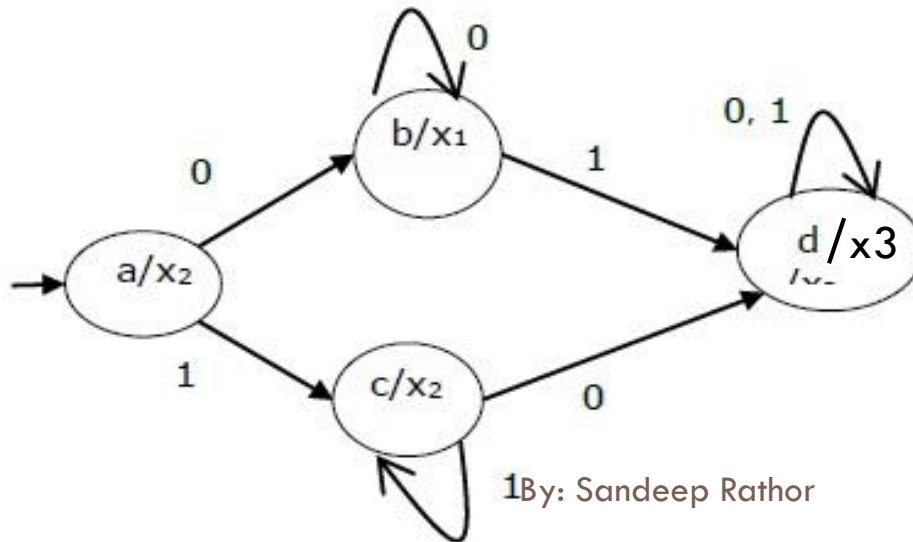
δ is the input transition function where $\delta: Q \times \Sigma \rightarrow Q$

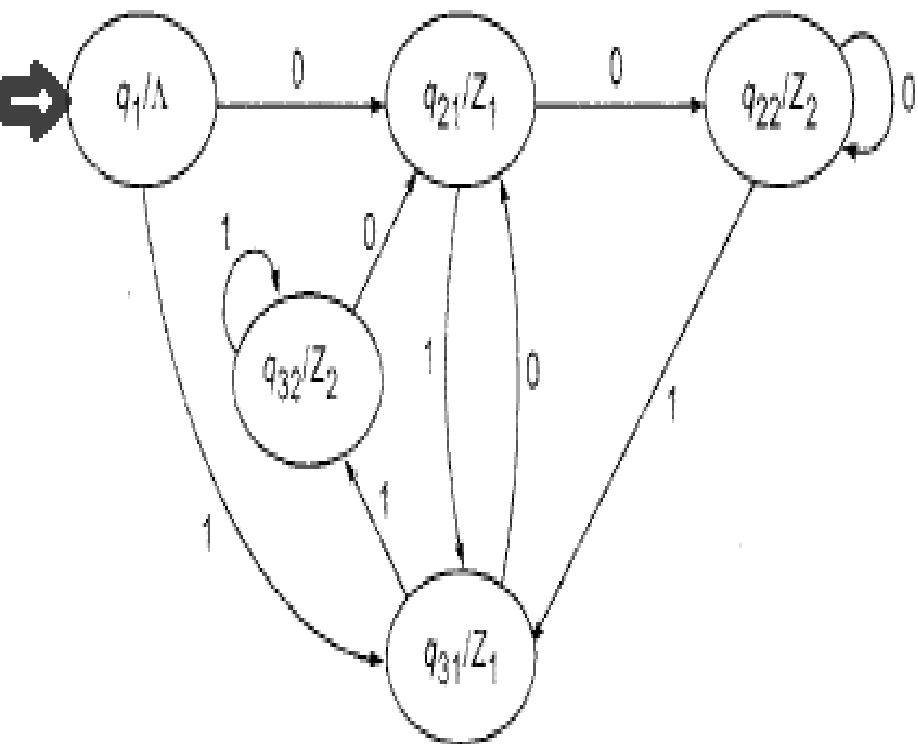
λ is the output function where $\lambda: Q \rightarrow \Delta$

q_0 is the initial state ($q_0 \in Q$).

Example of Moore Machine

Present state	Next State		Output
	Input = 0	Input = 1	
→ a	b	c	x2
b	b	d	x1
c	c	d	x2
d	d	d	x3





Present State	Next State	Output
---------------	------------	--------

	0	1	Δ
--	---	---	----------

$\rightarrow q0$	q21	q31	Λ
------------------	-----	-----	-----------

q21	q22	q31	Z1
-----	-----	-----	----

q22	q22	q31	Z2
-----	-----	-----	----

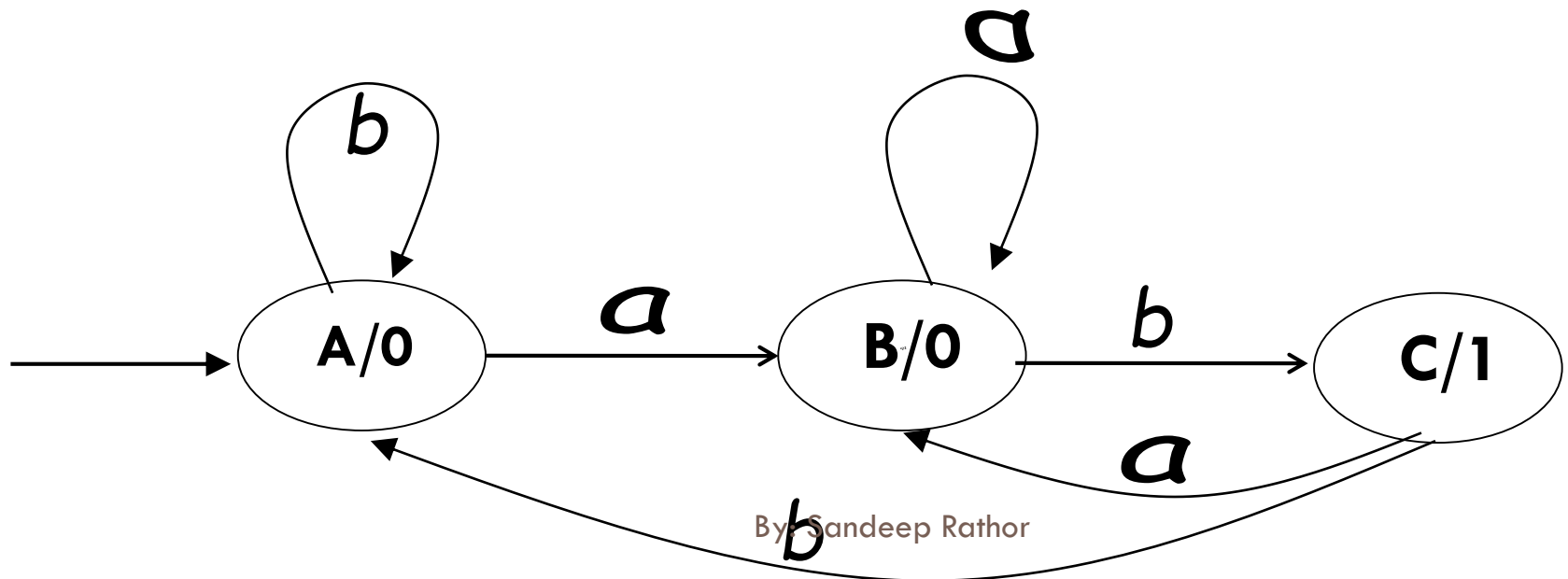
q31	q21	q32	Z1
-----	-----	-----	----

q32	q21	q32	Z2
-----	-----	-----	----

Construction of Moore Machine

Question1: Construct a Moore machine that takes set of all strings $\{a,b\}$ as input and prints '1' as output for every occurrence of 'ab' as substring.

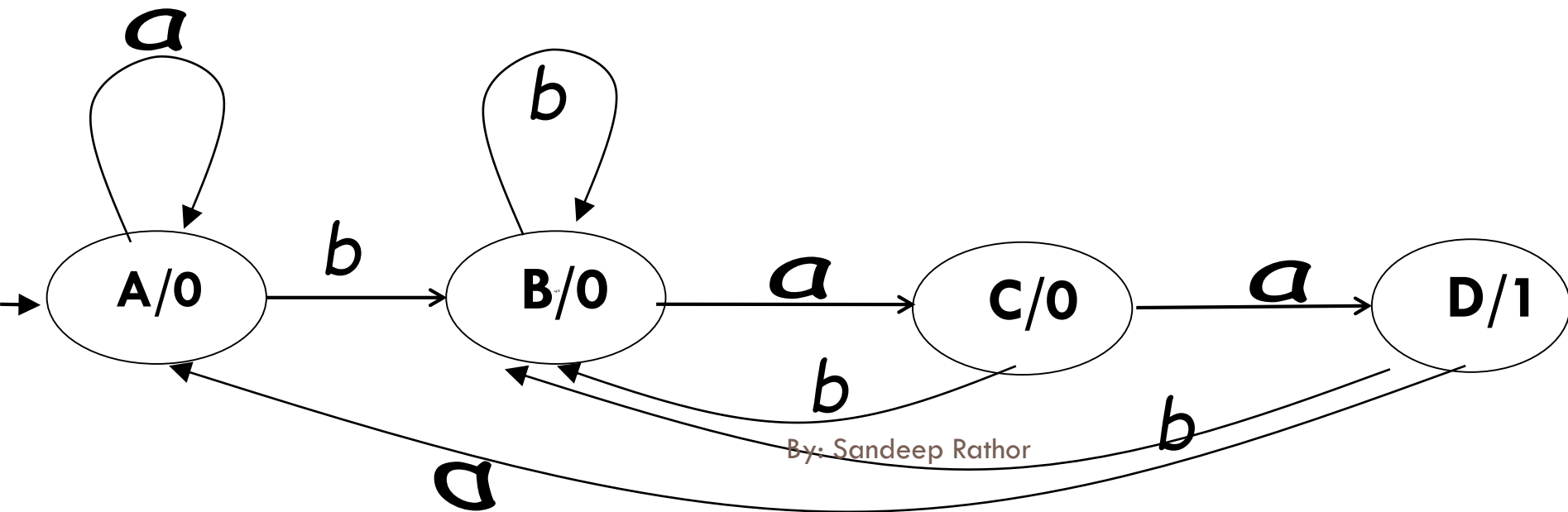
Solution: $\Sigma = \{a,b\}$
 $\Delta = \{0,1\}$



Construction of Moore Machine Contd...

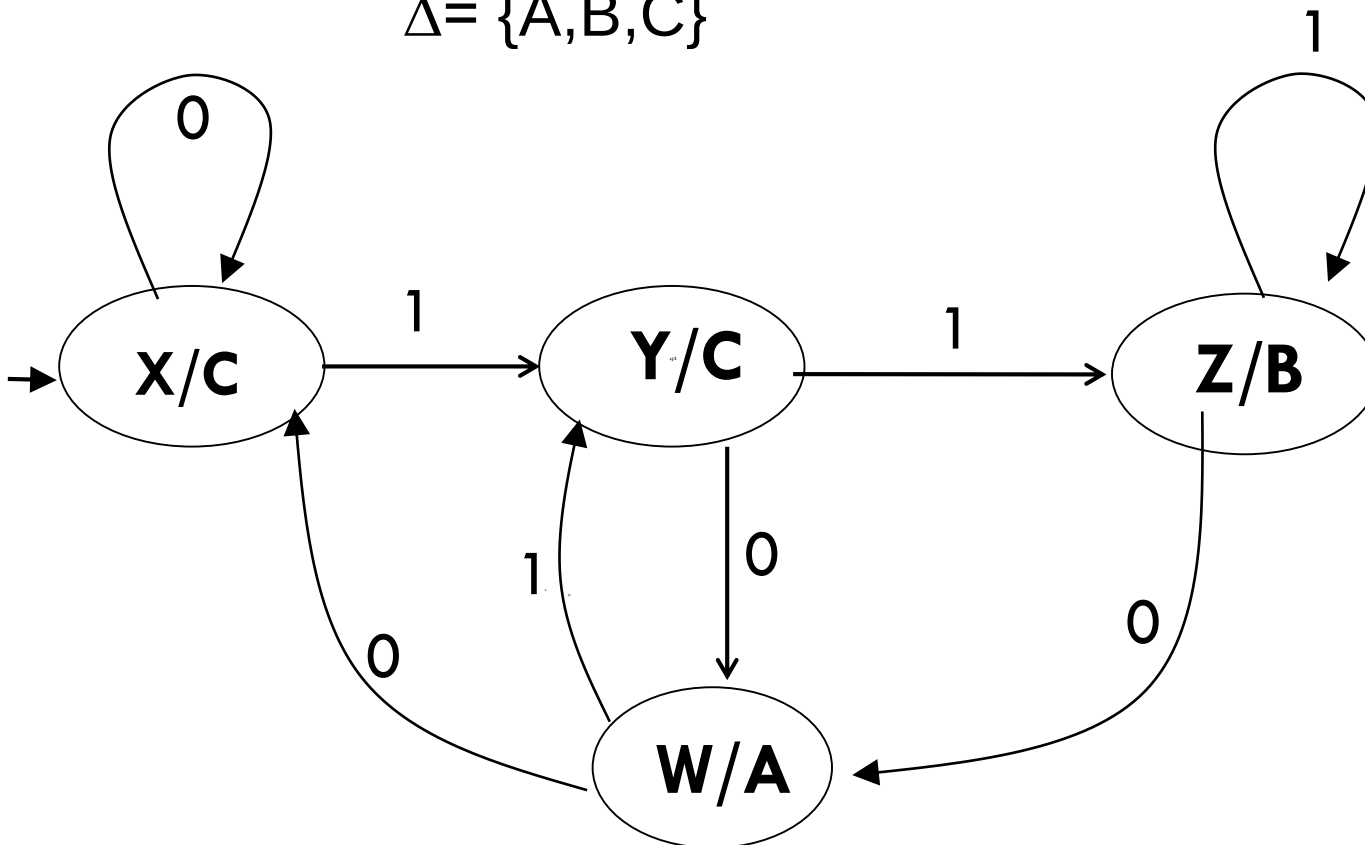
Question2: Construct a Moore machine that takes set of all strings over $\{a,b\}$ and counts no. of occurrences of substring 'baa'.

Solution: $\Sigma = \{a,b\}$
 $\Delta = \{0,1\}$



Question3: Construct a Moore machine that takes set of all strings over $\{0,1\}$ and produce 'A' as output if input ends with '10' or produces 'B' as output if ends with '11' otherwise 'C'.

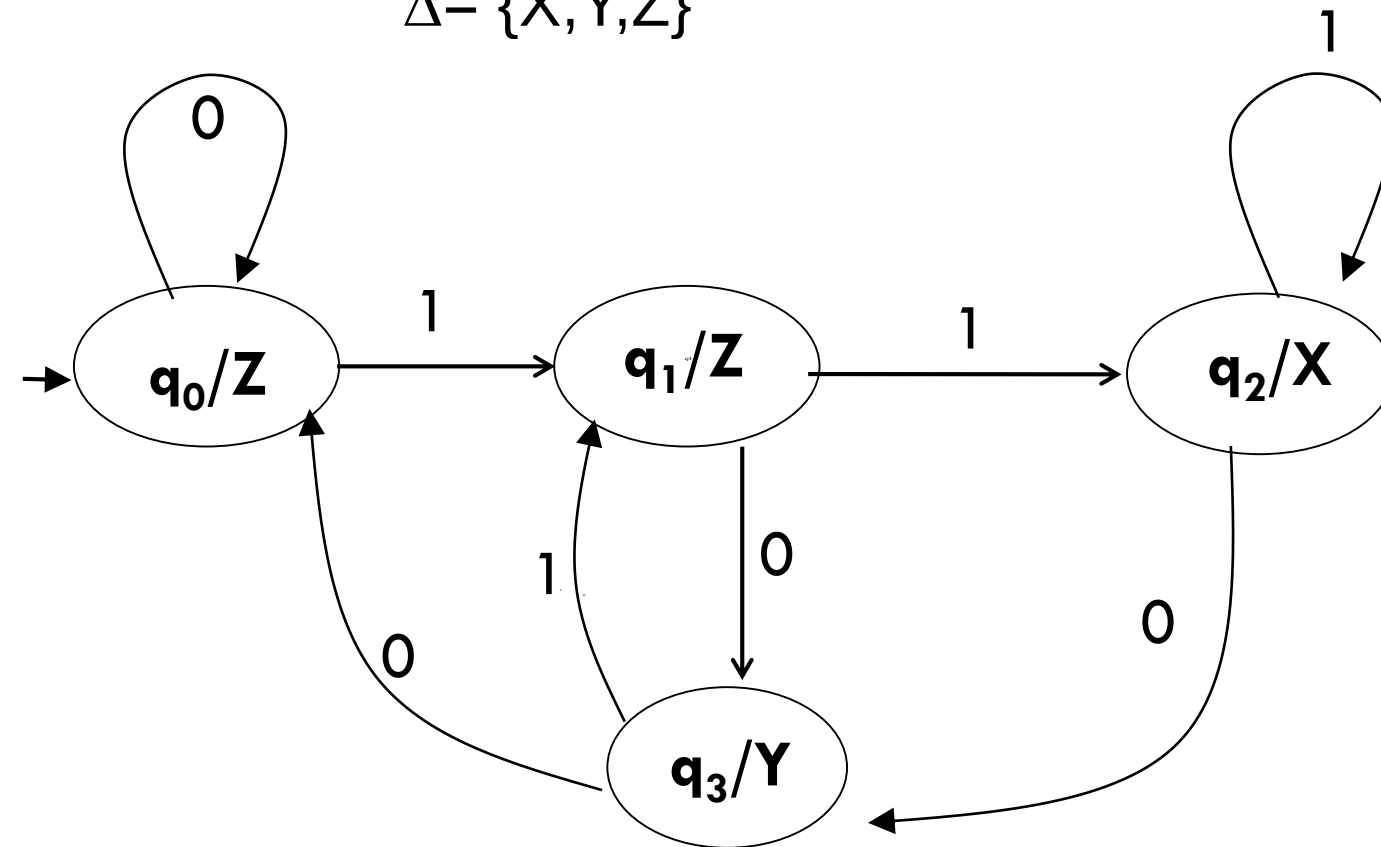
Solution: $\Sigma = \{0,1\}$
 $\Delta = \{A,B,C\}$



Question4: Construct a Moore machine that takes set of all strings over $\{0,1\}$ and produce 'X' as output if input ends with '11' or produces 'Y' as output if ends with '10' otherwise 'Z'.

Solution:

$$\Sigma = \{0,1\}$$
$$\Delta = \{X,Y,Z\}$$



By: Sandeep Rathor

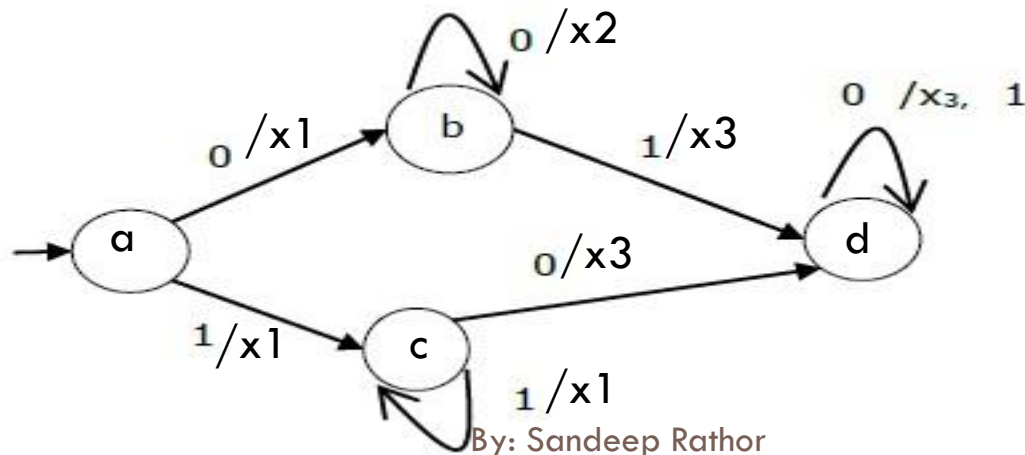
Mealy Machine

A Mealy machine can be described by a 6 tuple
i.e. $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$ where –

- Q is a finite set of states.
- Σ is a finite set of symbols called the input alphabet.
- Δ is a finite set of symbols called the output alphabet.
- δ is the input transition function where $\delta: Q \times \Sigma \rightarrow Q$
- λ is the output function where $\lambda : Q \times \Sigma \rightarrow \Delta$
- q_0 is the initial state ($q_0 \in Q$).

Example of Mealy Machine

Present state	Next state			
	input = 0		input = 1	
	State	Output	State	Output
→ a	b	x1	c	x1
b	b	x2	d	x3
c	d	x3	c	x1
d	d	x3	d	x2

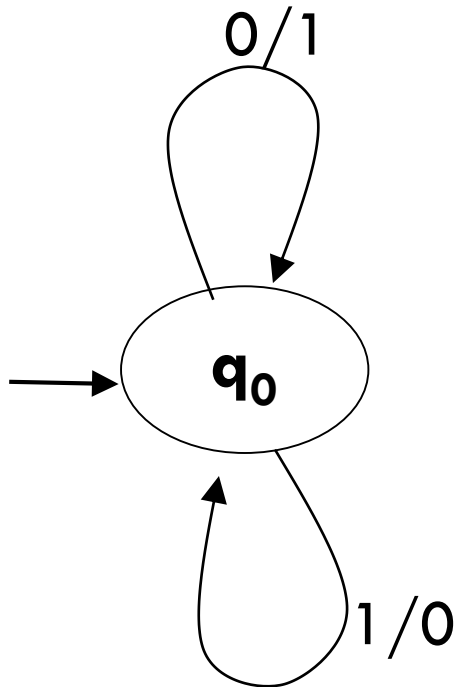


Construction of Mealy Machine

Question1: Construct a mealy machine that takes binary number as input and produces **1's complement** of that number as output. Assume that string is read LSB to MSB.

Solution:

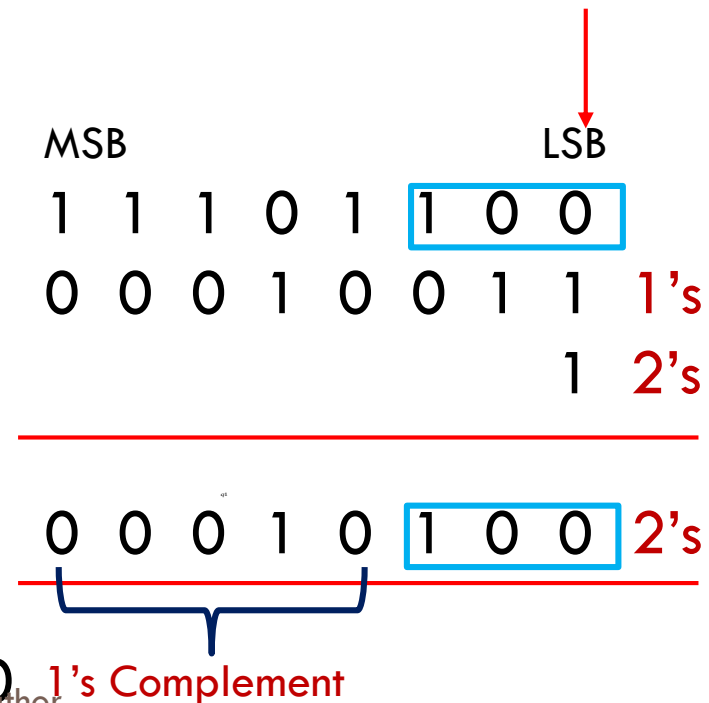
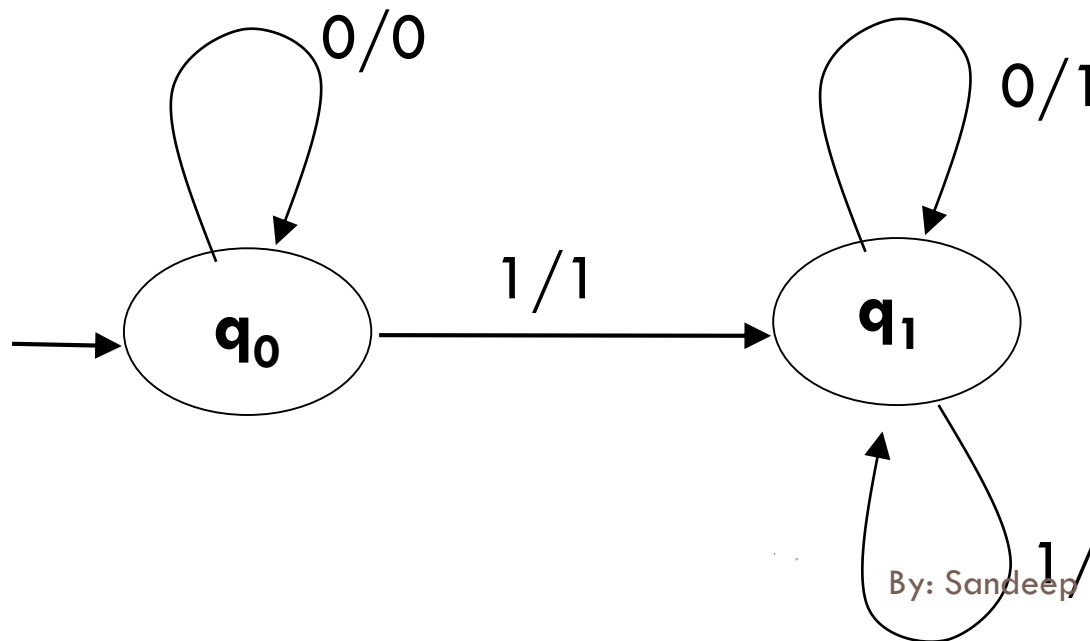
$$\Sigma = \{0,1\}$$
$$\Delta = \{0,1\}$$



Construction of Mealy Machine

Question 1: Construct a mealy machine that takes binary number as input and produces **2's complement** of that number as output. Assume that string is read LSB to MSB and end carry is discarded.

Solution: Hint

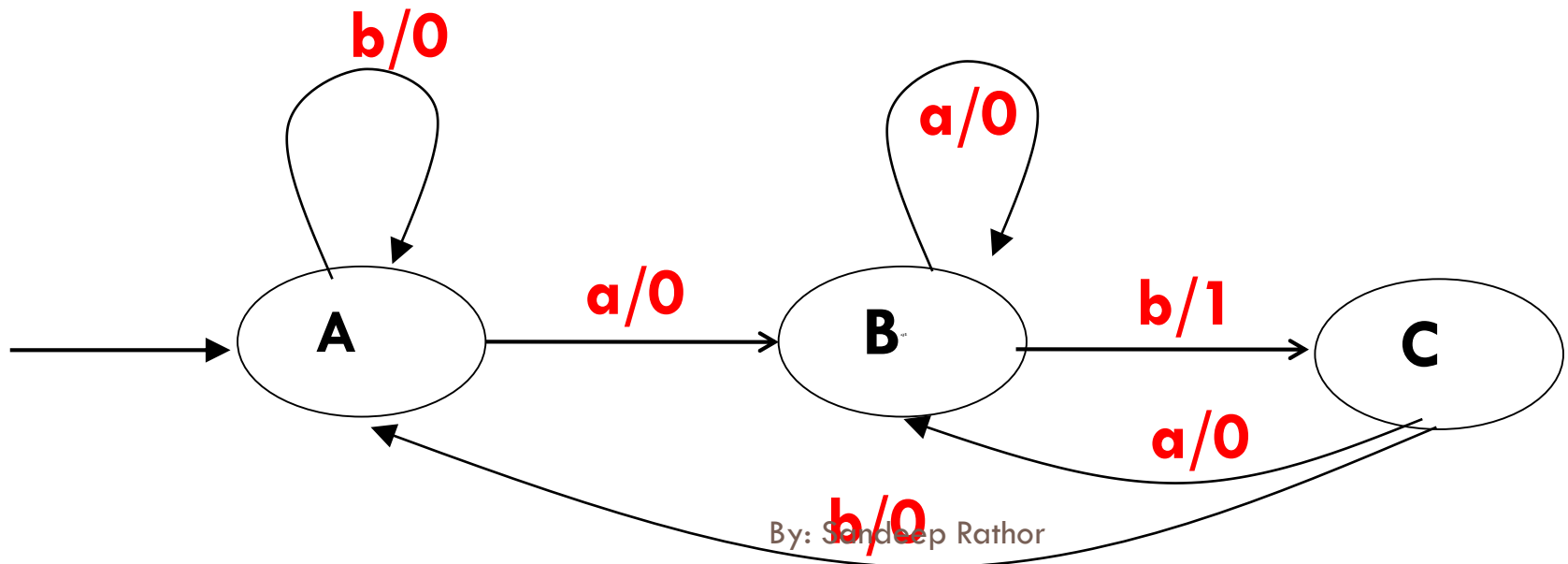
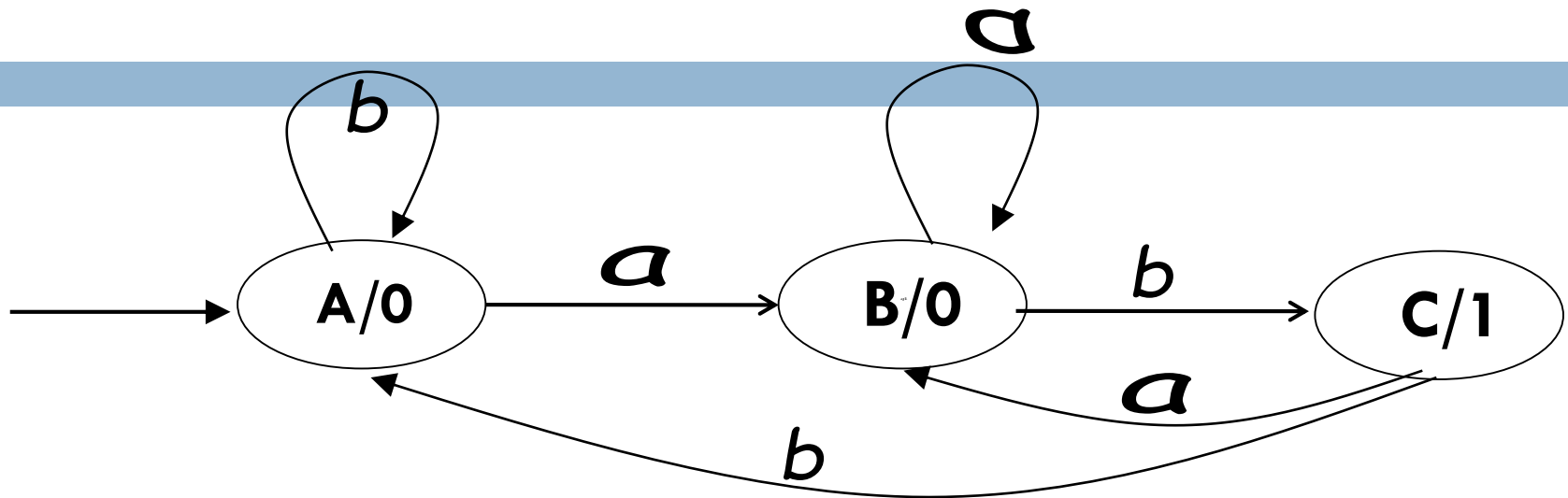


Difference b/w Moore & Mealy Machine

Moore Machine	Mealy Machine
Output depends only upon the present state.	Output depends both upon the present state and the present input
Generally, it has more states than Mealy Machine.	Generally, it has fewer states than Moore Machine.
The value of the output function is a function of the current state and the changes at the clock edges, whenever state changes occur.	The value of the output function is a function of the transitions and the changes, when the input logic on the present state is done.
In Moore machines, more logic is required to decode the outputs resulting in more circuit delays. They generally react one clock cycle later.	Mealy machines react faster to inputs. They generally react in the same clock cycle.

By: Sandeep Rathor

Convert given Moore M/C to Mealy



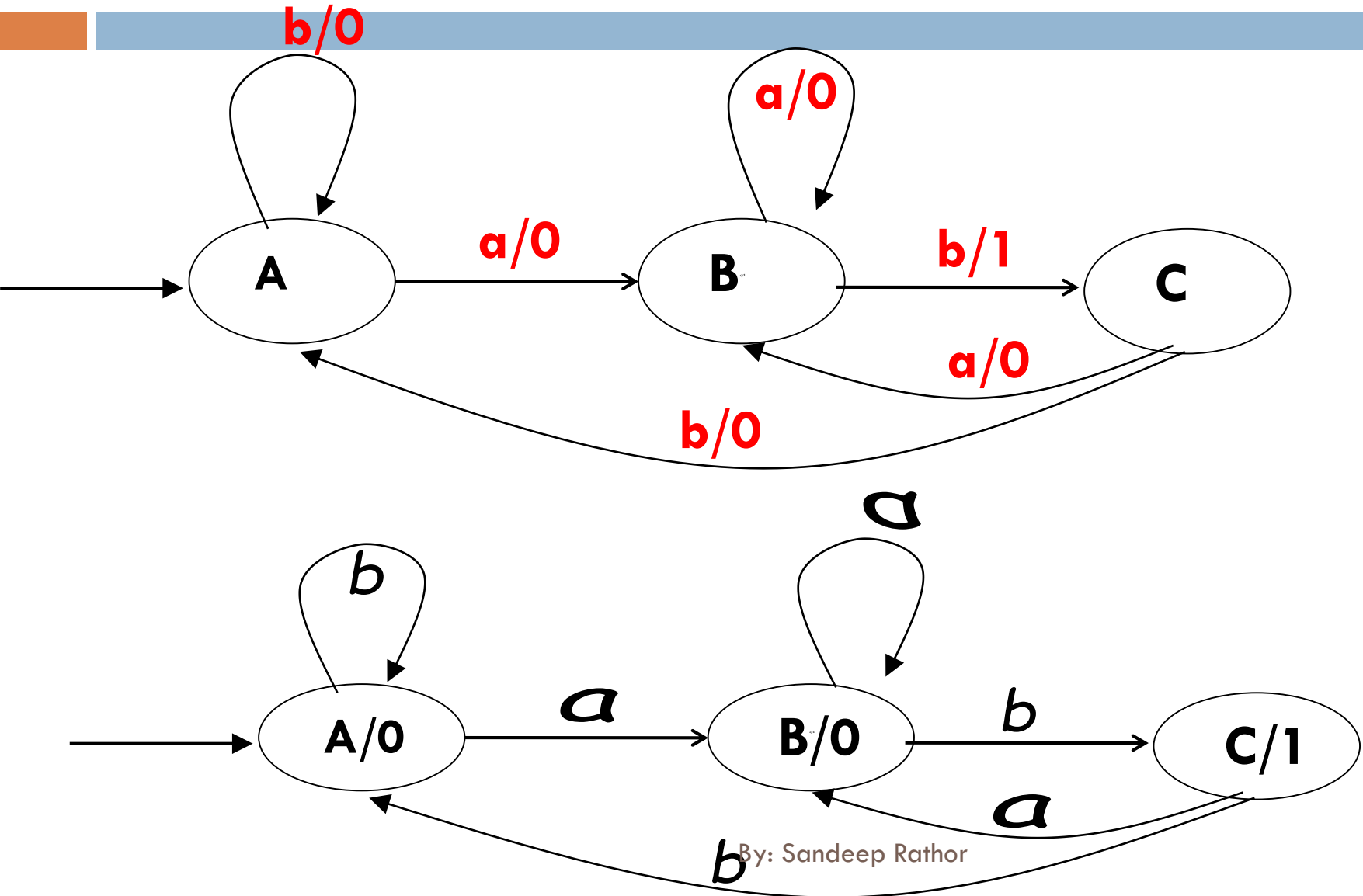
Convert given Moore M/C to Mealy

Present State	Next State		Output
	a = 0	a = 1	
→ a	d	b	1
b	a	d	0
c	c	c	0
d	b	a	1

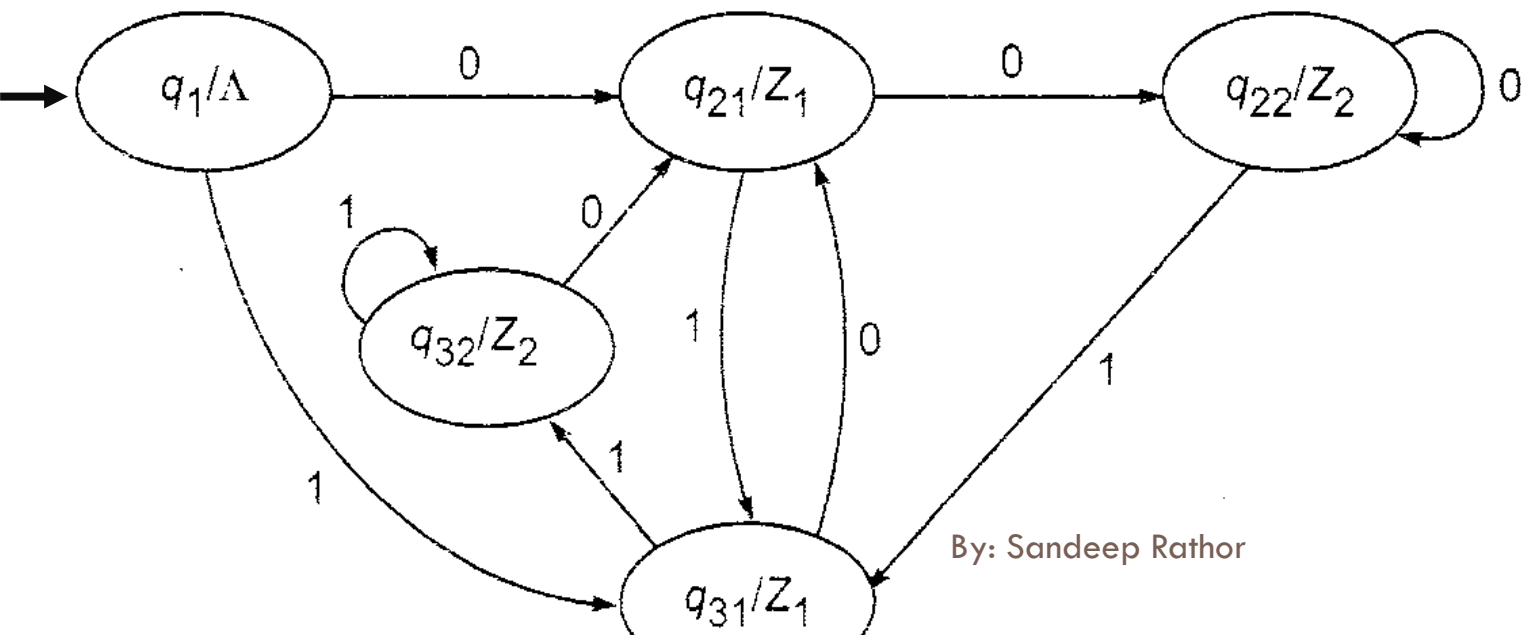
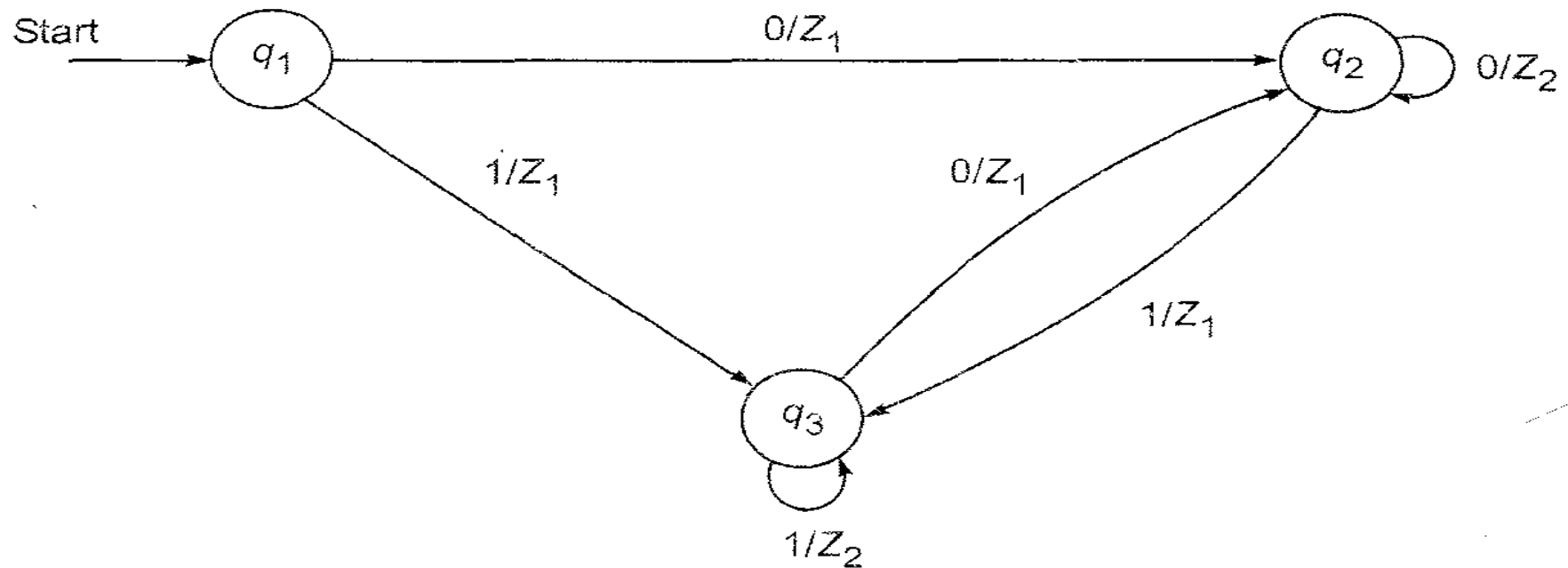
Present State	Next State			
	a = 0		a = 1	
	State	Output	State	Output
=> a	d	1	b	0
b	a	1	d	1
c	c	0	c	0
d	b	0	a	1

By: Sandeep Rathor

Convert given Mealy M/C to Moore



Convert given Mealy M/C to Moore



By: Sandeep Rathor

REGULAR EXPRESSION

By: Sandeep Rathor

Regular Expressions

- RE are used for representing certain sets of string in an algebraic form.
- It describe the language that is accepted by **Finite Automata**.
- The symbols that appear in RE are letters of alphabets Σ , symbol for null string ϵ , parenthesis, star operator and plus sign.

A Regular Expression can be recursively defined as:

- ϵ is a Regular Expression indicates the language containing an empty string. ($L(\epsilon) = \{\epsilon\}$)
- ϕ is a Regular Expression denoting an empty language. ($L(\phi) = \{ \}$)

Regular expressions: Rule

- Union of two RE, R_1 and R_2 , written as **$R_1 + R_2$** , is also a RE.
- Concatenation of two RE, R_1 and R_2 , written as **$R_1 R_2$** , is also a RE.
- Iteration of RE, R written as **R^*** , is also a RE.

Represent these sets by RE

$\{\epsilon, 0, 00, 000, 0000, \dots\}$

Ans: 0^*

$\{\epsilon, ab\}$

Ans: $\epsilon + ab$

$\{01, 10\}$

Ans: $01 + 10$

$\{\epsilon, 10, 01\}$

Ans: $\epsilon + 10 + 01$

Set of all strings ending in b.

Ans: $(a+b)^*b$

Set of all strings starting with a and ending with ba

Ans: $a(a+b)^*ba$

(a) The set of all strings over $\{0, 1\}$ with three consecutive 0's.

$(0+1)^*000(0+1)^*$

(b) The set of all strings over $\{0, 1\}$ beginning with 00.

$00(0+1)^*$

(c) The set of all strings over $\{0, 1\}$ ending with 00 and beginning with 1.

$1(0+1)^*00$

(d) all the string containing exactly two 0's.

$1^*01^*01^*$

(e)

- All strings containing an even number of 0's:

$$1^* + (1^*01^*0)^*1^*$$

- All strings having at least two occurrences of the substring 00:

$$(1 + 0)^* 00(1 + 0)^* 00(1 + 0)^* + (1 + 0)^* 000(1 + 0)^*$$

Find a regular expression corresponding to the language of strings of even lengths over the alphabet of $\{a, b\}$.

$$(aa + ab + ba + bb)^*$$

Find the Regular Expression of the following:

1. The set of all strings containing exactly 2a's:

$b^*ab^*ab^*$

2. The set of all strings containing at least 2a's:

$(a+b)^* a (a+b)^* a (a+b)^*$

3. The set of all strings containing at most 2a's:

$b^*+b^*ab^*+b^*ab^*ab^*$

4. The set of all strings containing substring aa:

$(a+b)^* aa (a+b)^*$

- Find a regular expression corresponding to the language of all strings over the alphabet $\{ a, b \}$ that do not end with ab .

$(a + b)^*(a + bb)$

- Find a regular expression corresponding to the language of all strings over the alphabet $\{ a, b \}$ that contain exactly two a 's.

□ $b^*ab^*ab^*$

Identities Related to Regular Expressions

Given R, P, L, Q as regular expressions, the following identities hold

$$\emptyset^* = \varepsilon$$

$$\varepsilon^* = \varepsilon$$

$$RR^* = R^*R$$

$$R^*R^* = R^*$$

$$(R^*)^* = R^*$$

$$(PQ)^*P = P(QP)^*$$

$$(a+b)^* = (a^*b^*)^* = (a^*+b^*)^* = (a+b^*)^* = a^*(ba^*)^*$$

$$R + \emptyset = \emptyset + R = R \text{ (The identity for union)}$$

$$R \varepsilon = \varepsilon R = R \text{ (The identity for concatenation)}$$

$$\emptyset L = L \emptyset = \emptyset \text{ (The annihilator for concatenation)}$$

$$R + R = R \text{ (Idempotent law)}$$

$$L(M + N) = LM + LN \text{ (Left distributive law)}$$

$$(M + N)L = ML + NL \text{ (Right distributive law)}$$

$$\varepsilon + RR^* = \varepsilon + R^*R = R^*$$

By: Sandeep Rathor



Arden's Theorem

By: Sandeep Rathor

Arden's Theorem

If P and Q are two regular expressions over Σ and if P does not contain ε , then the following equation in R given by $R = Q + RP$ has a unique solution i.e., $R = QP^*$.

The Arden's Theorem is useful for checking the equivalence of two regular expressions as well as in the conversion of DFA to a regular expression.

Proof

Part I: Prove that $R = QP^*$ is the solution of this equation

$$R = Q + RP$$

Replace R by QP^* on both sides

$$\text{LHS} = QP^*$$

$$\text{RHS} = Q + QP^*P$$

$$= Q(\varepsilon + P^*P)$$

$$= QP^* \quad // \text{ (As we know that } \varepsilon + A^*A = A^*)$$

$$= \text{LHS}$$

Thus, $R = QP^*$ is the solution of the equation $R = Q + RP$.

Part II : Prove that this is the only solution of this equation.

$$R = Q + RP$$

Replace R by $Q + RP$ on RHS

$$\begin{aligned} R &= Q + (Q + RP) P \\ &= Q + QP + RP^2 \end{aligned}$$

Keep replacing R by $Q + RP$

$$\begin{aligned} R &= Q + QP + (Q + RP) P^2 \\ &= Q + QP + QP^2 + RP^3 \end{aligned}$$

...

$$= Q + QP + QP^2 + QP^3 + \dots + QP^i + RP^{i+1}$$

By: Sandeep Rathor


$$R = Q + QP + QP^2 + QP^3 + \dots + QP^i + RP^{i+1}$$

$$= Q (\epsilon + P + P^2 + \dots + P^i) + RP^{i+1} \quad \text{for } i \geq 0$$

We claim that any soln of $R = Q + RP$ must be equivalent to QP^*

Let $w \in R$ and $|w| = i$.

then w belongs to set $Q (\epsilon + P + P^2 + \dots + P^i) + RP^{i+1}$,

as P does not contain null. RP^{i+1} has no string of length less $i+1$ so,

w is not in set RP^{i+1}

It means w belongs to the set $Q (\epsilon + P + P^2 + \dots + P^i)$

and hence it is equivalent to QP^*



Arden's Theorem

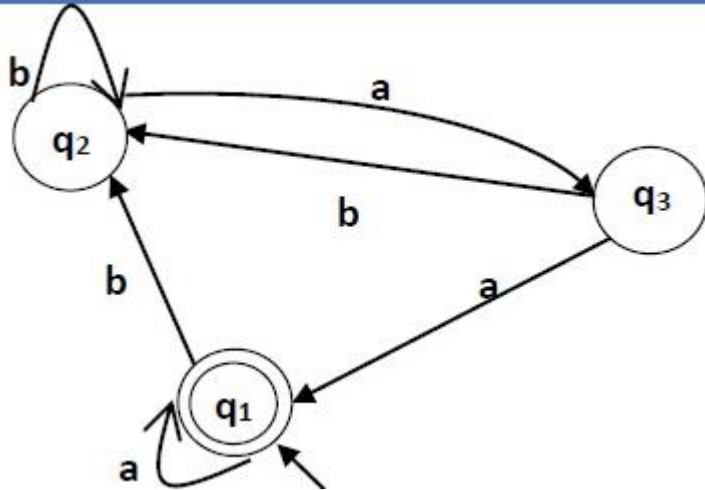
DFA to RE

By: Sandeep Rathor

Assumptions for Applying Arden's Theorem

- The transition diagram must not have NULL transitions
- It must have only one initial state

Find the RE



Step 1: Construct the equations

$$q_1 = q_1a + q_3a + \varepsilon$$

$$q_2 = q_1b + q_2b + q_3b$$

$$q_3 = q_2a$$

Step 2: Solve the equations

$$(a + b(b + ab)^*aa)^*$$

$$q_1 = q_1a + q_3a + \varepsilon$$

$$q_2 = q_1b + q_2b + q_3b$$

$$q_3 = q_2a$$

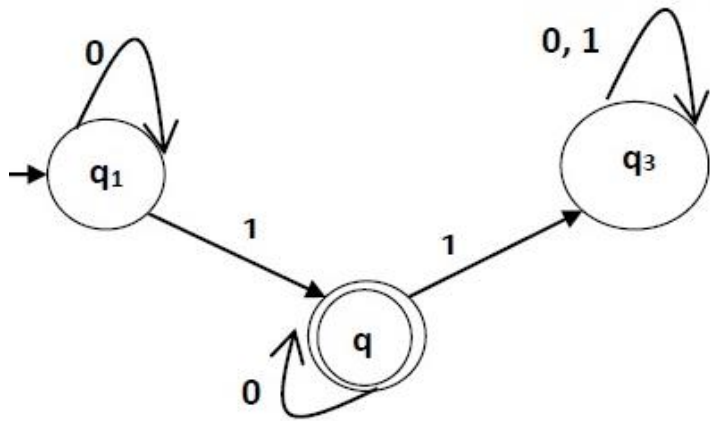
Now, we will solve these three equations –

$$\begin{aligned} q_2 &= q_1b + q_2b + q_3b \\ &= q_1b + q_2b + (q_2a)b \text{ (Substituting value of } q_3) \\ &= q_1b + q_2(b + ab) \\ &= q_1b (b + ab)^* \text{ (Applying Arden's Theorem)} \end{aligned}$$

$$\begin{aligned} q_1 &= q_1a + q_3a + \varepsilon \\ &= q_1a + q_2aa + \varepsilon \text{ (Substituting value of } q_3) \\ &= q_1a + q_1b(b + ab^*)aa + \varepsilon \text{ (Substituting value of } q_2) \\ &= q_1(a + b(b + ab)^*aa) + \varepsilon \\ &= \varepsilon (a + b(b + ab)^*aa)^* \\ &= (a + b(b + ab)^*aa)^* \end{aligned}$$

Hence, the regular expression is **$(a + b(b + ab)^*aa)^*$** .

Find the RE



Step 1: Construct the equations

$$q_1 = q_1 0 + \varepsilon$$

$$q_2 = q_1 1 + q_2 0$$

$$q_3 = q_2 1 + q_3 (0 + 1)$$

Step 2: Solve the equations

0^*10^*

Solution –

Here the initial state is q_1 and the final state is q_2

Now we write down the equations –

$$q_1 = q_1 0 + \epsilon$$

$$q_2 = q_1 1 + q_2 0$$

$$q_3 = q_2 1 + q_3 0 + q_3 1$$

Now, we will solve these three equations –

$$q_1 = \epsilon 0^* [As, \epsilon R = R]$$

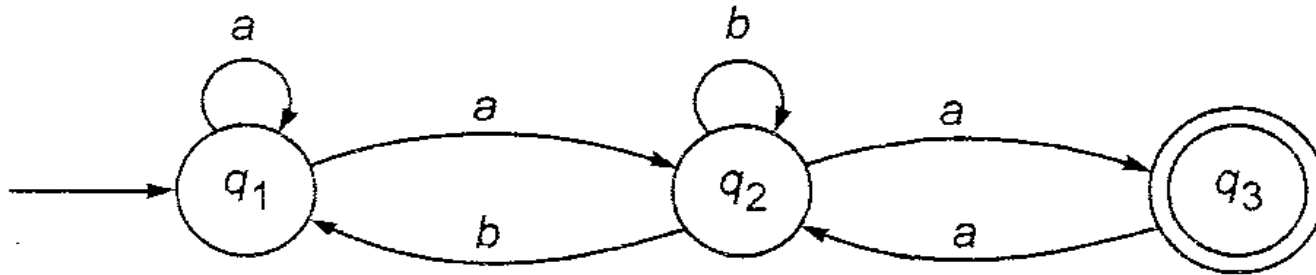
$$So, q_1 = 0^*$$

$$q_2 = 0^* 1 + q_2 0$$

$$So, q_2 = 0^* 1 (0)^* [By Arden's theorem]$$

Hence, the regular expression is $0^* 1 0^*$.

For Practice



$$q1 = q_1 a + q_2 b + \varepsilon$$

$$q2 = q_1 a + q_2 b + q_3 a$$

$$q3 = q_2 a$$

$$q2 = q_1 a + q_2 b + q_2 a a$$

$$= q_1 a + q_2 (b + a a)$$

$$= q_1 a (b + a a)^*$$

$$q_1 = q_1 a + q_1 a (b + a a)^* b + \varepsilon$$

$$= q_1 (a + a (b + a a)^* b) + \varepsilon$$

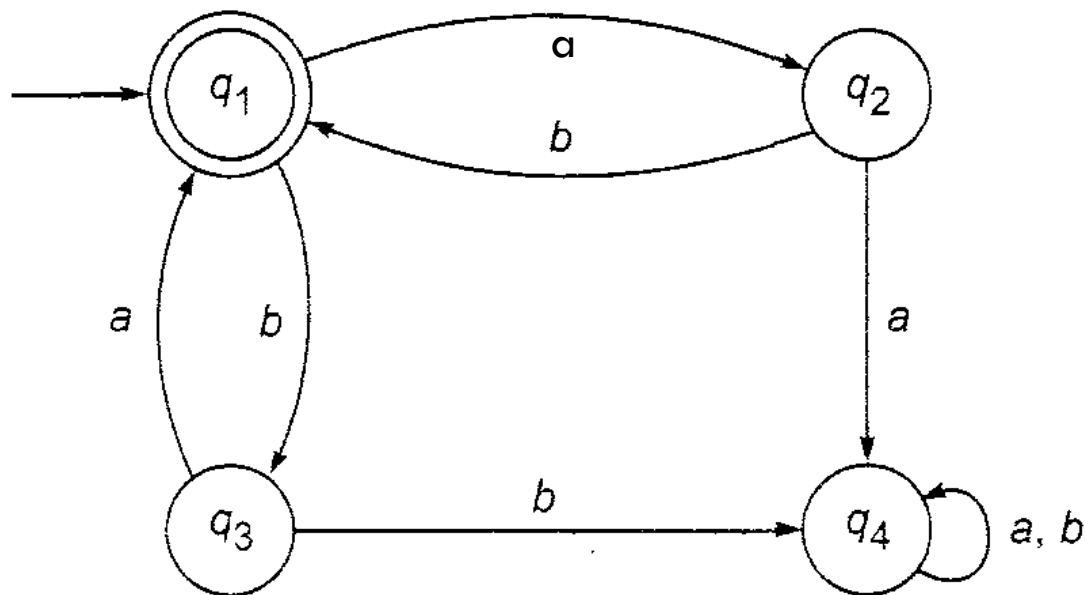
$$q_1 = \varepsilon (a + a (b + a a)^* b)^*$$

$$q2 = (a + a (b + a a)^* b)^* a (b + a a)^*$$

$$q3 = (a + a (b + a a)^* b)^* a (b + a a)^* a$$

Q3 is the final state. So RE

$$(a + a (b + a a)^* b)^* a (b + a a)^* a$$



$$\mathbf{q_1 = q_2b + q_3a + \Lambda}$$

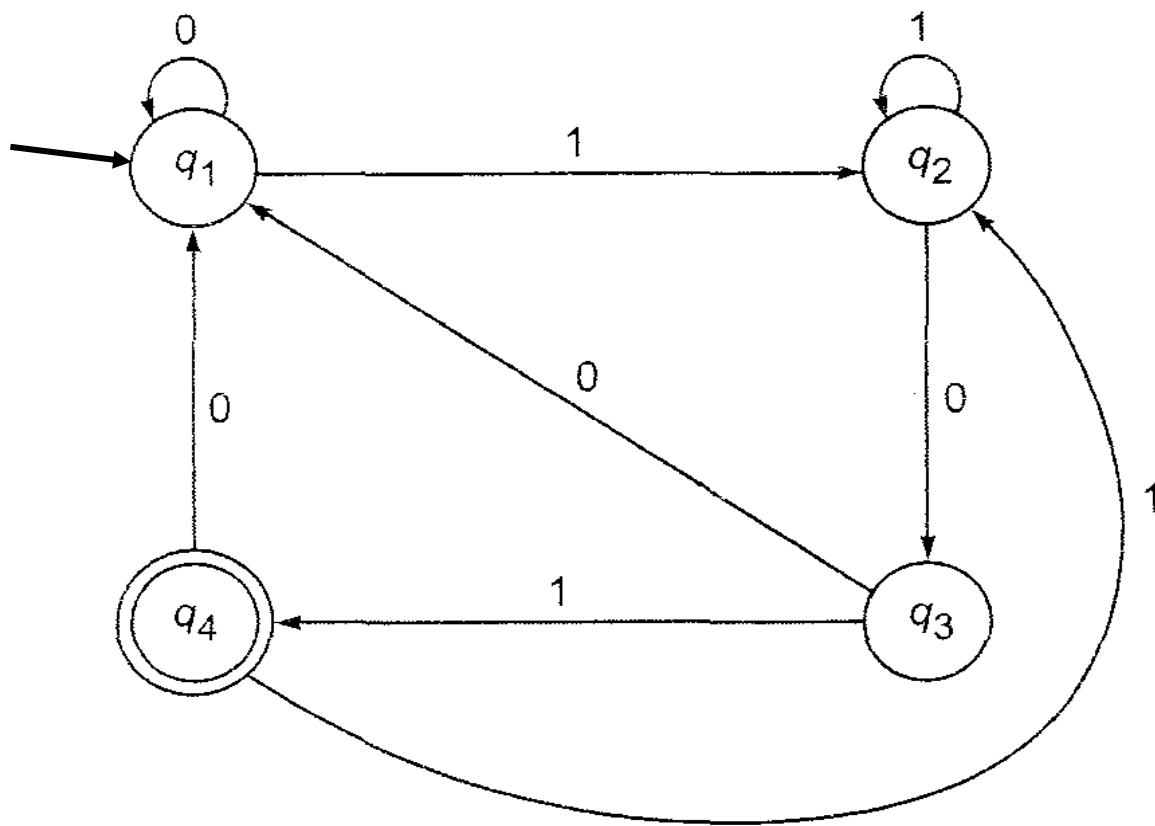
$$\mathbf{q_2 = q_1a}$$

$$\mathbf{q_3 = q_1b}$$

$$\mathbf{q_4 = q_2a + q_3b + q_4a + q_4b}$$

$$\mathbf{q_1 = q_1ab + q_1ba + \Lambda = q_1(ab + ba) + \Lambda}$$

$$\mathbf{q_1 = \Lambda(ab + ba)^* = (ab + ba)^*}$$



$$q_1 = q_1 0 + q_3 0 + q_4 0 + \Lambda$$

$$q_2 = q_1 1 + q_2 1 + q_4 1$$

$$q_3 = q_2 0$$

$$q_4 = q_3 1$$

$$q_4 = q_3 1 = (q_2 0) 1 = q_2 0 1$$

$$q_2 = q_1 1 + q_2 1 + q_2 011 = q_1 1 + q_2 (1 + 011)$$

$$q_2 = (q_1 1)(1 + 011)^* = q_1 (1(1 + 011)^*)$$

$$q_1 = q_1 0 + q_2 00 + q_2 010 + \Lambda$$

$$= q_1 0 + q_2 (00 + 010) + \Lambda$$

$$= q_1 0 + q_1 1(1 + 011)^* (00 + 010) + \Lambda$$

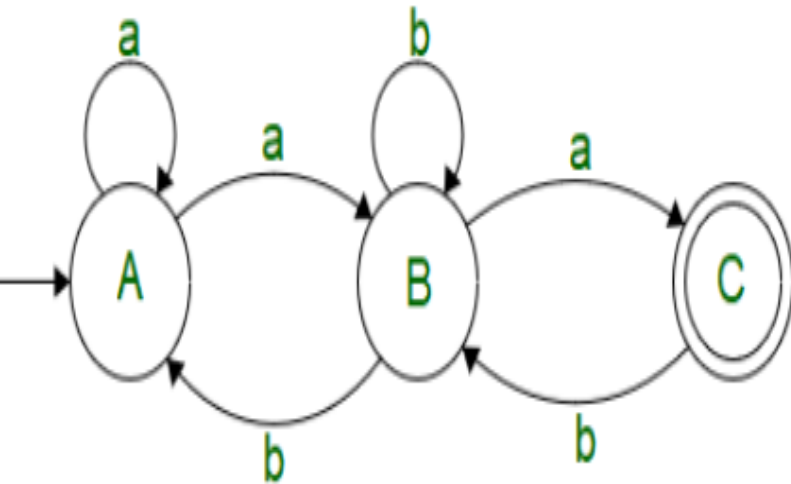
$$q_1 = \Lambda(0 + 1(1 + 011)^* (00 + 010))^*$$

Using Arden's theorem

$$q_4 = q_2 01 = q_1 1(1 + 011)^* 01$$

$$= (0 + 1(1 + 011)^* (00 + 010))^* (1(1 + 011)^* 01)$$

Find the RE



Step 1: Construct the equations

$$A = Aa + Bb + \varepsilon$$

$$B = Aa + Bb + Cb$$

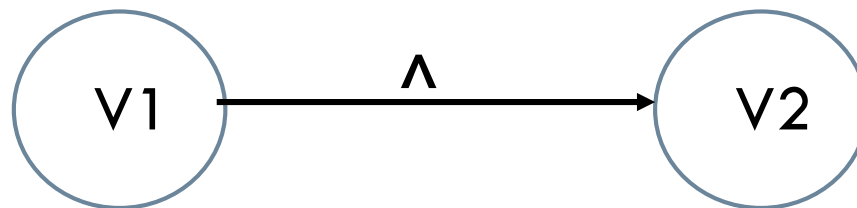
$$C = Ba$$

Step 2: Solve the equations

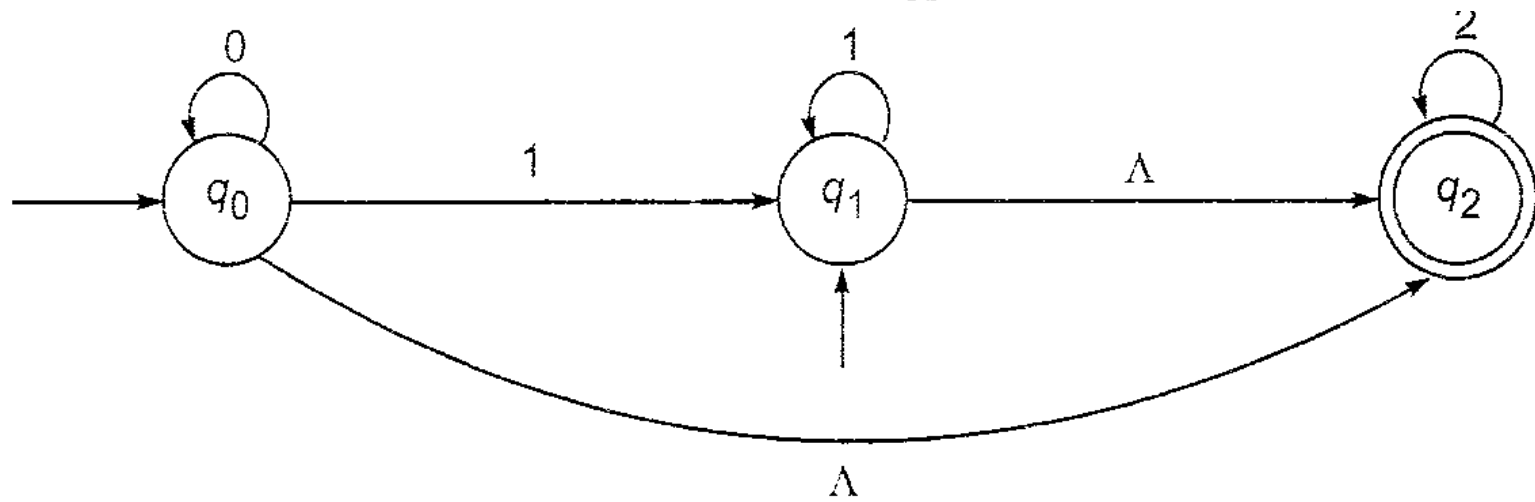
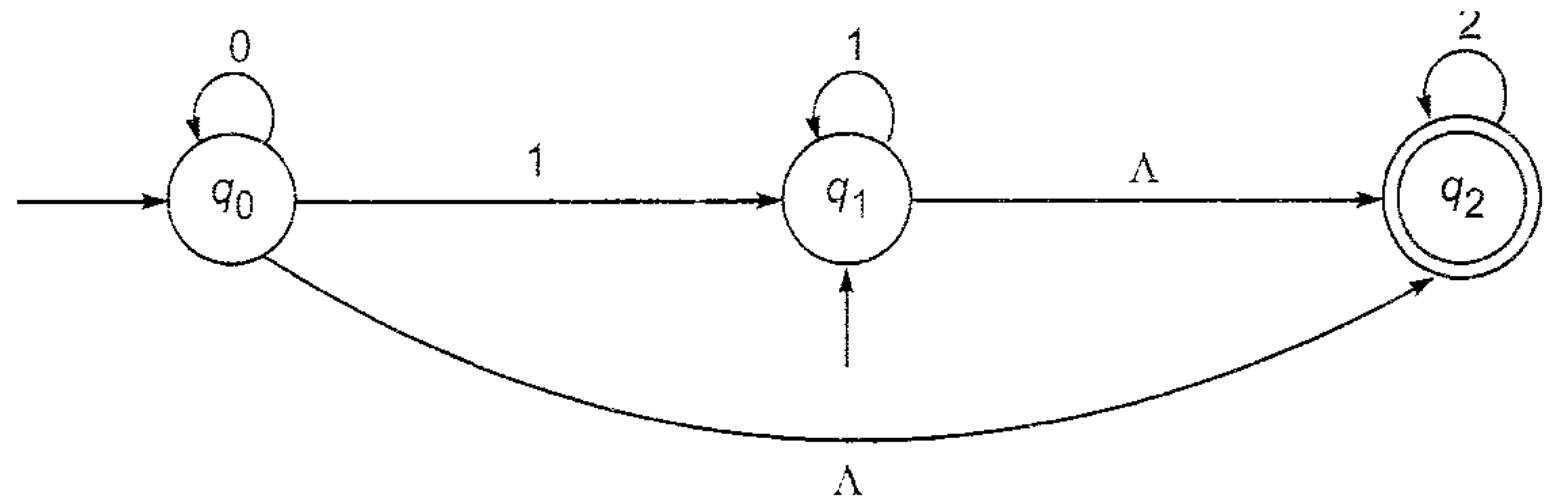
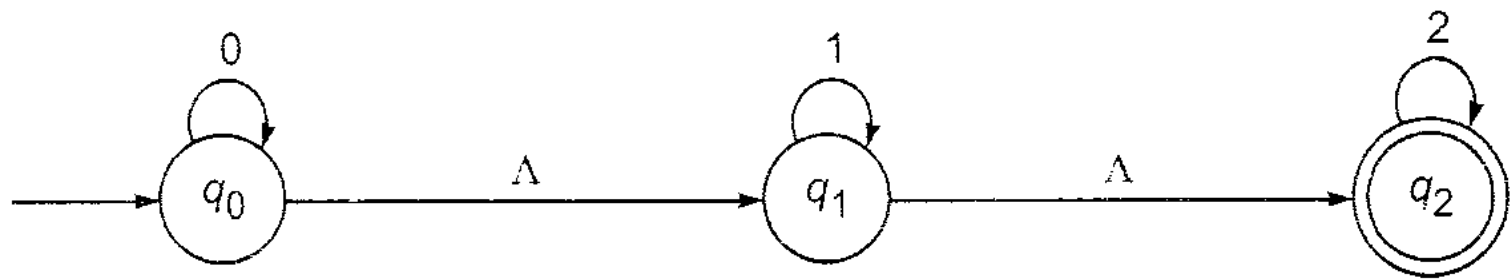
$$(a + a(b + ab)^*b)^* a (b + ab)^* a$$

Elimination of Null Moves

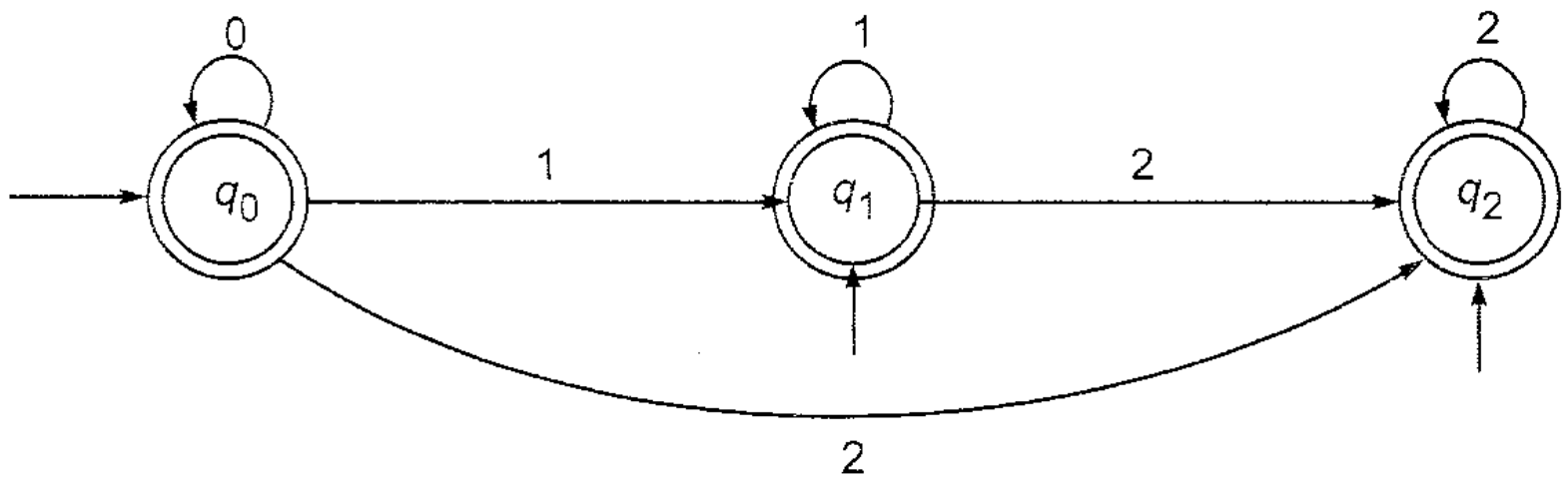
- **Step 1** Find all the edges starting from $V2$.
- **Step 2** Duplicate all these edges starting from $V1$ without changing the edge labels.
- **Step 3** If $V1$ is an initial state, make $V2$ also as initial state.
- **Step 4** If $V2$ is a final state. make $V1$ also as the final state.



By: Sandeep Rathor



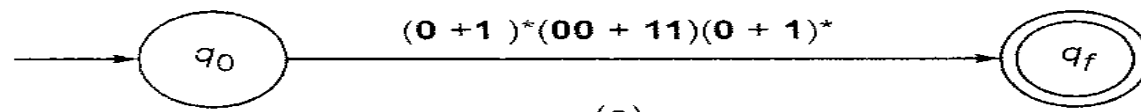
By: Sandeep Rathor



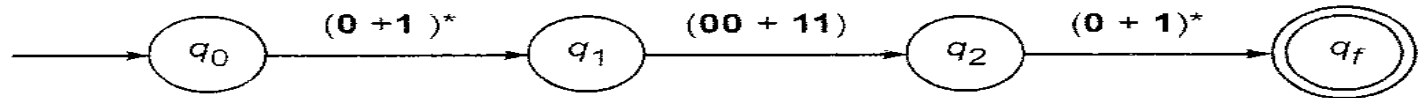
By: Sandeep Rathor

RE to DFA

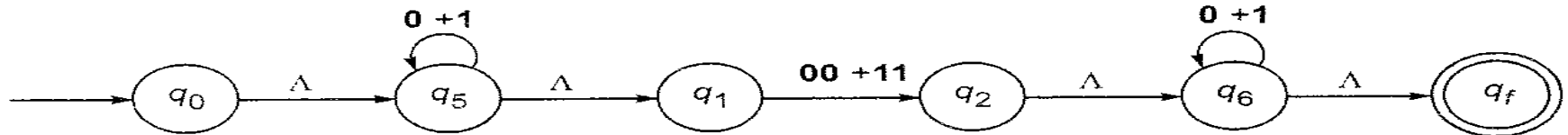
$(0 + 1)^*(00 + 11)(0 + 1)^*$



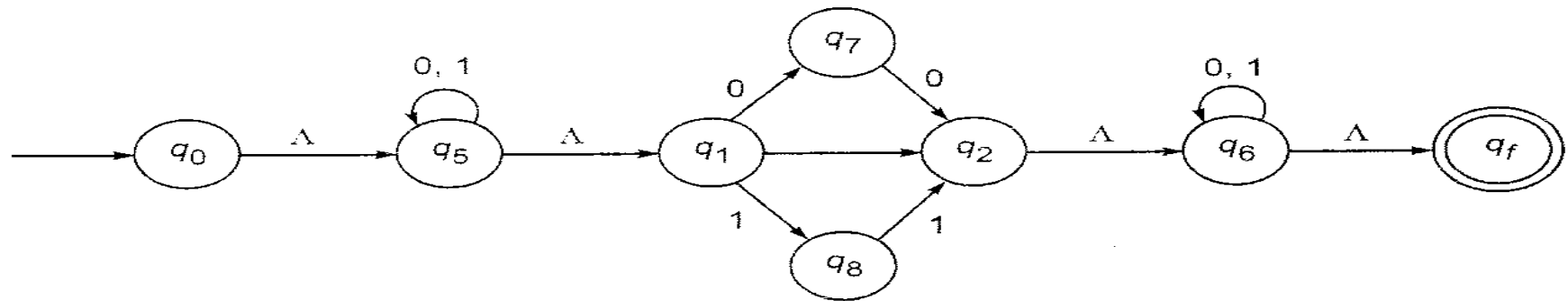
(a)



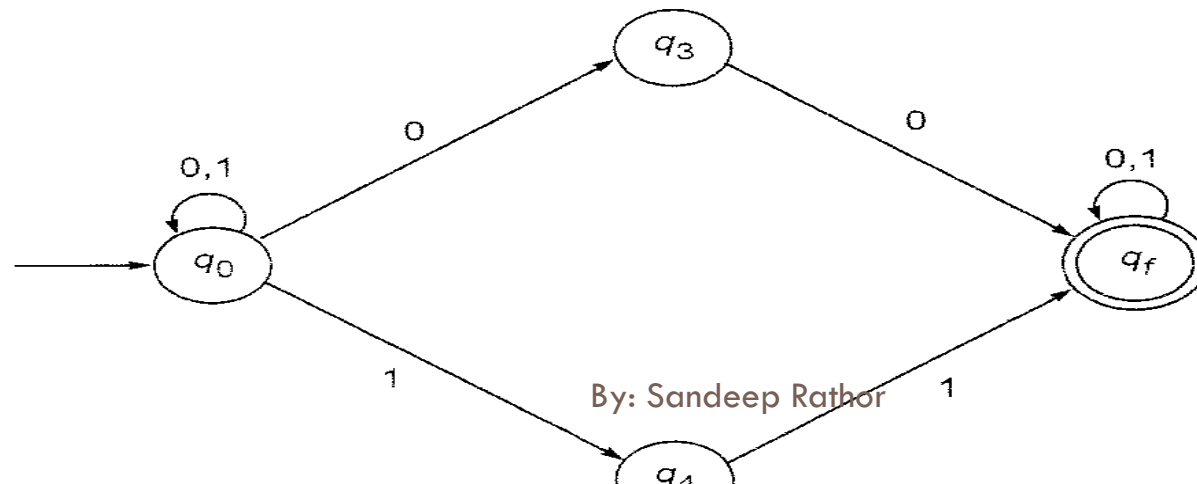
(b)



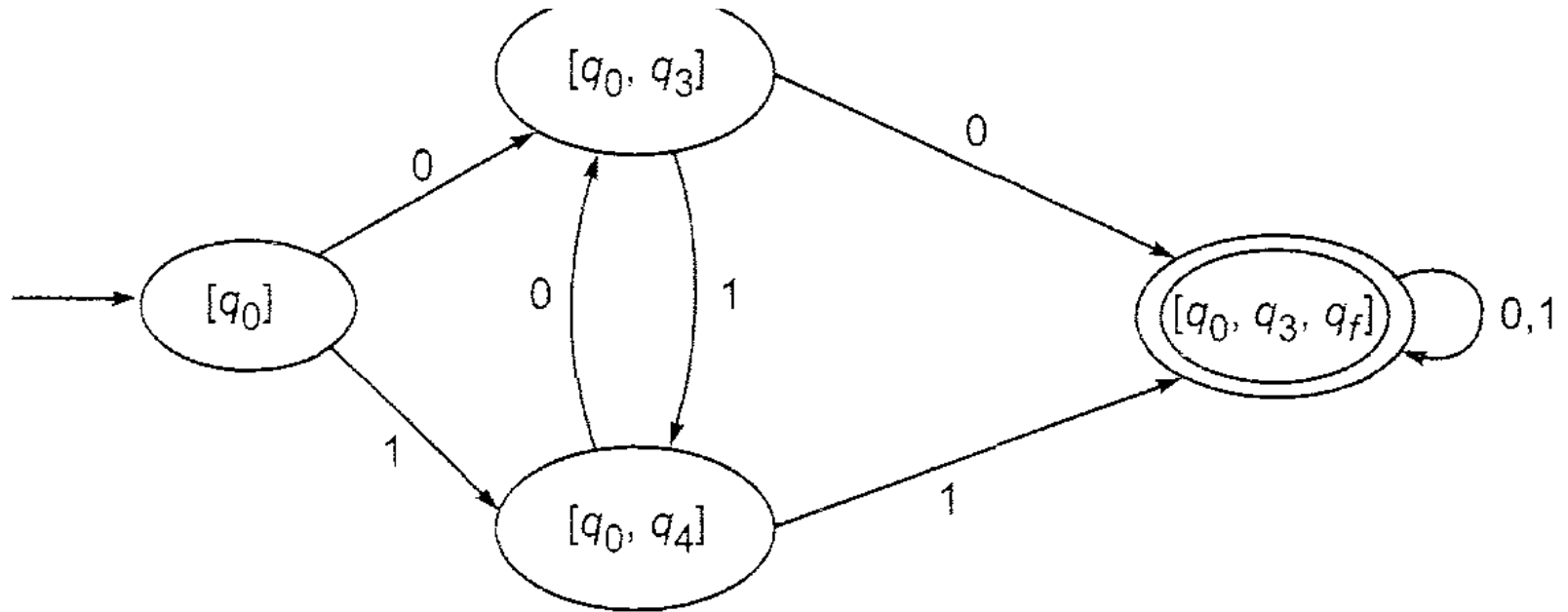
(c)



(d)



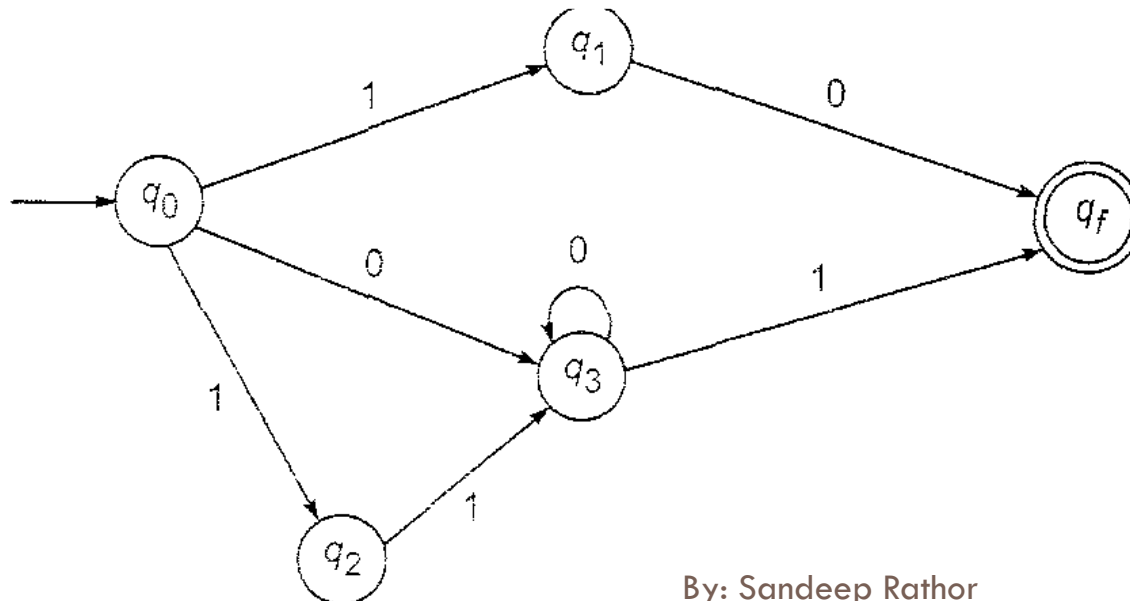
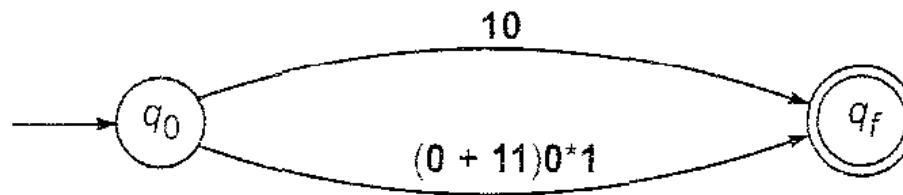
By: Sandeep Rathor



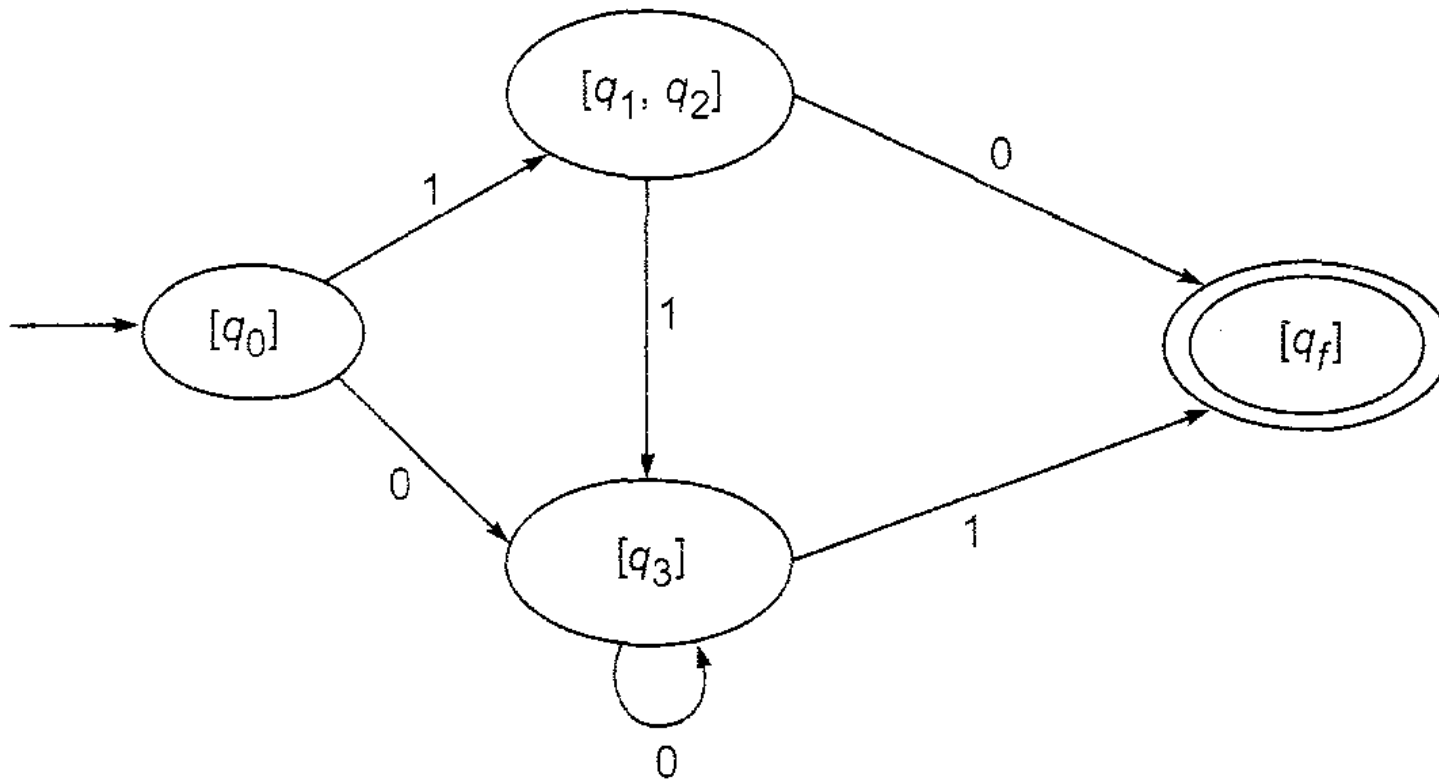
DFA corresponding to given RE

Find DFA from given RE

$10 + (0 + 11)0^*1$



By: Sandeep Rathor



Final DFA as per the given RE

Pumping lemma for Regular Sets

Statement:

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton with n states. Let L be the regular set accepted by M . Let $w \in L$ and $|w| \geq m$. If $m \geq n$, then there exists x, y, z such that $w = xyz$, $|y| \geq 1$, $x \in A$ and $xy^iz \in L$ for each $i \geq 0$.

***Pumping** Repeating a section of the string an arbitrary number of times (≥ 0), with the resulting string remaining in the language.

OR

If A is a Regular Language, then there is a number p (the pumping length) where if s is any string in A of length at least p , then s may be divided into 3 pieces, $s = xyz$, satisfying the following conditions:

- a. For each $i \geq 0$, $xy^iz \in A$,
- b. $|y| > 0$, and
- c. $|xy| \leq p$.

- Let $M = (Q, \Sigma, \delta, q_1, F)$ be a DFA recognizing A and p be the number of states of M . Let $s = s_1 s_2 \dots s_n$ be a string in A with length n , where $n \geq p$. Let $r_1, \dots, r_{n+1}$ be the sequence of states M enters when processing s . $r_{i+1} = \delta(r_i, s_i)$ for $1 \leq i \leq n$. The sequence has length $n+1$, which is at least $p+1$.
- Among the first $p+1$ elements in the sequence, two must be the same state, via the pigeonhole principle. The first is called r_j , and the second is r_l . Because r_l occurs among the first $p+1$ places in a sequence starting at r_1 , we have $l \leq p+1$.

□

Now let $x = s_1 \dots s_{i-1}$, $y = s_i \dots s_{l-1}$, and $z = s_l \dots s_n$.
As x takes M from r_1 to r_i , y takes M from r_i to r_l ,
and z takes M from r_l to r_{n+1} , which is an accept
state, M must accept xy^iz for $i \geq 0$.

We know $i \neq l$, so $|y| > 0$; and $l \leq p + 1$, so
 $|xy| \leq p$.

Thus, we have satisfied all conditions of the
pumping lemma.

- $B = \{0^n 1^n \mid n \geq 0\}$ Is this Language a Regular Language?
- Assume B is Regular, then Pumping Lemma must hold. p is the pumping length given by the PL.
- Choose s to be $0^p 1^p$. Because $s \in B$ and $|s| \geq p$, PL guarantees s can be split into 3 pieces, $s = xyz$, where for any $i \geq 0$, $xy^i z \in B$. Consider 3 cases:
 - 1. y is only 0s. $xyyz$ has more 0s than 1s, thus a contradiction via condition 1 of PL.
 - 2. y is only 1s. Also a contradiction.
 - 3. y is both 0s and 1s. $xyyz$ may have same number of 1s and 0s, but will be out of order, with some 1s before 0s, also a contradiction. Contradiction is unavoidable, thus B is not Regular.

Prove that the language $\{a^k b^k \mid k \geq 0\}$ is not regular.

Prove that $L = \{a^k b^k \mid k \geq 0\}$ is not regular.

Step 1:

Suppose L is regular & is accepted by a FA having n states.

Step 2:

- ♦ Let $w = a^k b^k$ where $k > n$
- ♦ $w \in L$
- ♦ We can write $w = xyz$ where
 - $|xy| \leq n$
 - $|y| > 0$
- ♦ Since $k > n$, we have
 - $x = a^p$
 - $y = a^q$
 - $z = a^r b^n$
 - where $p + q + r = n$ & $q \neq 0$

Step 3:

- ♦ Let $i = 2$
- ♦ $w' = xy^i z$ where
 - $w' = a^p a^{2q} a^r b^n$
 - $w' = a^{p+2q+r} b^n$
 - $w' = a^{n+q} b^n$
 - Since $q \neq 0$, w' has more a 's than b 's
- ♦ Hence $w' \notin L$
- ♦ This is a contradiction, so L is not Regular